



SAPIENZA
UNIVERSITÀ DI ROMA

Sistemi Operativi 1

UNIVERSITÀ DEGLI STUDI DI ROMA "LA
SAPIENZA"

FACOLTÀ DI INGEGNERIA DELL'INFORMAZIONE,
INFORMATICA E STATISTICA

CORSO DI LAUREA TRIENNALE IN INFORMATICA

Autore
LORENZO SABATINO

Basato sulle lezioni di
FABIO DE GASPARI

25 settembre 2024



Prefazione

Salve a tutti!

Mi chiamo Lorenzo Sabatino e sono uno studente del corso di Informatica presso l'Università La Sapienza di Roma. Durante il mio percorso accademico, ho affrontato numerosi esami trascrivendo gli appunti presi a lezione e trasformandoli in veri e propri libri di studio.

Ho voluto condividere questo materiale con voi perché credo che possa essere un valido aiuto per tutti gli studenti di Informatica, sia quelli alle prime armi che chi è già avanti nel percorso di studi. Ho scritto e formattato questi testi con cura, utilizzando il linguaggio di markup $\text{\LaTeX}$ per garantire chiarezza e leggibilità. Ho anche arricchito il contenuto con diagrammi e schemi e ho integrato spunti tratti da lezioni online e da libri di testo.

Tuttavia, è importante sottolineare che questi libri non sostituiscono le lezioni frontali e non sono privi di errori. La mia intenzione è stata quella di fornire un supporto aggiuntivo allo studio, ma è indispensabile partecipare alle lezioni e approfondire i concetti con la guida dei docenti. Inoltre, tenete presente che, data la natura in continua evoluzione dell'informatica, alcune informazioni potrebbero diventare obsolete col passare del tempo.

Nonostante ciò, ho deciso di condividere questi libri perché credo che possano essere d'aiuto a molti studenti e spero che li troviate utili per il vostro percorso accademico.

Se desiderate contattarmi per qualsiasi domanda, commento o correzione, potete scrivere all'indirizzo email scrittinformatica@gmail.com. Sarò lieto di rispondere alle vostre richieste.

Grazie per aver scelto di leggere questi libri e in bocca al lupo per i vostri studi!

Questo testo è stato integrato con il libro [1].

Indice

Prefazione	1
1 Introduzione al corso	5
1.1 Componenti di un computer mono-processore	5
1.2 Interruzioni	6
1.3 Miglioramento degli accessi	8
1.4 Gerarchia della memoria	8
1.5 Storia dei sistemi operativi	8
2 Gestione dei processi	11
2.1 Stati di un processo	11
2.2 Descrizione di un processo	13
2.3 Controllo di un processo	14
2.4 Esecuzione del sistema operativo	15
2.5 Gestione dei processi UNIX SVR4	15
3 Thread	17
3.1 Processi e thread	17
3.2 Tipi di thread	18
3.3 Processi e thread in Linux	18
4 Scheduling	21
4.1 Tipi di scheduling	21
4.2 Algoritmi di scheduling	22
4.3 Scheduling tradizionale di UNIX	26
4.4 Scheduling su architetture multiprocessore	27
4.5 Scheduling in Linux	27
5 Gestione della memoria	29
5.1 Requisiti per la gestione della memoria	29
5.2 Partizionamento della memoria	31
5.3 Paginazione e segmentazione	33
5.4 Memoria virtuale: hardware e strutture di controllo	34
5.5 Memoria virtuale e sistema operativo	40
5.6 Gestione della memoria in Linux	42
6 Gestione dell'Input/Output	43
6.1 Dispositivi di I/O	43
6.2 Organizzazione della funzione di I/O	44
6.3 Problemi nel progetto del sistema operativo	45
6.4 Buffering dell'I/O	45
6.5 Scheduling del disco	47
6.6 Cache del disco	49
6.7 RAID	50
6.8 I/O in Linux	52

7	File System	55
7.1	Directory	55
7.2	Gestione della memoria secondaria	55
7.3	Gestione dei file in UNIX	58
7.4	Gestione dei file su Windows	59
8	Gestione della concorrenza	61
8.1	Mutua esclusione: supporto hardware	62
8.2	Semafori	62
8.3	Mutua esclusione: soluzioni software	68
8.4	Passaggio di messaggi	70
8.5	Problema dei lettori/scrittori	71
8.6	Equivalenze	73
9	Deadlock	75
9.1	Risorse	75
9.2	Prevenzione del deadlock	77
9.3	Evitare il deadlock	77
9.4	Rilevare il deadlock	80
9.5	Deadlock e Linux	81
10	Sicurezza nei sistemi operativi	83
10.1	Autenticazione	83
10.2	Controlli di accesso	84
10.3	Protezione in UNIX	84
10.4	Password	85
10.5	Buffer overflow	87
	Bibliografia	89

Capitolo 1

Introduzione al corso

Come avrai notato il titolo di questo libro è "Sistemi Operativi 1", questo perché il corso è diviso in due moduli. In particolare in questo corso verranno affrontati i seguenti argomenti: gestione dei processi; scheduling dei processi; gestione della memoria principale; gestione della concorrenza tra processi; gestione del deadlock; gestione dell'input/output; gestione dei file system; nozioni basilari di sicurezza.

Possiamo immaginare i livelli di un computer come divisi in: *Hardware*, utilizzato dagli sviluppatori dei sistemi operativi; *Sistema Operativo* e *Utilities*, utilizzati dai programmatori; *Applicazioni*, principale target di un normale utente.

Quindi cosa fa effettivamente un **sistema operativo**? Il suo scopo primario è fornire un insieme di servizi sia per gli sviluppatori che per gli utenti dialogando e gestendo le risorse hardware.

I principali servizi offerti da un sistema operativo sono: esecuzioni di programmi (app e servizi); fornire l'accesso ai dispositivi di input/output; accesso al sistema operativo stesso (shell); sviluppo di programmi (compilatori, editor e debugger); rilevamento e reazione ad errori; accounting (tutte le azioni che registrano, misurano e documentano le risorse concesse ad un utente); gestione delle risorse (può concedere il controllo del processore ad altri programmi).

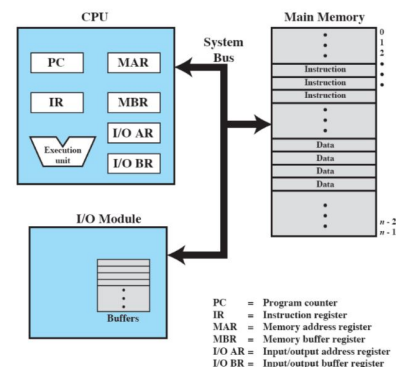
Quindi un sistema operativo è un programma, il primo programma che va in esecuzione all'accensione della macchina, il cui scopo principale è controllare l'esecuzione degli altri programmi. Nel fare questo deve essere efficiente, conveniente e capace di evolvere.

Il *kernel* (nucleo) è la parte del sistema operativo che si trova sempre in memoria principale, pronta per essere eseguita, e contiene le funzioni più usate. Nei moderni sistemi operativi si differenziano i kernel *monolitici* e *microkernel*. Nel primo tutte le parti del sistema operativo vengono caricate in memoria all'accensione del computer, mentre nel secondo, solo una minima parte del kernel è in memoria, il resto viene caricato quando serve.

1.1 Componenti di un computer mono-processore

Un computer è formato da: una *CPU* detta anche processore che contiene dei registri fondamentali per l'esecuzione di programmi; una RAM o *memoria principale* volatile; dei *dispositivi di input e output* (comprendono dispositivi di memoria secondaria non volatile, dispositivi per la comunicazione, tastiera, monitor, mouse...). Queste componenti devono poter comunicare tra loro e lo fanno attraverso i BUS di sistema.

I registri del processore si dividono in due macro-categorie: i registri visibili al programmatore che sono fondamentali per eseguire alcune istruzioni o per ridurre gli accessi alla memoria principale; i registri non visibili che sono utilizzati per controllare l'uso del processore stesso o per comunicare con la memoria e i dispositivi di



I/O.

Un sistema operativo si basa sul concetto che di per sé un computer "sa fare" solo una cosa: *fetch* e *execute* di istruzioni milioni di volte. Ovvero, nella fase di *fetch*, il processore preleva il contenuto del Program Counter da cui ottiene un indirizzo, il cui contenuto viene inserito nell'Instruction Register; nella fase di *execute*, prende il contenuto dell'IR e lo esegue dopo aver incrementato il valore del PC.

Esempio 1.1.1 – Esecuzione di un programma

Vediamo un esempio di esecuzione di un programma con delle istruzioni a 16 bit di cui i primi 4 rappresentano l'Opcode mentre gli ultimi 12 l'Address. Con Opcode 0001 otteniamo una *load*, con 0010 una *store*, con 0101 un'*add*.

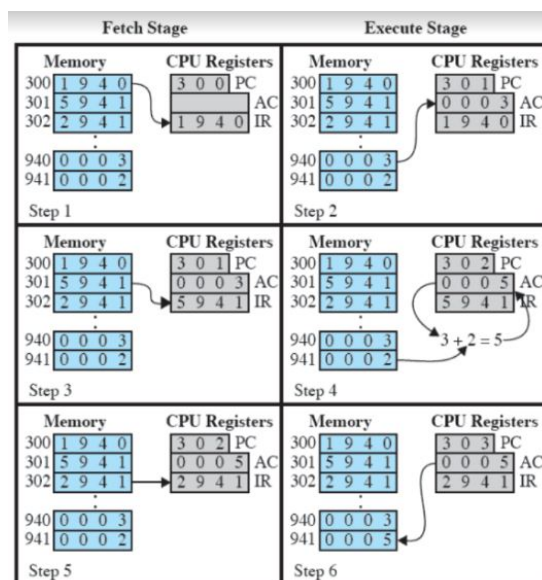


Figura 1.1: Esempio di esecuzione di un programma

Notiamo come la RAM, a partire dall'indirizzo 300 contiene un mini-programma di 3 istruzioni. Nella prima fase di *fetch* viene inserito nell'IR l'istruzione 1940 e viene incrementato il PC, successivamente, dato che i primi quattro bit dell'istruzione (0001) rappresentano una *load*, viene caricata dalla memoria il valore contenuto nell'indirizzo 940 e inserito nel registro temporaneo AC. Nella seconda fase l'Opcode dell'istruzione risulta un'*add* (0101) che viene effettuata tra il valore in AC e il valore contenuto all'indirizzo 941. Infine viene effettuata una *store* all'indirizzo 941 del contenuto attuale di AC.

1.2 Interruzioni

Un argomento fondamentale dell'interazione tra hardware e software sono le interruzioni.

Un'**interruzione** (interrupt) ha il compito di interrompere la normale esecuzione del processore deviando l'esecuzione del processo ed eseguendo un software di sistema che fa parte del sistema operativo.

Le interruzioni si dividono in due classi: interruzioni *sincrone* e interruzioni *asincrone*. Le interruzioni da programma sono le uniche sincrone ovvero, vengono generate come immediata conseguenza di una certa istruzione e per i processori Intel vengono chiamate *exception*. Le interruzioni asincrone, invece, vengono tipicamente sollevate dopo l'istruzione che le ha causate e ne esistono di vari tipi: le interruzioni da input/output vengono generate dal controllore di un dispositivo di I/O e segnalano il completamento o l'errore di un'operazione di I/O; le interruzioni da fallimento hardware sono le più imprevedibili in quanto vengono generate da errori relativi all'hardware; inter-

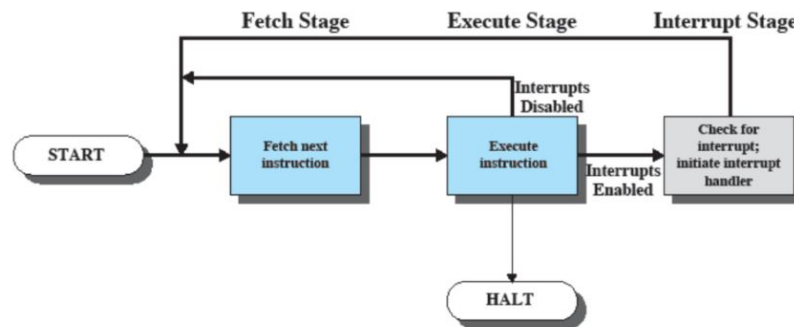


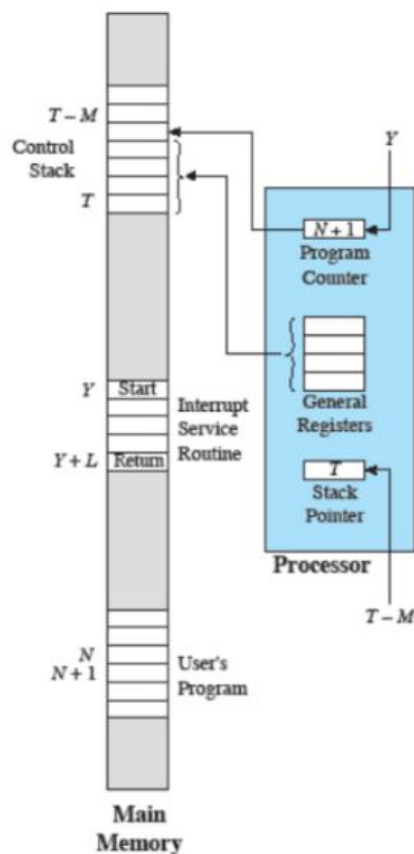
Figura 1.2: Esecuzione di istruzioni con interruzioni

ruzioni da comunicazione tra CPU utilizzate per gestire la comunicazione tra più CPU; interruzioni da timer sono generate da un timer all'interno del processore e permettono al sistema operativo di effettuare delle operazioni ad intervalli regolari.

Per le interruzioni asincrone, una volta che l'*handler* (gestore di interruzioni) è terminato, si riprende dall'istruzione subito successiva a quella dove si è verificata l'interruzione. Quindi ad ogni ciclo fetch-execute, viene anche controllato se c'è stata un'interruzione. Se così fosse, il programma viene sospeso e viene eseguita una funzione che gestisce l'interruzione (interrupt-handler routine). Nel punto in cui avviene questa "chiamata", il programma utente non prevedeva affatto di effettuare la chiamata all'interrupt-handler quindi è necessario che il sistema operativo e hardware collaborino per salvare le informazioni.

Esempio 1.2.1 – Gestione delle interruzioni

Il processore sta eseguendo l'istruzione all'indirizzo N quando arriva un'interruzione, da gestire con l'*handler* all'indirizzo Y .



Nel momento in cui viene emessa l'interruzione il processore finisce di eseguire l'istruzione N , riconosce il segnale dell'interruzione, inserisce PSW^a e PC dentro il *Control Stack*, carica il nuovo PC in base all'interrupt (Y). Una volta eseguite le istruzioni dell'*Interrupt Service Routine* viene ripristinata la memoria dei registri e del PC e si passa all'esecuzione dell'istruzione $N + 1$.

^aProgram Status Word è un registro speciale presente in un processore che contiene informazioni sullo stato attuale dell'esecuzione del programma.

In alcuni casi si possono generare più interrupt dando vita ad interrupt *sequenziali* se gestiti uno

dopo l'altro o interrupt *annidati* quando un'interrupt ne genera altri.

1.3 Miglioramento degli accessi

Il più vecchio modo di gestire la comunicazione con i dispositivi di I/O veniva effettuata nel seguente modo: la CPU manda al dispositivo un comando; la CPU aspetta una risposta con un ciclo; la CPU effettua il trasferimento e la scrittura in memoria. Notiamo come questo processo è molto inefficiente in quanto completamente controllato da una CPU che nella maggior parte dei casi aspetta una risposta dal dispositivo. Un primo miglioramento lo troviamo con l'aggiunta delle interruzioni: si elimina il ciclo di attesa del processore in quanto è il dispositivo di I/O stesso a mandare l'interrupt una volta pronto. Nonostante questo notevole miglioramento c'è ancora un problema, la CPU deve ancora prendere il dato dai registri per poi scriverlo in memoria e ricompaiono le attese del processore quando si sovrappongono più richieste contemporaneamente. Per risolvere questi problemi viene sfruttato il *DMA*¹, generando un singolo interrupt per blocco trasferito e leggendo i dati direttamente dalla memoria, e la *multiprogrammazione*, eseguendo più programmi contemporaneamente (la sequenza con cui i programmi sono eseguiti dipende dalla loro priorità).

1.4 Gerarchia della memoria

Possiamo immaginare il mondo delle memorie come diviso in tre macro-categorie: *Inboard Memory* comprende le memorie che si trovano direttamente sulla scheda madre del computer (Registri, Cache, Memoria principale); *Outboard Storage* rappresenta le memorie che si trovano sui dispositivi di input/output (Dischi magnetici, CD, DVD, Penne USB...); *Off-line Storage* riguarda quei dispositivi che si trovano al di fuori del computer.

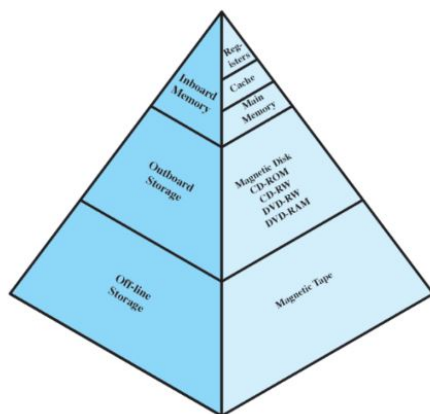


Figura 1.3: Gerarchia della memoria

Seguendo la Figura 1.3 dall'alto verso il basso: diminuisce la velocità di accesso; diminuisce il costo al bit; aumenta la capacità; diminuisce la frequenza di accesso alla memoria da parte del processore.

La CPU normalmente accede molto spesso alla memoria volatile della *Inboard Memory* e raramente alle memorie secondarie (*Outboard* e *Offline Storage*).

Anche nell'*Inboard Memory* ci sono importanti differenze di velocità, infatti, la velocità del processore è maggiore della velocità di accesso alla memoria principale. Per evitare eccessivi tempi di attesa, tutti i computer hanno una memoria *cache* che contiene copie di porzioni della memoria principale e che sfrutta il principio di *località dei riferimenti*². Quando la CPU vuole leggere un indirizzo, se il blocco corrispondente è presente nella cache allora il dato viene caricato (*HIT*), altrimenti il dato viene preso dalla RAM e il blocco copiato nella cache (*MISS*) attraverso una funzione di mappatura. Invece quando un dato deve esse-

re sovrascritto si utilizza generalmente la politica di rimpiazzo *LRU*³ e la memoria principale si può aggiornare in due modi: tramite il *Write Through* dove ad ogni modifica viene aggiornato il blocco in memoria; con il *Write Back* dove il blocco viene aggiornato solo quando viene sostituito.

1.5 Storia dei sistemi operativi

Nel corso della storia i sistemi operativi hanno attraversato diverse fasi "evolutive" che è importante conoscere per capire gli schemi che sono stati utilizzati per l'aggiornamento dell'hardware, aggiunta di nuovi servizi o correzione di errori.

¹Direct Memory Access è quel meccanismo che permette di accedere direttamente alla memoria interna per scambiare dati, in lettura e/o scrittura, senza coinvolgere l'unità di controllo (CPU).

²E' la tendenza di un processore ad accedere ripetutamente allo stesso insieme di posizioni di memoria in un breve periodo di tempo.

³Least Recently Used, si rimpiazza il blocco usato meno di recente.

Negli anni '40 non c'era nessun sistema operativo e venivano utilizzati degli interruttori per l'input e delle stampanti per gli output. Quasi fin da subito gli input furono migliorati attraverso delle schede perforate con un tipo di computazione prettamente seriale.

Il primo sistema operativo nacque quando si iniziarono ad inserire più programmi da eseguire chiamati *jobs* e un loro particolare linguaggio di controllo. Questo sistema funzionava particolarmente bene per i sistemi *batch* (non interattivi). Tutto ciò iniziò a mettere in luce delle necessità a cui bisognava iniziare a pensare: *protezione della memoria* per prevenire eventuali riscritture di codice da parte di un job; *timer* per impedire l'enorme esaurimento di tempo da parte di alcuni job; *istruzioni privilegiate* per permettere al sistema di controllo dei job di effettuare operazioni più alte. Così, come avviene tuttora, si passò ad eseguire i job solo in modalità *utente*, bloccando l'accesso ad alcune funzioni, e i monitor in modalità *sistema* (o kernel) permettendo l'utilizzo di istruzioni privilegiate o l'accesso ad aree protette della memoria.

Nonostante ciò, più del 96% del tempo della CPU era sprecato ad aspettare i dispositivi di I/O. Si iniziò a pensare alla *multiprogrammazione* eseguendo più programmi contemporaneamente durante i tempi di attesa dei processi migliorando di gran lunga le prestazioni del sistema operativo.

A partire dagli anni '60 la richiesta di job interattivi iniziò a diventare sempre più esigente. Ogni utente infatti era collegato, tramite un terminale, ad un unico grande computer (main frame), e si iniziarono a sviluppare dei sistemi di condivisione del tempo (*Time Sharing System*), dove il tempo del processore era condiviso tra più utenti.

Tutto questo portò ad ulteriori sviluppi interessanti che sono tutt'ora validi per i sistemi operativi: il concetto di processo; la gestione della memoria; sicurezza e protezione delle informazioni (privacy); gestione dello scheduling e delle risorse; strutturazione del sistema.

Il *processo* riunisce in un unico concetto il job non-interattivo e quello interattivo ed è un'unità di computazione caratterizzata da: almeno un flusso di esecuzione (thread); uno stato corrente; un insieme di risorse di sistema ad esso associate.

Quando si parla invece di strutturazione del sistema si intende raggruppare insieme delle funzioni analoghe in livelli.

Capitolo 2

Gestione dei processi

Il compito fondamentale di un sistema operativo è la gestione dei processi, ovvero eseguire diverse computazioni. Infatti un sistema operativo deve poter permettere l'esecuzione alternata di processi multipli; assegnare le risorse ai processi; permettere ai processi di scambiarsi informazioni; permettere la sincronizzazione tra processi.

Un **processo** è a tutti gli effetti un'istanza di un programma in esecuzione caratterizzata da una sequenza di istruzioni, da uno stato corrente e da un insieme associato di risorse.

Va specificato che per esecuzione di un processo si intende l'esecuzione di un programma che ancora non è terminato. Questo non significa che il processo sia effettivamente in esecuzione su un processore, infatti la decisione di esecuzione di un processo è uno dei compiti fondamentali di un sistema operativo.

Solo eseguendo un programma si crea un processo. Nei sistemi operativi moderni il programma è sempre registrato su un disco rigido, fanno eccezioni i processi creati dal sistema operativo stesso.

2.1 Stati di un processo

Ogni processo ha tre macro-fasi: *creazione*, *esecuzione*, *terminazione*. La terminazione, se presente, può essere o esplicitamente prevista (es. click sulla X della finestra), oppure non prevista (in caso di errori).

Finché un processo è in esecuzione ha associato un *Process Control Block* creato dal sistema operativo. Questo block contiene sufficienti informazioni per permettere al sistema operativo di gestire più processi contemporaneamente. Queste informazioni sono: un *identificatore* per differenziare i processi; uno *stato* (es. *running*...); *priorità*; *hardware context* contiene il valore corrente dei registri della CPU; dei *puntatori alla memoria* che definiscono l'immagine del processo; delle *informazioni sullo stato dell'I/O*; delle eventuali *informazioni di accounting*.

Il comportamento di un processo è caratterizzato dalla sequenza di istruzioni che vengono eseguite, questa sequenza è detta *trace* (traccia). Il *dispatcher*, invece, è un piccolo programma contenuto in memoria che ha il compito di sospendere un processo per eseguirne un altro.

Identifier
State
Priority
Program counter
Memory pointers
Context data
I/O status information
Accounting information
⋮

Esempio 2.1.1 – Gestione dei processi

Si considerino 3 processi in esecuzione tutti in memoria compreso il dispatcher.

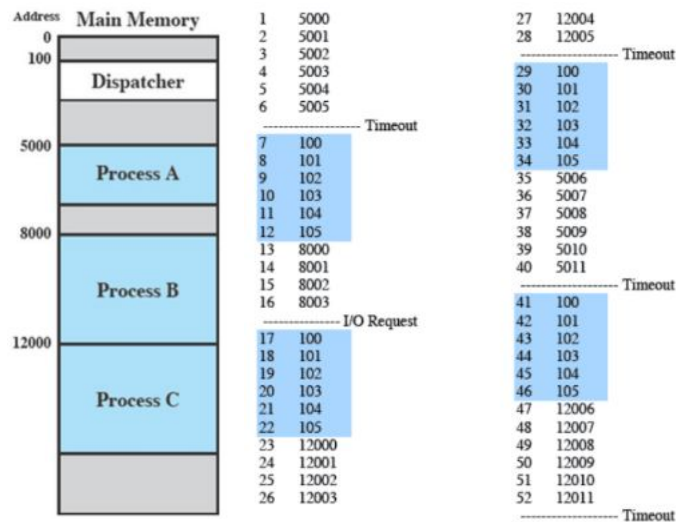


Figura 2.1: Esempio di esecuzione di un programma

La sequenza di istruzioni sul processore non è sequenziale come mostrato nella figura di sinistra, ma dobbiamo immaginare una coda di processi in attesa e un dispatcher che muove i processi dalla CPU alla coda e viceversa, finché un processo non viene completato.

In ogni sistema operativo ci sono $n \geq 1$ processi, infatti come minimo c'è un processo master (interfaccia grafica o testuale) che attende comandi dall'utente e non può essere terminato a meno che non viene spento il computer. Nel momento in cui l'utente esegue un comando quasi sempre si crea un nuovo processo. Questo concetto, secondo cui un processo ne crea un altro, viene chiamato *process spawning*. Il processo padre è il processo originale che crea il processo figlio passando da n a $n + 1$ processi in esecuzione.

La terminazione di un processo può avvenire per normale completamento, (*HALT*) generando un'interruzione che viene gestita dall'handler del sistema operativo, o per uccisioni (*kill*) dal sistema operativo stesso, per errori, o dall'utente, tramite GUI.

A questo punto i cinque stati dei processi sono divisi in: creazione, terminazione ed esecuzione che comprende *Running*, *Ready* o *Blocked* quando il processo viene messo in attesa (ad es. per aspettare una risposta dai dispositivi di I/O).

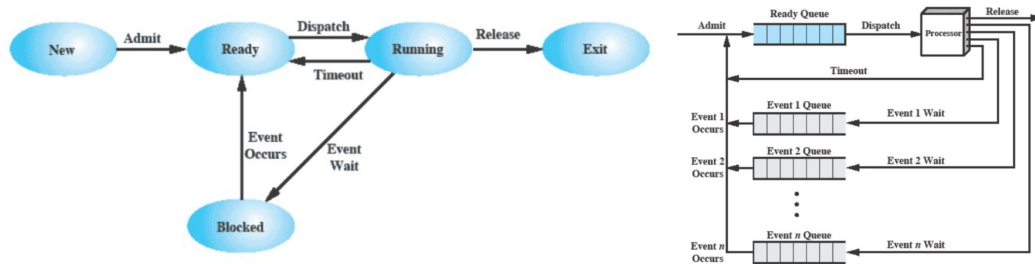


Figura 2.2: Stati di un processo

A livello di strutture dati troviamo n code: una per i processi Ready che possono andare direttamente in esecuzione nel processore e $n - 1$ code per i processi Blocked che fungono da code di attesa. I sistemi operativi, infatti, non hanno una coda per tutti i casi di block ma ne hanno una per ogni evento bloccante in modo da aumentare l'efficienza del sistema.

Molti processi che finiscono in attesa stanno di fatto occupando spazio prezioso in RAM, per questo motivo vengono spostati su disco così da liberare la memoria, questi processi vengono chiamati processi sospesi (*Suspended*). Ai cinque stati originali se ne aggiungono due: (*Blocked/Suspend*) e (*Ready/Suspend*).

Notiamo ora che quando viene creato un nuovo processo c'è subito una scelta da fare, ovvero inserire il processo in RAM con stato Ready per essere scelto dal dispatcher oppure su disco con stato Ready/Suspend.

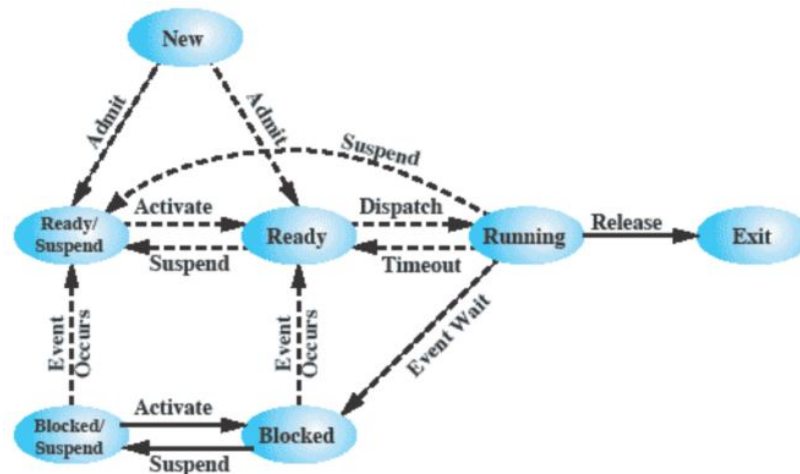


Figura 2.3: Stati sospesi di un processo

Abbiamo visto che i processi vengono sospesi per risparmiare risorse di memoria, in realtà i motivi per sospendere un processo possono essere diversi: *swapping* quando il sistema operativo ha bisogno di rilasciare abbastanza memoria da portarci dentro un processo Ready; il sistema operativo sospetta che il processo stia causando problemi; per richieste utente interattive come il debugging; il processo viene eseguito periodicamente; il padre vorrebbe poter sospendere l'esecuzione del figlio per esaminarlo, modificarlo o coordinare l'attività dei figli.

2.2 Descrizione di un processo

Un altro dei compiti principali di un sistema operativo è la gestione dell'uso delle risorse da parte dei processori. Poiché il sistema operativo deve gestire sia i processi che le risorse, deve conoscere lo stato interno di entrambe le entità, a tal proposito, per ogni entità costruisce o mantiene una o più tabelle.

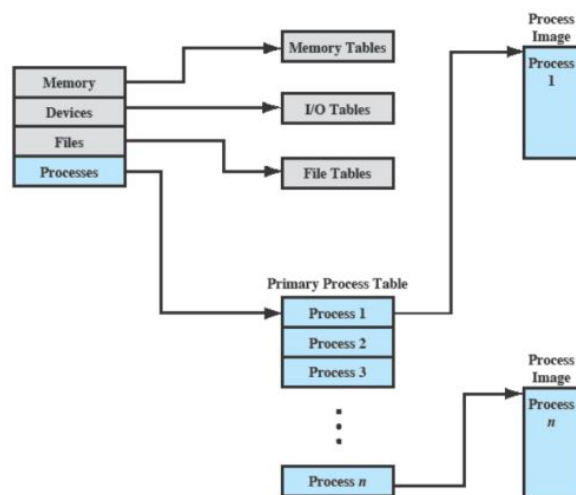


Figura 2.4: Tabelle di controllo del sistema operativo

In questa sezione ci soffermeremo solamente sulle tabelle dei processi e le altre tabelle saranno approfondite nei capitoli dedicati.

Lo stato interno di una *tabella dei processi* contiene: uno *stato* corrente (Ready, Running, Blocked...); un *identificatore* per distinguere i processi; una *locazione di memoria*; altre informazioni utili. Molte di queste informazioni si trovano nel *Process Control Block (PCB)* e vengono chiamate *attributi* di un processo, mentre il processo vero e proprio chiamato *Process Image* (immagine del processo),

per ovvi motivi di spazio, non si trova nel PCB ma in memoria. La Process Image è l'insieme del programma sorgente, dati, stack delle chiamate e PCB. Anche l'esecuzione di una sola istruzione cambia l'immagine del processo perché vengono modificati uno o più registri o celle di memoria.

Attributi di un processo

Gli attributi di ciascun blocco di controllo possono essere raggruppate in tre categorie: identificazione; stato; controllo.

1. Ad ogni processo è assegnato un numero identificativo *PID* (Process IDentifier). Questo identificativo è così importante che molte tabelle del sistema operativo, per realizzare i collegamenti con la tabella dei processi, usano direttamente i PID. Fanno parte degli identificatori anche il processo padre (Parent PID, o PPDI) e l'utente che ha eseguito il processo;
2. Lo stato del processore, da non confondere con lo stato del processo, è formato dal contenuto dei registri del processore stesso;
3. Fanno parte invece delle informazioni di controllo:
 - *Informazioni per il controllo del processo*: lo stato (Ready, Suspended, Blocked...); la priorità; informazioni sullo scheduling; l'evento da attendere per tornare ad essere Ready;
 - *Supporto per strutture dati*: puntatori ad altri processi;
 - *Comunicazioni tra processi*: flag, segnali, messaggi per supportare comunicazioni tra processi;
 - *Permessi speciali*;
 - *Gestione della memoria*: puntatori ad aree di memoria che gestiscono l'uso della memoria virtuale, ad es. pagine virtuali attualmente in uso;
 - *Uso delle risorse*: file aperti, uso di risorse (compreso il processore).

2.3 Controllo di un processo

Abbiamo visto come la maggior parte dei processori supporta almeno due modalità di esecuzione: modo di sistema (*kernel*) in cui si ha il pieno controllo del sistema (si può eseguire qualsiasi istruzione o accedere a qualsiasi locazione di memoria); modo utente (*user*) è la modalità di esecuzione dei programmi utente dove molte operazioni sono vietate.

Le operazioni effettuate dal kernel sono: la *gestione dei processi* tramite PCB che permettono la creazione, terminazione, scheduling, dispatching, process switching, sincronizzazione e comunicazione tra processi; la *gestione della memoria principale* che prevede l'allocazione/de-allocazione di spazio per i processi e la gestione della memoria virtuale; la *gestione dell'I/O* per la gestione dei buffer e delle cache e l'assegnazione delle risorse di I/O ai processi; le *funzioni di supporto* per la gestione degli interrupt/eccezioni, accounting e monitoraggio.

In un sistema di questo tipo, dove molti processi hanno necessità di I/O, nasce l'esigenza di passare da modalità kernel a quella utente e viceversa. L'idea è molto semplice: un processo utente inizia sempre in modalità utente; passa a modalità sistema a seguito di un interrupt, in questa fase l'hardware prima di invocare l'interrupt handler cambia la modalità da utente a sistema; l'ultima istruzione dell'interrupt handler, prima di restituire il controllo al processo utente, effettua nuovamente lo switch in modalità utente. Con questo schema, un processo utente può cambiare la modalità a sé stesso, ma solo per eseguire software di sistema. Per richiedere un servizio a livello kernel del sistema operativo si usano delle chiamate di sistema (o *system call*), in effetti anche la creazione di un nuovo processo è una *system call*.

Per creare un processo il sistema operativo deve: assegnare al processo un PID unico; allocargli spazio in memoria principale; inizializzare il process control block; inserire il processo nella giusta coda (ready o ready/suspended); creare o espandere altre strutture dati.

Lo switching tra processi, invece, può essere scatenato da varie cause: interruzione esterna all'esecuzione dell'istruzione corrente (include i timer per evitare la prolungata esecuzione di un processo); eccezione associata all'istruzione corrente; chiamata esplicita al sistema operativo (*system call*). Per eseguire lo switching bisogna seguire vari passaggi in kernel mode: salvare il contesto del programma (registri e PC); aggiornare il process control block, attualmente in running; spostare il process

control block nella coda appropriata (ready, blocked, ready/suspend); scegliere un altro processo da eseguire; aggiornare il process control block del processo selezionato; aggiornare le strutture dati per la gestione della memoria; ripristinare il contesto del processo selezionato.

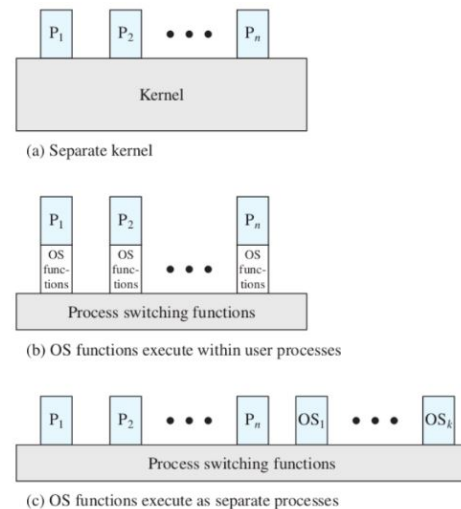
2.4 Esecuzione del sistema operativo

A questo punto nasce spontanea una domanda: *il sistema operativo è un processo?* Abbiamo definito il sistema operativo come un insieme di programmi eseguiti sul processore che permette agli altri programmi di andare in esecuzione, per poi riprendere il controllo tramite interrupt. In questo senso il sistema operativo è a tutti gli effetti un processo, *ma quindi da chi viene controllato?*

In realtà dipende dal tipo di sistema operativo: ci sono sistemi operativi, meno recenti, che usano il kernel separato, altri che usano delle funzioni eseguite all'interno del processo utente e altri ancora che uniscono le due modalità.

- Il kernel viene eseguito al di fuori dei processi e il sistema operativo è eseguito come un'entità separata, con privilegi più elevati che ha una sua zona di memoria dedicata per i dati, per il codice sorgente e per lo stack.
- Nei sistemi che utilizzano l'esecuzione all'interno dei processi utente, il sistema operativo viene eseguito nel contesto di un processo utente e non c'è bisogno di un process switch ma basta cambiare modalità di esecuzione da utente a sistema.

- Nel sistema operativo basato sui processi, invece, tutti gli aspetti sono visti come dei processi (tranne le funzioni di switch). Il sistema operativo stesso è implementato come un insieme di processi di sistema con privilegi più alti.



2.5 Gestione dei processi UNIX SVR4

UNIX SVR4 usa il modello dove la maggior parte del sistema operativo viene eseguito all'interno dei processi utente. UNIX utilizza due categorie di processi: processi di sistema eseguiti in modalità kernel e processi utente per eseguire programmi ed utilità. Il diagramma degli stati dei processi (vedi Figura 2.5) è diviso in nove stati diversi che comprendono: *User Running* in esecuzione in modalità utente; *Kernel Running* in esecuzione in modalità kernel o sistema; *Ready to Run in Memory* può andare in esecuzione non appena il kernel lo seleziona; *Asleep in Memory* non può essere eseguito finché un qualche evento non si manifesta e il processo è in memoria; *Ready to Run Swapped* può andare in esecuzione (non è in attesa di eventi esterni), ma prima dovrà essere portato in memoria; *Sleeping Swapped* non può essere eseguito finché un qualche evento non si manifesta e il processo non è in memoria primaria; *Preempted* il kernel ha appena tolto l'uso del processore a questo processo (preemption), per fare un context switch; *Created* appena creato, ma ancora non pronto all'esecuzione; *Zombie* terminato, ma resta nelle tabelle dei processi perché il processo che l'ha creato possa prendersi il suo valore di ritorno.

Un insieme di strutture dati forniscono al sistema operativo le informazioni per gestire i processi. Queste informazioni sono raggruppate in livello utente, livello registro e livello di sistema.

A livello utente troviamo: *Process text* il codice sorgente (in linguaggio macchina) del processo; *Process data* sezione di dati del processo dove vengono mappati i valori delle variabili; *User stack* ovvero lo stack delle chiamate del processo; *Shared Memory*, la memoria condivisa con altri processi, usata per le comunicazioni tra processi.

Il livello registro contiene: *Program counter*, l'indirizzo della prossima istruzione del process text da eseguire; *Processor status register*, il registro di stato del processore, relativo a quando è stato swappato l'ultima volta; *Stack pointer* è il puntatore alla cima dello user stack; *General purpose registers* rappresenta il contenuto dei registri accessibili al programmatore, relativo a quando è stato swappato l'ultima volta.

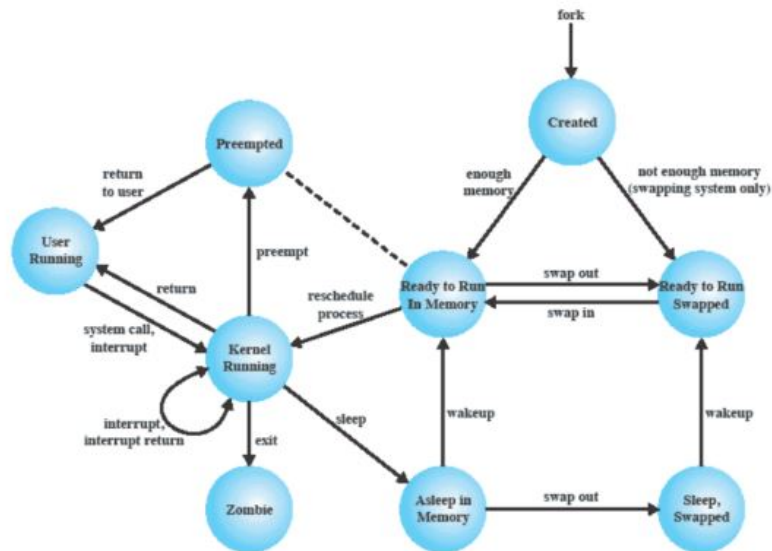


Figura 2.5: Stati dei processi in UNIX

Il livello sistema ha: *Process table entry*, il puntatore alla tabella di tutti i processi, dove individua quello corrente; *U area* contiene le informazioni per il controllo del processo; *Per process region table* definisce il mapping tra indirizzi virtuali ed indirizzi fisici (page table); *Kernel stack* è lo stack delle chiamate, separato da quello utente, usato per le funzioni da eseguire in modalità sistema.

Process status	Current state of process.	Process table pointer	Indicates entry that corresponds to the U area.
Pointers	To U area and process memory area (text, data, stack).	User identifiers	Real and effective user IDs. Used to determine user privileges.
Process size	Enables the operating system to know how much space to allocate the process.	Timers	Record time that the process (and its descendants) spent executing in user mode and in kernel mode.
User identifiers	The real user ID identifies the user who is responsible for the running process. The effective user ID may be used by a process to gain temporary privileges associated with a particular program; while that program is being executed as part of the process, the process operates with the effective user ID.	Signal-handler array	For each type of signal defined in the system, indicates how the process will react to receipt of that signal (exit, ignore, execute specified user function).
Process identifiers	ID of this process; ID of parent process. These are set up when the process enters the Created state during the fork system call.	Control terminal	Indicates login terminal for this process, if one exists.
Event descriptor	Valid when a process is in a sleeping state; when the event occurs, the process is transferred to a ready-to-run state.	Error field	Records errors encountered during a system call.
Priority	Used for process scheduling.	Return value	Contains the result of system calls.
Signal	Enumerates signals sent to a process but not yet handled.	I/O parameters	Describe the amount of data to transfer, the address of the source (or target) data array in user space, and file offsets for I/O.
Timers	Include process execution time, kernel resource utilization, and user-set timer used to send alarm signal to a process.	File parameters	Current directory and current root describe the file system environment of the process.
P_link	Pointer to the next link in the ready queue (valid if process is ready to execute).	User file descriptor table	Records the files the process has open.
Memory status	Indicates whether process image is in main memory or swapped out. If it is in memory, this field also indicates whether it may be swapped out or is temporarily locked into main memory.	Limit fields	Restrict the size of the process and the size of a file it can write.
		Permission modes fields	Mask mode settings on files the process creates.

Figura 2.6: Process table entry e U area

La creazione di un processo UNIX viene effettuata tramite la chiamata di sistema `fork()`. In seguito a ciò, il sistema operativo, in Kernel Mode: alloca una entry nella tabella dei processi per il nuovo processo (figlio); assegna un PID unico al processo figlio; copia l'immagine del padre, escludendo dalla copia la memoria condivisa (se presente); incrementa i contatori di ogni file aperto dal padre, per tenere conto del fatto che ora sono anche del figlio; assegna al processo figlio lo stato Ready to Run; fa ritornare alla fork il PID del figlio al padre, e 0 al figlio. Dopodiché, il Kernel può scegliere tra: continuare ad eseguire il padre, switchare al figlio o switchare ad un altro processo.

Capitolo 3

Thread

Fino ad ora abbiamo visto dei processi in esecuzione che possono alternarsi oppure essere in esecuzione contemporanea in base al numero di processori che si trovano sulla macchina. Tuttavia, per alcune applicazioni è importante essere organizzate in modo parallelo ed è il programmatore stesso a suddividere l'applicazione in diverse esecuzioni, ciascuna esecuzione si chiama *thread*. Un esempio tipico di thread è un'applicazione grafica.

Diversi thread di uno stesso processo condividono tutte le risorse (memoria, file, dispositivi di I/O...) tranne lo stack delle chiamate e il processore.

Ma quali sono i vantaggi di utilizzare i thread che non possono essere soddisfatti attraverso il solo utilizzo alternato o parallelo dei processi? In verità utilizzare i thread, come vedremo, è più semplice ed efficiente per la creazione, terminazione, switching e comunicazione tra thread.

3.1 Processi e thread

Si può dire quindi che il concetto di processo incorpora due caratteristiche: gestione delle risorse, dove i processi vanno intesi come blocco unico, e scheduling/esecuzione, dove i processi possono contenere diverse tracce (thread).

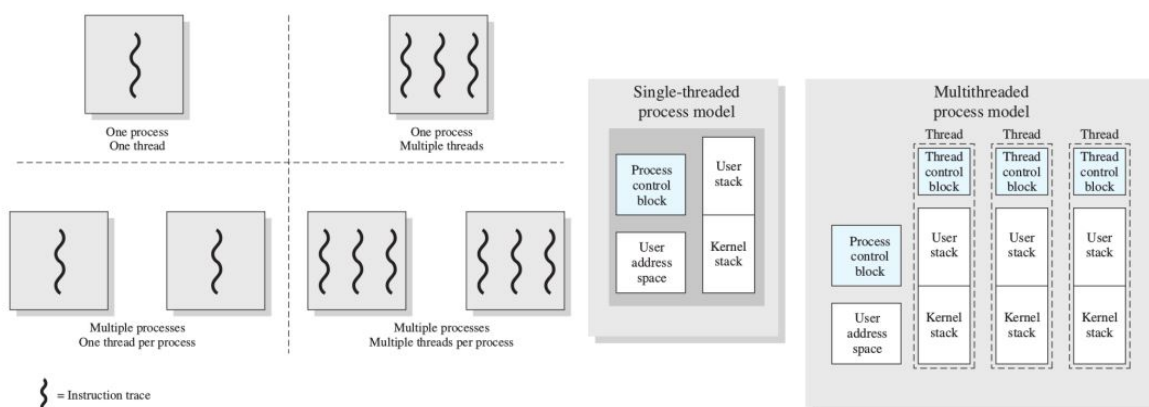


Figura 3.1: Gestione dei processi e thread

In base al numero di processi in esecuzione possiamo trovare, nel sistema operativo, uno o più thread per processo. In entrambe i casi rimangono il PCB e l'indirizzo o immagine comune a tutti i thread, mentre, nel multithread, ogni thread viene gestito attraverso un *Thread control block* per lo scheduling, uno stack delle chiamate utente e uno stack per le chiamate di sistema.

Ogni processo viene creato con un thread, dopodiché il programmatore può eseguire un'opportuna chiamata di sistema che: genera un nuovo thread (*spawn*); blocca un thread (*block*); sblocca un thread (*unblock*); termina un thread (*finish*).

3.2 Tipi di thread

E' necessaria una distinzione tra thread a livello utente (*User Level Thread - ULT*) e thread a livello di sistema (*Kernel Level Thread - KLT*).

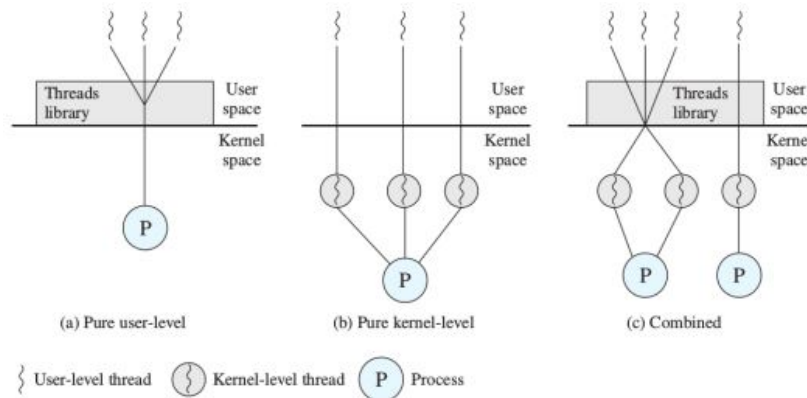


Figura 3.2: User Level Thread e Kernel Level Thread

Nei primi, il sistema operativo vede solo un processo, e non ha nessuna idea che al suo interno è diviso in thread, perché ci sono delle opportune librerie (*Threads library*) che gestiscono direttamente i thread del processo. Con i KLT, invece, il sistema operativo è a conoscenza della divisione in thread del processo e le librerie di thread si utilizzano direttamente le system call del sistema operativo.

Con gli ULT: è più facile ed efficiente lo switch; si può avere una politica di scheduling diversa per ogni applicazione; permettono di usare i thread anche sui sistemi operativi che non li offrono nativamente. Ma allo stesso tempo: se un thread si blocca, si bloccano tutti i thread di quel processo; se ci sono effettivamente più processori o più cores, tutti i thread del processo ne possono usare uno solo; se il sistema operativo non ha i KLT, non si possono utilizzare thread per le routine del sistema operativo stesso.

3.3 Processi e thread in Linux

In Linux l'unità di base sono i thread che vengono chiamati LWP (*Lightweight process*). Sono presenti sia gli ULT che i KLT ma vengono utilizzati in maniera diversa: i KLT vengono usati principalmente dal sistema operativo e qualsiasi utente può scrivere degli ULT, che vengono poi mappati in KLT (pthreads). Ad ogni thread, inoltre, viene associato un PID unico che lo identifica e un suo PCB. L'entry del PCB che dà il PID comune a tutti i thread di un processo è il TGID (*Thread Group Identifier*) che coincide con il primo thread del processo. Nel PCB di Linux, la struttura *thread_info* è organizzata anche per contenere il kernel stack, ovvero, lo stack delle chiamate da usare quando il processo passa in modalità sistema. Il *thread_group* punta agli altri thread di questo stesso processo e *parent* e *real_parent* puntano al padre del processo, ci sono anche link per i fratelli e i figli.

Lo schema degli stati in Linux è come quello a 5 stati (vedi Figura 2.2) ovvero, non fa un'esplícita menzione a processi suspended. Tuttavia, internamente il kernel usa questi stati: *Task Running*: include sia Ready che Running; *Task interruptible*, *Task uninterruptible*, *Task stopped*, *Task traced*: sono tutti Blocked, la differenza la fa il motivo per cui sono blocked; *Exit Zombie*, *Exit Dead*: sono entrambi Exit.

Segnali e interrupt in Linux

Non bisogna confondere segnali con interrupt (o eccezioni): i segnali possono essere inviati da un processo ad un altro tramite system call. Quando ciò succede, il segnale appena lanciato viene aggiunto all'opportuno campo del PCB del processo ricevente e, quando il processo viene nuovamente schedato per l'esecuzione, il kernel controlla prima se ci sono segnali pendenti e nel caso esegue un'opportuna funzione chiamata *signal handler*.

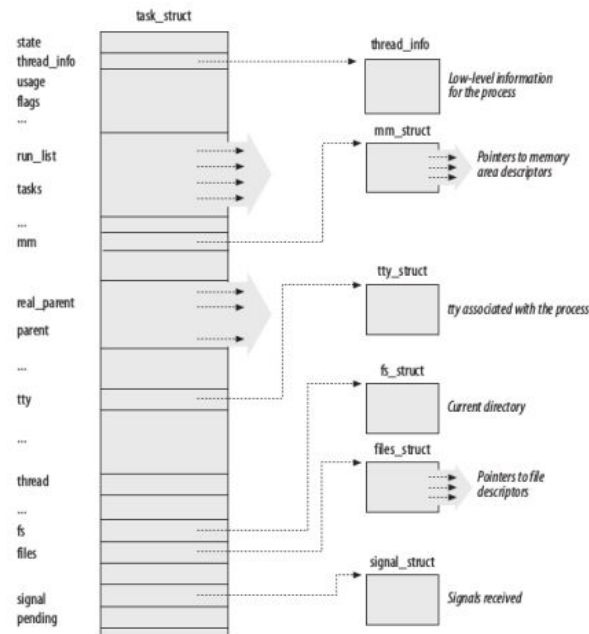


Figura 3.3: PCB in Linux

Esempio 3.3.1 – Segnali e interrupt in Linux

Si esegue un programma C scritto male, che accede ad una zona di memoria senza averla prima richiesta. Il processore fa scattare un'eccezione e viene eseguito l'opportuno exception handler (in kernel mode), che essenzialmente manda il segnale SIGSEGV al processo colpevole. Quando il processo colpevole verrà selezionato nuovamente per andare in esecuzione, il kernel vedrà dal PCB che c'è un segnale pendente, e farà in modo che venga eseguita l'azione corrispondente. L'azione di default per tale segnale è: fai terminare il processo, ma può essere riscritta dall'utente.

Capitolo 4

Scheduling

Un sistema operativo deve allocare diverse risorse tra diversi processi che ne fanno richiesta contemporaneamente. Tra le diverse risorse che i processi possono richiedere, la più ovvia è il processore. Questa risorsa viene allocata tramite lo *scheduling* (pianificazione).

Quindi lo scopo dello **scheduling** è assegnare ad ogni processore i processi da eseguire, man mano che i processi stessi vengono creati e distrutti, tenendo conto dei tempi di risposta, del throughput e dell'efficienza del processore.

Altri obiettivi dello scheduling sono: distribuire il tempo di esecuzione in modo equo (fair) tra i vari processi; gestire le priorità dei processi quando necessario; evitare la starvation (letteralmente, morte per fame) dei processi; usare il processore in modo efficiente; avere un overhead¹ basso.

4.1 Tipi di scheduling

Esistono vari tipi di scheduling a seconda della frequenza con cui vengono eseguiti:

- *Long-term scheduling* (scheduling di lungo termine), decide l'aggiunta ai processi da essere eseguiti;
- *Medium-term scheduling* (scheduling di medio termine) decide l'aggiunta ai processi che sono in memoria principale;
- *Short-term scheduling* (scheduling di breve termine) decide quale processo, tra quelli pronti, va eseguito da un processore;
- *I/O scheduling* (scheduling per l'input/output) decide a quale processo, tra quelli con una richiesta pendente per l'I/O, va assegnato il corrispondente dispositivo di I/O.

Quasi tutte le transizioni degli stati dei processi sono decise dai vari tipi di scheduler (vedi Figura 4.1). In particolare, il Long-term scheduling sceglie se un processo appena creato deve essere Ready o Ready/Suspended; il Medium-term scheduling decide tra le versioni di Suspend o non Suspend e viceversa; lo Short-term scheduling si occupa solamente di quei processi Ready, in coda, che aspettano di essere eseguiti.

Long-term scheduler

Il Long-term scheduler (LTS) decide quali programmi sono ammessi nel sistema per essere eseguiti. E' di tipo FIFO (First In First Out), ovvero il primo processo in input è il primo ad essere ammesso, e tiene conto dei criteri di priorità, requisiti per l'I/O e tempo di esecuzione atteso. Dato che, il LTS, decide quali processi vanno in RAM, controlla anche il grado di multiprogrammazione.

Quindi tipicamente: i lavori batch vengono accodati e il LTS li prende man mano che lo ritiene "giusto"; i lavori interattivi vengono ammessi fino a "saturazione" del sistema; se si sa quali processi

¹Utilizzato per definire le risorse accessorie, richieste in più rispetto a quelle strettamente necessarie per ottenere un determinato scopo.

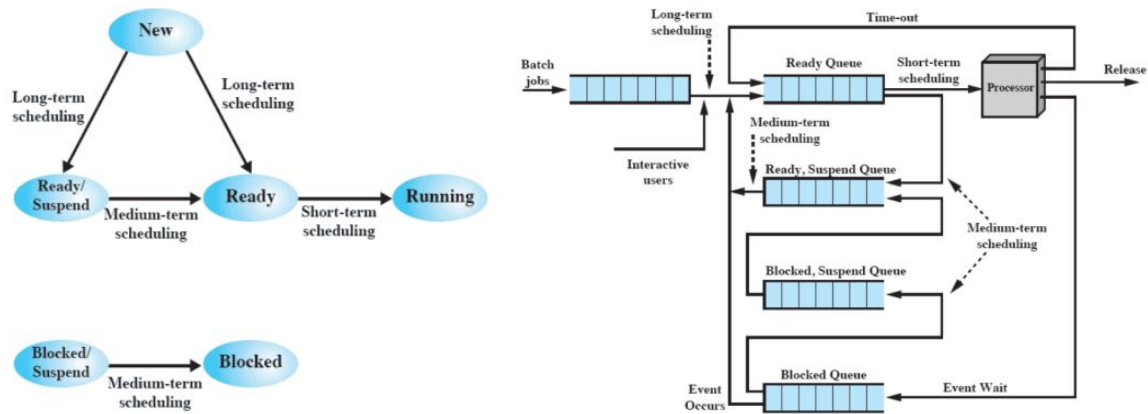


Figura 4.1: Processi e scheduling

sono I/O bound e quali CPU-bound², può mantenere un giusto mix tra i 2 tipi, oppure bilanciare le richieste ai dispositivi di I/O. Il LTS può essere chiamato in causa anche quando non ci sono nuovi processi, ad esempio quando termina un processo o quando alcuni processi sono idle³ da troppo tempo.

Medium-term scheduler

Il Medium-term scheduler (MTS) è parte della funzione di swapping per processi, ovvero del passaggio da memoria secondaria a principale e viceversa. Questo passaggio è basato sulla necessità di gestire il grado di multiprogrammazione.

Short-term scheduler

Lo Short-term scheduler (STS), chiamato anche dispatcher, è lo scheduler eseguito più frequentemente ed è tipicamente invocato sulla base di eventi come interruzioni di clock, interruzioni di I/O, chiamate di sistema o segnali.

Lo scopo dello STS è allocare tempo di esecuzione su un processore per ottimizzare il comportamento dell'intero sistema, dipendentemente da determinati indici prestazionali.

4.2 Algoritmi di scheduling

In questa sezione verranno presentati alcuni criteri fondamentali per lo scheduling che possono essere raggruppati in due macro-gruppi: criteri per l'utente e criteri per il sistema, e criteri prestazionali e non prestazionali.

- Il *Turn-around Time* è il tempo tra la creazione di un processo e il suo completamento, comprende anche i vari tempi di attesa (I/O, processore);
- Il *Response Time* è il tempo tra la sottomissione di una richiesta e l'inizio della risposta. Lo scheduler deve minimizzare il tempo di risposta medio e massimizzare il numero di utenti con un buon tempo di risposta.
- La *Deadline* è utilizzata nei casi in cui si possono specificare scadenze dei processi, mentre la *Predicibilità* richiede che non deve esserci troppa variabilità nei tempi di risposta e/o di ritorno;
- Il *Throughput* serve per massimizzare il numero di processi completati per unità di tempo o che abbiano cominciato a produrre output. E' una misura di quanto lavoro viene effettuato che dipende anche dal tempo medio di un processo;
- L'*Utilizzo del Processore* rappresenta la percentuale di tempo in cui il processore viene utilizzato;

²Sono processi che utilizzano molte risorse di I/O o CPU.

³Tempo durante il quale un mezzo non effettua nessuna attività ed è in attesa.

- Con il *Bilanciamento delle risorse*, lo scheduler deve far sì che le risorse del sistema siano usate il più possibile. I processi che useranno meno le risorse attualmente più usate dovranno essere favoriti.
- Con *Fairness e Priorità*, se non ci sono indicazioni dagli utenti o dal sistema, tutti i processi devono essere trattati allo stesso modo, se invece ci sono priorità, lo scheduler deve favorire i processi a priorità più alta. Occorrerà pertanto avere più code, una per ogni livello di priorità;

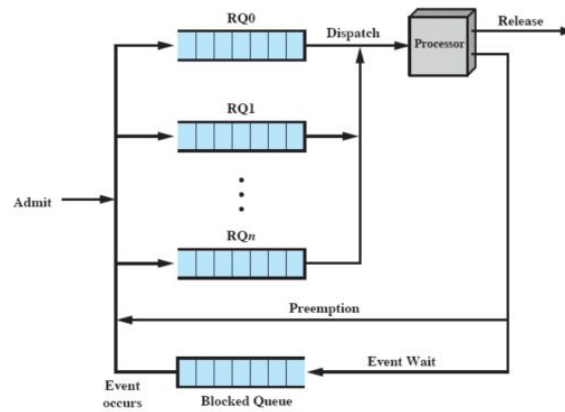


Figura 4.2: Code per la gestione delle priorità

- *Priorità e Starvation*: un processo a bassa priorità potrebbe soffrire di starvation⁴ a causa di un altro processo a priorità più alta. Una soluzione da adottare potrebbe essere: man mano che l'età del processo aumenta, la priorità cresce.

In Figura 4.3 sono presentati i più famosi algoritmi di scheduling con le rispettive caratteristiche. Nei sistemi operativi moderni, generalmente, si utilizzano più combinazioni di questi algoritmi.

	FCFS	Round Robin	SPN	SRT	HRRN	Feedback
Selection Function	$\max[w]$	constant	$\min[s]$	$\min[s - e]$	$\max\left(\frac{w + s}{s}\right)$	(see text)
Decision Mode	Non-preemptive	Preemptive (at time quantum)	Non-preemptive	Preemptive (at arrival)	Non-preemptive	Preemptive (at time quantum)
Throughput	Not emphasized	May be low if quantum is too small	High	High	High	Not emphasized
Response Time	May be high, especially if there is a large variance in process execution times	Provides good response time for short processes	Provides good response time for short processes	Provides good response time	Provides good response time	Not emphasized
Overhead	Minimum	Minimum	Can be high	Can be high	Can be high	Can be high
Effect on Processes	Penalizes short processes; penalizes I/O bound processes	Fair treatment	Penalizes long processes	Penalizes long processes	Good balance	May favor I/O bound processes
Starvation	No	No	Possible	Possible	No	Possible

Figura 4.3: Politiche di scheduling

⁴Si intende l'impossibilità perpetua, da parte di un processo pronto all'esecuzione, di ottenere le risorse sia hardware sia software di cui necessita per essere eseguito.

- La *funzione di selezione* è quella che sceglie effettivamente il processo da mandare in esecuzione. I parametri da cui dipende sono: w = tempo trascorso in attesa, e = tempo trascorso in esecuzione, s = tempo totale richiesto, incluso quello già servito (e).
- La *modalità di decisione* specifica in quali istanti di tempo la funzione di selezione viene invocata. Ci sono 2 possibilità: preemptive o non-preemptive;
 - *Non-Preemptive*: se un processo è in esecuzione, allora viene eseguito fino alla sua terminazione o fino ad una richiesta di I/O (o comunque ad una richiesta bloccante);
 - *Preemptive*: il sistema operativo può interrompere un processo in esecuzione anche se il processo non è terminato o se ci sono delle richieste bloccanti. La preemption di un processo può avvenire o per l'arrivo di nuovi processi (appena forkati) o per un interrupt di I/O.

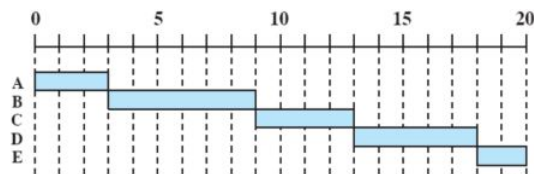
Esempio 4.2.1 – Scenario comune di algoritmi di scheduling

Supponiamo di avere 5 processi batch divisi come segue:

Processo	Tempo di arrivo	Tempo di esecuzione
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

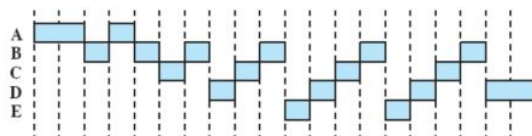
Vediamo come si comportano i vari algoritmi di scheduling:

- Il *FCFS* (First Come First Served) è non-preemptive, tutti i processi sono aggiunti alla coda dei processi ready e quando un processo smette di essere eseguito, si passa al processo che ha aspettato di più nella coda.



Il FCFS favorisce i processi che usano molto la CPU (CPU-bound) ma non è molto efficiente perché un processo "corto" deve attendere molto prima di andare in esecuzione;

- Il *Round-Robin* usa la preemption, basandosi su un clock. Talvolta viene chiamato time slicing (tempo a fette), perché ogni processo ha una fetta di tempo.



Un'interruzione di clock viene generata ad intervalli periodici. Quando l'interruzione di clock arriva, il processo attualmente in esecuzione viene rimesso nella coda dei ready e viene selezionato il prossimo processo. Ovviamente, se il processo in esecuzione arriva ad un'istruzione di I/O prima dell'interruzione, allora viene spostato nella coda dei blocked.

Con il round-robin, i processi CPU-bound sono favoriti perché usano tutto (o quasi) il loro quanto di tempo, invece gli I/O bound ne usano solo una porzione, fino alla richiesta di I/O. Ne risulta che il round-robin non è equo, oltre che inefficiente per l'I/O. La soluzione consiste nell'implementazione del *round-robin virtuale*, dove dopo un completamento di I/O, il processo non va in coda ai ready, ma va in una nuova coda che ha priorità su quella dei ready, però, solo per la porzione del quanto che ancora gli rimaneva da completare (vedi Figura 4.4).

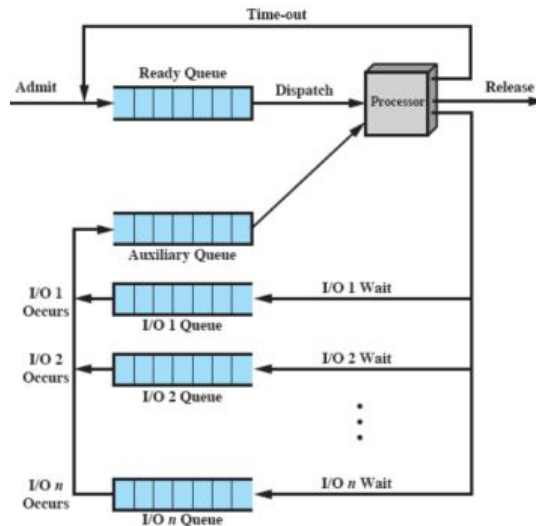
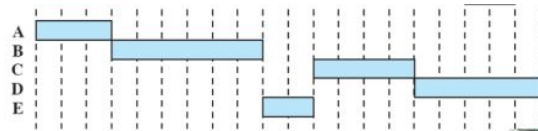


Figura 4.4: Round-robin virtuale

- Lo *SPN* (Shortest Process Next) è non-preemptive e prevede che il prossimo processo da mandare in esecuzione è quello più breve, dove per "breve" si intende il processo, tra quelli ready, il cui tempo di esecuzione stimato è minore. Quindi in sostanza i processi corti scavalcano quelli lunghi.



Con lo *SPN* la predicibilità dei processi lunghi è ridotta ovvero, è più difficile dire quando andranno in esecuzione perché dipende molto da come si comportano gli altri processi nel sistema. Inoltre, se il tempo di esecuzione stimato si rivela inesatto, il sistema operativo può abortire il processo e i processi lunghi potrebbero soffrire di starvation.

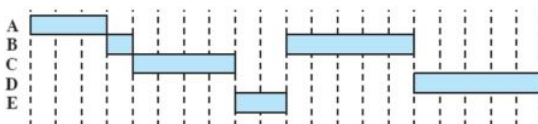
Per stimare il tempo di esecuzione dei processi si parte dal concetto che in alcuni sistemi ci sono processi (sia batch che interattivi) che sono eseguiti svariate volte. Allora, per questi tipi di processi, si può usare il passato (T_i) per predire il futuro (S_i). Una prima soluzione può essere svolta attraverso la media dei $T_1, \dots, T_n$ tempi passati:

$$S_{n+1} = \frac{1}{n} \sum_{i=1}^n T_i \quad S_{n+1} = \frac{1}{n} T_n + \frac{n-1}{n} S_n$$

Questa formula, però, dà lo stesso peso a tutte le istanze, quando invece sarebbe meglio far pesare di più quelle più recenti (exponential averaging):

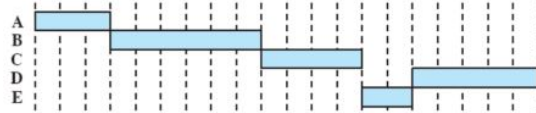
$$S_{n+1} = \alpha T_n + (1 - \alpha) S_n \quad S_{n+1} = \alpha T_n + \dots + \alpha(1 - \alpha)^i T_{n-i} + \dots + (1 - \alpha)^n S_1$$

- Lo *SRT* (Shortest Remaining Time) è come lo *SPN* ma è preemptive. In questo caso l'algoritmo non tiene conto del time quantum, perché un processo può essere interrotto solo quando ne arriva uno nuovo, appena creato. Quindi viene effettuata una stima del tempo richiesto per l'esecuzione, e viene scelto quello più breve.



- Nel *HRRN* (Highest Response Ratio Next) viene massimizzato il rapporto tra il tempo trascorso in attesa e il tempo totale richiesto dal processo con la seguente formula:

$$\frac{w + s}{s} = \frac{\text{tempo trascorso in attesa} + \text{tempo totale richiesto}}{\text{tempo totale richiesto}}$$



4.3 Scheduling tradizionale di UNIX

Nei moderni scheduler, gli algoritmi visti precedentemente non vengono più utilizzati in modo puramente isolato ma si tende a combinare diversi approcci al fine di sfruttare al meglio le caratteristiche specifiche dei diversi algoritmi. Questo è il caso dello scheduling tradizionale di UNIX che combina il concetto di proprietà con il round robin. In questo schema, un processo resta in esecuzione per al massimo un secondo, a meno che non termini o si blocchi, e ci sono diverse code, a seconda dei livelli di priorità, dove viene effettuato il round robin. Le priorità vengono ricalcolate ogni secondo e le priorità iniziali sono basate sul tipo di processo: i processi del sistema operativo partono con priorità più alte rispetto ai processi utente. Per il calcolo delle priorità viene utilizzata una particolare formula, $P_j(i)$ (priority), che indica la priorità del processo j all'inizio di i :

$$CPU_j(i) = \frac{CPU_j(i-1)}{2} \quad P_j(i) = Base_j + \frac{CPU_j(i)}{2} + nice_j$$

$CPU_j(i)$ rappresenta la misura di quanto il processo j ha usato il processore nell'intervallo i , con exponential averaging dei tempi passati, nel caso dei processi running viene incrementato di 1 ogni $\frac{1}{60}$ s. Con $Base_j$ si indica il valore iniziale di priorità basato sul tipo di processo e con $nice_j$ un processo può auto-declassarsi come avente bassa priorità.

Esempio 4.3.1 – Scheduling in Unix

Supponiamo di avere tre processi con $Base_A = Base_B = Base_C = 60$ e $nice_A = nice_B = nice_C = 0$, vediamo come cambia la priorità durante l'esecuzione dei tre processi.

Time	Process A		Process B		Process C	
	Priority	CPU count	Priority	CPU count	Priority	CPU count
0	60	0	60	0	60	0
		1		1		
		2		2		
		•		•		
		•		•		
		60		60		
1	75	30	60	0	60	0
				1		
				2		
				•		
				•		
				60		
2	67	15	75	30	60	0
						1
						2
						•
						•
						60
3	63	7	67	15	75	30
		8				
		9				
		•				
		67				
4	76	33	63	7	67	15
				8		
				9		
				•		
				67		
5	68	16	76	33	63	7

Figura 4.5: Esempio di scheduling in UNIX

A partire dal secondo 0 tutti e tre i processi hanno priorità 60 e viene eseguito il processo A che incrementa il suo valore di CPU count fino a 60 (il processo viene eseguito per tutto il quanto di tempo). Quando arriva l'interrupt per la scelta del processo da eseguire vengono

ricalcolati, per tutti e tre i processi, i nuovi valori di $CPU_j(i) = \frac{CPU_j(i-1)}{2} = \frac{60}{2} = 30$ e $P_j(i) = Base_j + \frac{CPU_j(i)}{2} + nice_j = 60 + \frac{30}{2} + 0 = 75$.
Vengono ripetuti questi passaggi per ogni secondo fino alla terminazione dei processi.

4.4 Scheduling su architetture multiprocessore

Nelle architetture multi-processore e/o multi-core, le singole CPU condividono la RAM e c'è un solo sistema operativo che controlla tutti i processi. In questo tipo di sistemi la scelta del processo da assegnare al processore_k può avvenire per assegnamento statico o per assegnamento dinamico. Nel primo caso quando un processo viene creato, gli viene assegnato un processore fino alla sua terminazione. Si può quindi utilizzare uno scheduler (come quelli visti precedentemente) per ogni processore con il vantaggio di essere di semplice realizzazione e con poco overhead, ma anche con la possibilità che un processore può rimanere idle. Con l'assegnamento dinamico, invece, un processo, nel corso della sua vita, potrà essere eseguito su processori diversi.

4.5 Scheduling in Linux

In Linux si cerca la massima velocità di esecuzione, tramite semplicità di implementazione, per questo motivo non ci sono né long-term né medium-term scheduler. Nel sistema operativo di Linux ci sono delle *runqueues*, sono le code da cui pesca il dispatcher (code dei Ready), e le *wait queues*, sono le code in cui i processi sono messi in attesa quando fanno una richiesta (code dei Blocked). Al contrario delle *runqueues*, le *wait queues* sono condivise tra processori.

Lo scheduling in Linux segue quello di UNIX con il preemptive e la priorità dinamica, a cui vengono aggiunte importate correzioni per essere più veloce possibile e per servire nel modo appropriato i processi real-time. La preemption può essere dovuta a 2 casi: o si esaurisce il quanto di tempo del processo attualmente in esecuzione o un altro processo passa da uno degli stati Blocked a Running.

Linux, inoltre, istruisce l'hardware di mandare un timer interrupt ogni 1ms e quindi il quanto di tempo per ciascun processo è multiplo di 1ms. Per scegliere il valore del quanto di tempo, Linux fa una distinzione tra processi:

- I *processi interattivi* (ad esempio azioni con mouse o tastiera) vengono fatti eseguire al massimo dopo 150ms, altrimenti l'utente percepisce una mancata risposta;
- I tempi dei *processi batch* (non interattivi) sono personalizzati dallo scheduler in quanto non c'è la necessità di una risposta immediata;
- I *processi real-time* (ad esempio riproduttori di audio e video) vengono riconosciuti da Linux solo con un'opportuna chiamata alla system call `sched_setscheduler`.

Ci sono essenzialmente tre classi di scheduling: SCHED_FIFO e SCHED_RR per i processi real-time e SCHED_OTHER per tutti gli altri. Verranno eseguiti prima i processi in SCHED_FIFO e SCHED_RR, poi quelli in SCHED_OTHER, infatti le prime due classi hanno un livello di priorità da 1 a 99, la terza da 100 a 139. Quindi ci sono 140 runqueues (una coda con priorità 0 è usata per casi particolari) per ogni CPU. I processi real-time non cambiano mai la priorità, inoltre un processo SCHED_FIFO non viene mai interrotto tranne se si blocca per I/O o se un processo con priorità maggiore passa da uno degli stati Blocked a Running.

Capitolo 5

Gestione della memoria

Cosa si intende per gestione della memoria (RAM) e perché gestirla? La memoria oggi è a basso costo, e con trend in diminuzione, tuttavia, ciò è più che bilanciato dal fatto che le moderne applicazioni richiedono sempre maggiore memoria. Se si lasciasse a ciascun processo la gestione della propria memoria, ogni processo userebbe semplicemente tutta la memoria a disposizione. Si potrebbe allora imporre dei limiti di memoria a ciascun processo ma diventa difficile per un programmatore scrivere un programma che rispetti tali limiti. Quindi i compiti di gestione della memoria passano al sistema operativo che illude i processi di avere tutta la memoria a disposizione eseguendo degli swap (scambi) di blocchi di dati nella memoria secondaria. Questa gestione di I/O è ovviamente più lenta del processore, pertanto il sistema operativo deve pianificare lo swap in modo intelligente, così da massimizzare l'efficienza del processore e gestire la memoria affinché ci sia sempre un numero ragionevole di processi pronti all'esecuzione.

5.1 Requisiti per la gestione della memoria

I requisiti essenziali per la gestione nella memoria sono: rilocalizzazione, protezione, condivisione, organizzazione logica e organizzazione fisica. Vediamoli in dettaglio:

- Il concetto di *rilocalizzazione* prevede che il programmatore non deve sapere in quale zona della memoria il programma viene caricato. Infatti potrebbe essere swappato su disco, e al ritorno in memoria principale potrebbe essere in un'altra posizione, potrebbe avere alcune pagine in RAM e altre su disco... I riferimenti alla memoria, inoltre, devono essere tradotti nell'indirizzo fisico "vero" attraverso una fase di preprocessing o a run-time.

Possiamo intendere un programma come formato da una zona per il programma scritto in linguaggio macchina, una per i dati (variabili globali) e una per lo stack delle chiamate (variabili locali), vedi Figura 5.1. Gli indirizzi del programma possono essere o istruzioni di salto (Branch instruction, ad esempio `jump`) o riferiti ai dati (Reference to data, ad esempio `lw`, `sw`...). Tutti questi indirizzi devono essere ricalcolati se il sistema operativo vuole garantire la rilocalizzazione.

Ogni programma, in compilazione, viene diviso in una serie di moduli compilati separatamente che vengono uniti da un *linker* per creare un programma eseguibile chiamato *Load module*. Il load module può essere direttamente caricato in RAM tramite un *Loader*, vedi Figura 5.1. A questo punto gli indirizzi di memoria vengono determinati o nel momento in cui il programma viene caricato in memoria (*preprocessing*) o a *run-time*, ovvero nel momento in cui si fa un riferimento alla memoria. Gli indirizzi possono essere di tre tipi: *indirizzi logici* dove il riferimento in memoria è indipendente dall'attuale posizionamento del programma in memoria, *indirizzi relativi* dove il riferimento è espresso come uno spiazzamento rispetto ad un qualche punto noto e *indirizzi fisici o assoluti* dove il riferimento è quello effettivo alla locazione di memoria.

Esempio 5.1.1 – Rilocalizzazione di indirizzi relativi

Ipotizziamo di eseguire l'istruzione `jump` all'indirizzo 400, essendo un indirizzo relativo, l'hardware del compilatore "sa" che quel valore deve essere sommato al Base register per ottenere il riferimento effettivo.

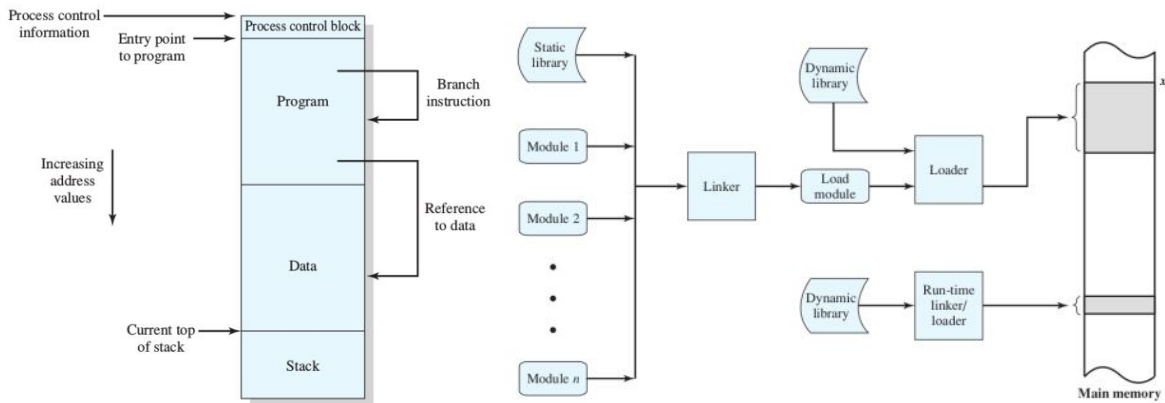
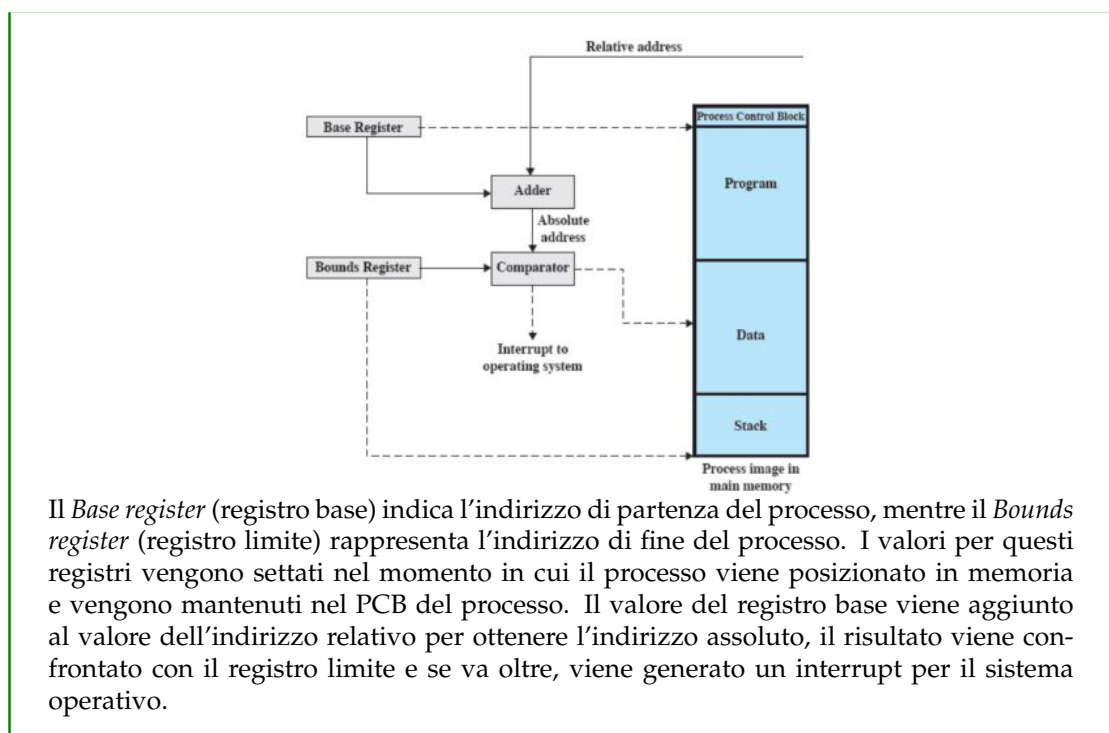


Figura 5.1: Indirizzi nei programmi



- Per *protezione* si intende che i processi non devono poter accedere a locazioni di memoria di un altro processo, a meno che non siano autorizzati. A causa della rilocalizzazione, deve essere necessariamente fatto a tempo di esecuzione con aiuto hardware.
- Il contrario della protezione è la *condivisione*, che deve permettere a più processi di accedere alla stessa zona di memoria, ovviamente solo se è effettivamente utile allo scopo perseguito dai processi. Un caso tipico è quando si creano più processi eseguendo lo stesso codice sorgente e, finché questi processi restano in esecuzione, è più efficiente che condividano il codice sorgente, visto che è lo stesso.
- A livello hardware, le memorie sono organizzate in modo lineare, ma a livello software, i programmi sono scritti in moduli, che possono essere compilati separatamente, avere diversi permessi... Secondo l'*organizzazione logica* il sistema operativo deve essere un tramite tra la visione ad alto livello dei programmatori e quella hardware delle memorie.
- L'*organizzazione fisica* è un altro compito del sistema operativo che deve gestire il flusso tra RAM e memoria secondaria.

5.2 Partizionamento della memoria

In questa sezione verranno presentate diversi tipi di strategie che sono state utilizzate nel corso degli anni per la gestione della memoria, fino ad arrivare alla memoria virtuale che è quella utilizzata ancora oggi.

Uno dei primi metodi per la gestione della memoria è il *partizionamento* che può essere di vari tipi: fisso, dinamico, semplice, segmentazione semplice, paginazione con memoria virtuale, segmentazione con memoria virtuale. È utile comprendere queste tecniche di partizionamento perché rappresentano le fondamenta della gestione della memoria, da cui la memoria virtuale stessa trae origine.

Partizionamento fisso

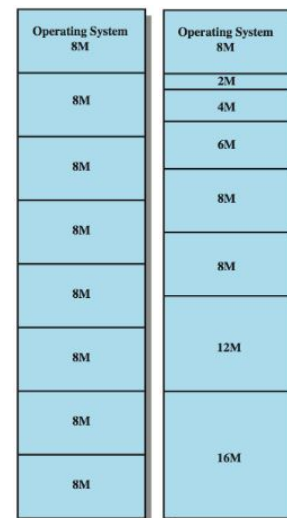
Nel *partizionamento fisso uniforme*, dopo l'accensione del sistema, la memoria RAM viene divisa in partizioni di uguale lunghezza. Se un processo ha una dimensione minore o uguale della misura di una partizione, allora può essere caricato in una partizione libera. Il sistema operativo può togliere un processo da una partizione e spostarlo su disco.

Da questo tipo di partizionamento nascono notevoli problematiche date dal fatto che un programma potrebbe non entrare in una partizione oppure c'è un uso inefficiente della memoria perché ogni programma, anche il più piccolo, occupa un'intera partizione, causando frammentazione interna¹.

Con il *partizionamento fisso variabile* vengono mitigati alcuni problemi del partizionamento fisso uniforme. Infatti la memoria viene sempre divisa in più parti fisse ma ogni partizione ha una dimensione diversa (maggiore o minore) rispetto alle altre.

E' necessario, in questo tipo di partizionamento, un algoritmo di posizionamento per decidere in quale blocco collocare il nuovo processo. La soluzione più ovvia è inserire un processo nella partizione più piccola che può contenerlo in modo da minimizzare la quantità di spazio sprecato.

Restano comunque dei problemi irrisolti dato che c'è un numero massimo di processi in memoria principale, corrispondente al numero di partizioni deciso inizialmente, e se ci sono molti processi piccoli, la memoria verrà usata in modo inefficiente.



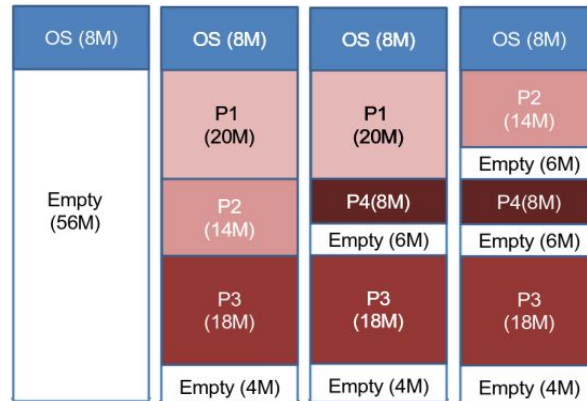
Partizionamento dinamico

Visti i problemi incontrati nel partizionamento fisso, si è deciso di passare al *partizionamento dinamico*, dove le partizioni variano sia in misura che in quantità e per ciascun processo viene allocata esattamente la quantità di memoria che serve.

Esempio 5.2.1 – Partizionamento dinamico

In un sistema operativo con 64M di RAM con partizionamento dinamico abbiamo 8M di memoria per il kernel e i restanti 56M sono vuoti. In un certo istante di tempo arrivano in sequenza tre processi: P_1 da 20M, P_2 da 14M e P_3 da 18M che vengono allocati in tre partizioni diverse. Supponiamo che arrivi un processo P_4 da 8M, non essendoci abbastanza spazio per una nuova partizione, il sistema operativo decide che P_4 è più importante di P_2 che lascia parte del suo spazio al nuovo processo. Ovviamente P_2 non viene terminato ma viene copiato su disco finché il sistema operativo non decide di riportarlo in RAM.

¹Quando un programma riceve un certo quantitativo di memoria, ma non utilizza completamente tutto lo spazio assegnato, la porzione di memoria inutilizzata all'interno di quel blocco è chiamata frammentazione interna.



Da notare che nel caso in cui arriva un nuovo processo da 8M, c'è abbastanza memoria ma non essendo contigua bisogna swappare su disco un processo.

Anche con questo tipo di partizione ci problemi dovuti alla frammentazione esterna² che si può risolvere compattando i processi di modo che siano contigui. Inoltre, il sistema operativo deve decidere a quale blocco libero assegnare un processo. L'algoritmo più ovvio è il *best-fit* che sceglie il blocco la cui misura è la più vicina a quella del processo da posizionare, nonostante l'apparenza, è quello con risultati peggiori perché lascia dei frammenti molto piccoli e costringe a fare spesso la compattazione. Un'alternativa è il *first-fit* che scorre la memoria dall'inizio e il primo blocco con abbastanza memoria viene subito scelto, è molto veloce anche se tende a riempire solo la prima parte della memoria e in media è la soluzione migliore. Un'ultima possibilità sarebbe il *next-fit* che opera come il *first-fit*, ma anziché partire ogni volta dall'inizio, parte dall'ultima posizione assegnata ad un processo e tende ad assegnare più spesso il blocco alla fine della memoria, che è quello più grosso.

Buddy System

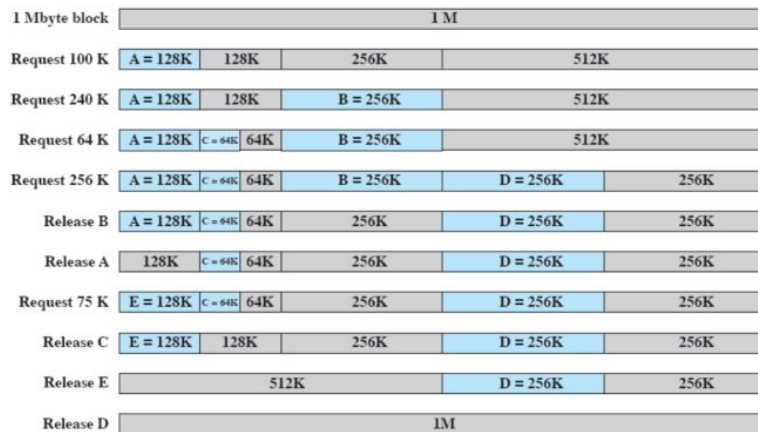
Il *Buddy System* (sistema del compagno) è un compromesso tra partizionamento fisso e dinamico. Sia 2^U la dimensione dello user space ed s la dimensione di un processo da mettere in RAM, si dimezza lo spazio fino a trovare un X tale che $2^{X-1} < s \leq 2^X$, con $L \leq X \leq U$, L serve per dare un lower bound in modo da non creare partizioni troppo piccole. In questo modo si trovano due diverse partizioni di cui una è utilizzata per il processo. Ovviamente, in questo sistema, occorre tener presente le porzioni già occupate e quando un processo finisce, se il buddy è libero si può fare una fusione.

Esempio 5.2.2 – Buddy System

In un sistema con 1M arriva un processo A che richiede 100K, vengono create le partizioni 512K, 256K e 128K e viene inserito il processo nella partizione da 128K. Supponiamo che arrivino tre processi B, C e D da 240K, 64K e 256K il primo e l'ultimo vengono inseriti rispettivamente nei blocchi da 128K e 256K e, per il secondo, viene effettuata la divisione del blocco di 128K in due da 64K e inserito il processo al suo interno.

Ora nel caso in cui vengono terminati i processi A e B, si dovrebbero fondere i blocchi liberati con i loro compagni (buddy) per creare delle porzioni più grandi, ma in entrambe i casi non è fattibile perché non vengono creati blocchi liberi da 256K per A e 512K per B. Quindi vengono semplicemente liberate delle partizioni di memoria per il nuovo processo E da 75K. Diversamente accade quando vengono terminati i processi C ed E, perché vengono fusi i quattro blocchi liberi di memoria in uno da 512K.

²Si verifica quando ci sono zone di memoria libera tra le regioni occupate dai processi, ma tali regioni libere non sono sufficientemente grandi o contigue per soddisfare le richieste di memoria di un nuovo processo.



5.3 Paginazione e segmentazione

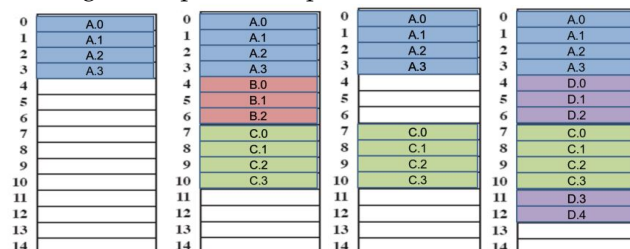
Per arrivare alla paginazione e segmentazione moderne, ovvero con memoria virtuale, dobbiamo prima analizzare quelle semplici.

Paginazione semplice

La paginazione semplice probabilmente non è mai stata usata, ma è importante per introdurre la memoria virtuale. In questo sistema sia la memoria che i processi vengono partizionati in piccoli pezzi di grandezza uguale, i pezzi di processi sono chiamati *pagine* e i pezzi di memoria sono chiamati *frame*. Ogni pagina, per essere usata, dev'essere collocata in un frame e, in generale, una pagina può essere posizionata in un qualunque frame. E' necessario quindi che i sistemi operativi mantengano una *tabella delle pagine* per ogni processo dove, per ogni pagina del processo, questa tabella indica in quale frame effettivo si trova, quando si genera un process switch, la tabella delle pagine del nuovo processo deve essere ricaricata. A questo punto, gli indirizzi di memoria relativi vengono modificati a favore di indirizzi contenenti un numero di pagina e uno spiazzamento al suo interno.

Esempio 5.3.1 – Paginazione semplice

La RAM è divisa in 15 frame e, in un certo istante di tempo, arrivano 3 processi A,B e C con rispettivamente 4,3 e 4 pagine che richiedono altrettanti frame per essere caricati. Supponiamo che termina il processo B e arriva un processo D con 5 pagine, se stessi usando il partizionamento dinamico il processo non sarebbe caricato senza compattazione, con la paginazione questo problema non sussiste perché le prime 3 pagine vengono collocate nei 3 frame disponibili e le altre 2 a seguire dopo il terzo processo.



Esempio 5.3.2 – Traduzione della tabella delle pagine

Supponiamo che la dimensione di una pagina sia 100 bytes e che la RAM dell'esempio precedente è di 1400 bytes in totale. Inoltre, i processi A, B, C e D richiedono rispettivamente 400, 300, 400 e 500 bytes. Nelle istruzioni dei processi, i riferimenti alla RAM sono relativi all'inizio del processo, quindi, ad esempio, per D ci saranno solo riferimenti compresi nell'intervallo [0, 499].

Supponiamo ora che attualmente il processo D sia in esecuzione, e che occorra eseguire l'istruzione $j = 343$, bisogna prima capire in quale pagina di D si trova 343, basta fare $343/100 = 3$. Poi occorre guardare la tabella delle pagine di D, la 3^a pagina corrisponde all'11° frame di RAM. Il frame 11 ha indirizzi che vanno da 1100 a 1199: basta fare $343 \bmod 100 = 43$, l'indirizzo vero è pertanto $11 * 100 + 43 = 1143$.

Osservazione. Con l'hardware dei calcolatori, bisogna usare la base 2, quindi anche la dimensione delle pagine è una potenza di 2.

Con la paginazione, gli indirizzi ci forniscono due informazioni: una prima parte di bit indica il numero di pagina nella tabella delle pagine e una seconda parte indica l'offset. Basterà quindi affiancare il numero del frame con quello dell'offset per ottenere l'indirizzo fisico.

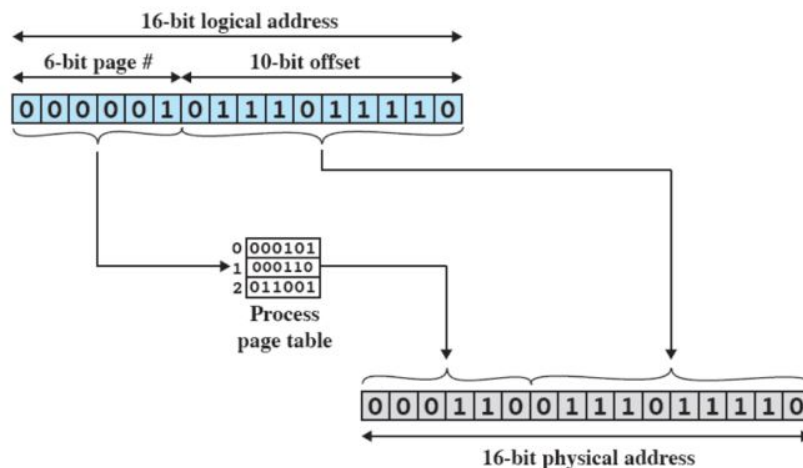


Figura 5.2: Esempio di indirizzi di paginazione

Segmentazione semplice

La segmentazione prevede che un programma può essere diviso in segmenti con lunghezza variabile e un limite massimo della dimensione. In questo senso è simile al partizionamento dinamico ma con una differenza fondamentale: il programmatore (o il compilatore) devono gestire esplicitamente la segmentazione dicendo quanti segmenti ci sono e qual è la loro dimensione per metterli effettivamente in RAM.

Per tradurre un indirizzo logico in uno fisico con la segmentazione, uso i primi bit dell'indirizzo per ottenere il numero di pagina dalla tabella dei segmenti, da questa estraggo il valore di inizio del processo per sommarlo all'offset (ultimi bit dell'indirizzo logico).

5.4 Memoria virtuale: hardware e strutture di controllo

Abbiamo già discusso che i riferimenti alla memoria sono indirizzi logici che vanno tradotti in indirizzi fisici a tempo di esecuzione e, con la paginazione e segmentazione, abbiamo visto che un processo può essere spezzato in più parti, che non necessariamente occuperanno una zona contigua di memoria principale. Se tutte le caratteristiche appena elencate sono vere, allora non occorre che tutte le pagine (o tutti i segmenti) di un processo siano in memoria principale. Infatti, per eseguire un processo, ho bisogno semplicemente che sia caricata in memoria la pagina che contiene l'istruzione da eseguire e eventualmente i dati di cui ha bisogno.

Si è passati così dalla paginazione semplice alla *paginazione con memoria virtuale* dove il sistema operativo porta in memoria principale solo alcuni pezzi (pagine) del programma chiamati *resident set* (insieme residente). Quando un processo accederà ad una zona di memoria che non si trova nel

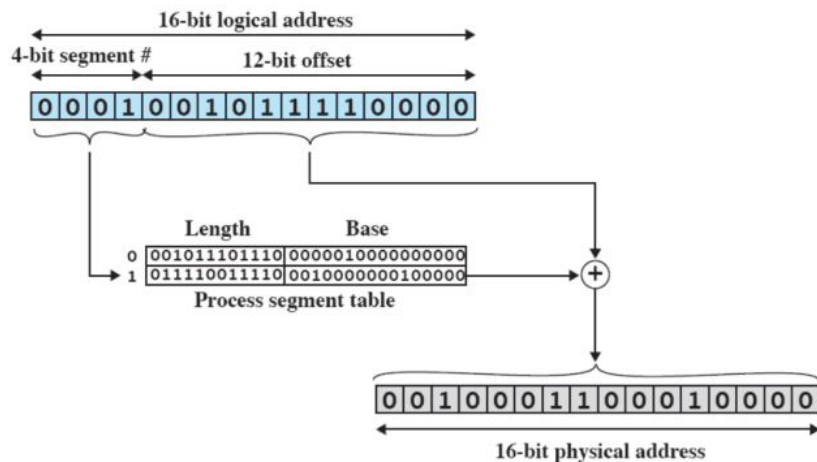


Figura 5.3: Esempio di indirizzi di segmentazione

resident set, verrà generato un interrupt di *page fault*³ e il processo passa in modalità Blocked. Successivamente il pezzo di processo che contiene l'indirizzo logico viene portato in memoria principale, il sistema operativo effettua una richiesta di lettura su disco e, quando l'operazione viene completata, si genera un nuovo interrupt per far tornare il processo Ready.

In definitiva, la **memoria virtuale** è uno schema di allocazione di memoria, in cui la memoria secondaria può essere usata come se fosse principale. Gli indirizzi usati nei programmi e quelli usati dal sistema sono diversi e c'è una fase di traduzione automatica dai primi nei secondi.

Tutto ciò permette la presenza di molti più processi in memoria principale (non necessariamente interi) rispetto alla paginazione e segmentazione semplici. Questo vuol dire che il processore è usato al meglio perché è molto probabile che ci sia sempre almeno un processo Ready caricato in memoria. Grazie a questi aspetti fondamentali, la memoria virtuale è tuttora il sistema principe utilizzato dai sistemi operativi per la gestione della memoria.

Località e memoria virtuale

Un effetto collaterale che si può generare con l'utilizzo della memoria virtuale è il *thrashing* (letteralmente bastonatura o sconfitta), quando il sistema operativo impiega la maggior parte del suo tempo a swappare pezzi di processi, anziché ad eseguire istruzioni, ovvero quando quasi ogni richiesta di pagina dà luogo ad un page fault. Per evitarlo, il sistema operativo cerca di indovinare quali pezzi di processo saranno usati con minore o maggiore probabilità in futuro, sulla base dello storico recente.

Secondo il *principio di località*, i riferimenti necessari ad un processo tendono ad essere vicini, sia che si tratti di dati che di istruzioni. Quindi solo pochi pezzi di processo saranno di volta in volta caricati, permettendo di prevedere abbastanza bene quali pezzi di processo saranno necessari nel prossimo futuro.

Esempio 5.4.1 – Pagine e località

Nell'esempio un po' datato di Figura 5.4, si nota bene che al passare del tempo (da sinistra a destra), i numeri delle pagine che vengono richieste (del basso all'alto) si trovano tutte in una regione comune. Ovvero, preso un istante di tempo, i riferimenti delle pagine richieste sono tutti vicini tra loro.

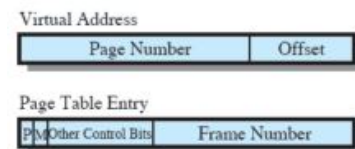
³E' un'eccezione generata quando un processo cerca di accedere a una pagina che è presente nel suo spazio di indirizzamento virtuale, ma che non è presente nella memoria fisica poiché mai stata caricata o perché precedentemente spostata su disco.



Figura 5.4: Esempio di località delle pagine

Paginazione con memoria virtuale

Abbiamo visto come ogni processo ha una sua tabella delle pagine e il control block di un processo punta a tale tabella. Ogni entry della tabella contiene delle informazioni aggiuntive: un bit di presenza (P) indica se la pagina è in RAM o se bisogna prelevarla dal disco; un bit di modifica (M) indica se la pagina è stata modificata in seguito all'ultima volta che è stata caricata in memoria principale; altri bit di controllo (Other Control Bits); il numero di frame (Frame Number).



In maniera più approfondita, la traduzione degli indirizzi è fatta dall'hardware, partendo da un indirizzo virtuale da cui si ottiene il numero di pagina e l'offset. Il numero di pagina va sommato con il Page Table Ptr, che indica il punto di partenza della page table, e moltiplicato per il numero di bytes di ogni singola entry della tabella delle pagine. In questo modo si ottiene il numero di frame che sostituisce il numero di pagina per ottenere l'indirizzo fisico in RAM (vedi Figura 5.5).

Un problema che potrebbe generarsi è che le tabelle delle pagine potrebbero contenere molti elementi, ad esempio con RAM di 1 GB, bastano 20 processi per occupare più di metà RAM con sole strutture di overhead. L'idea è di utilizzare delle tabelle delle pagine a due livelli, dove una tabella delle pagine di livello superiore punta alle tabelle delle pagine del processo. In questo sistema la traduzione degli indirizzi si complica leggermente perché l'indirizzo virtuale si spacchetta in tre parti e l'hardware deve consultare due tabelle delle pagine. La prima è indicizzata dalla prima parte dell'indirizzo e, il valore che si ottiene, va sommato con la seconda parte dell'indirizzo, che viene usato per accedere alla seconda tabella delle pagine (vedi Figura 5.5).

Spesso, con la memoria virtuale, viene utilizzata una memoria cache chiamata *Translation Lookaside Buffer* (TLB) che contiene gli elementi delle tabelle delle pagine che sono stati usati più di recente. Questa necessità nasce dal fatto che ogni riferimento alla memoria virtuale può generare due accessi alla memoria: uno per la tabella delle pagine e uno per prendere il dato. Quindi dato un indirizzo virtuale, il processore esamina dapprima il TLB, se la pagina è presente (*TLB HIT*), si prende il frame number e si ricava l'indirizzo reale. Altrimenti (*TLB MISS*), si prende la tabella delle pagine del processo se è presente in memoria, altrimenti si gestisce il page fault come già visto. Dopodiché, il TLB viene aggiornato includendo la pagina appena acceduta usando un qualche algoritmo

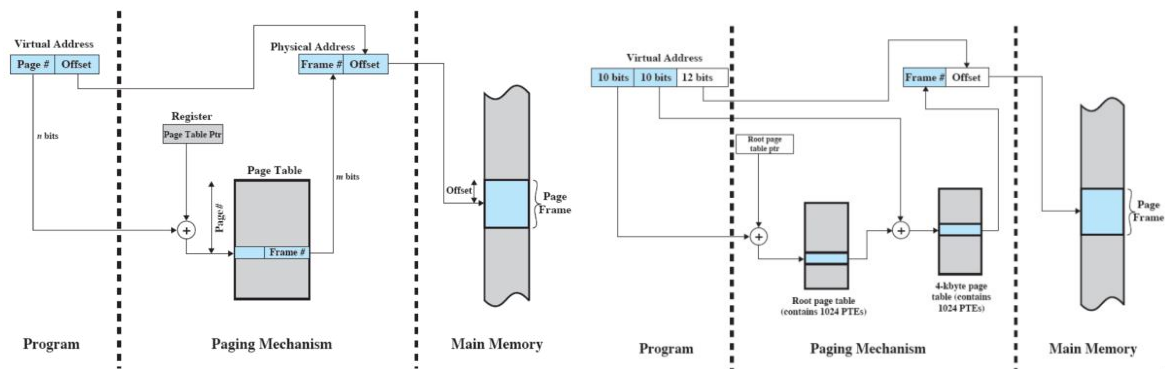


Figura 5.5: Traduzione di indirizzi con una e due tabelle delle pagine

di rimpiazzamento se il TLB è già pieno: solitamente LRU⁴.

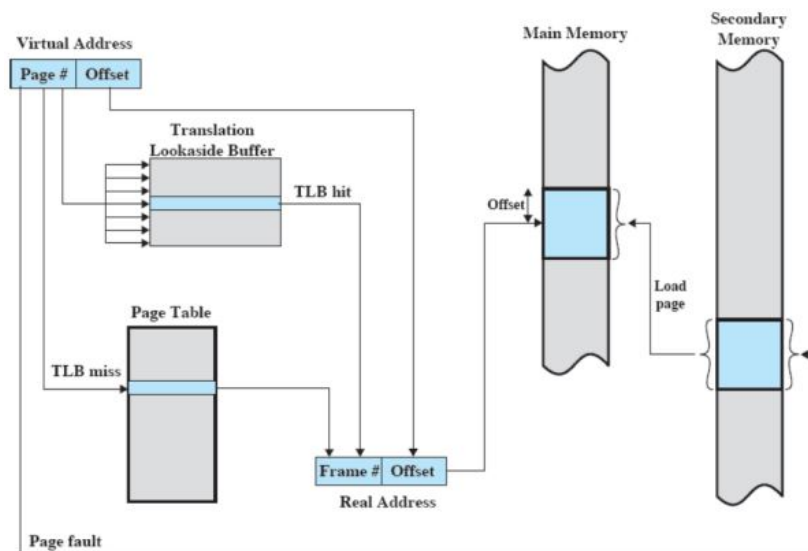


Figura 5.6: Funzionamento del TLB

Il TLB contiene solo alcuni elementi tratti dalle tabelle delle pagine, per questo, per trovare una specifica pagina, il sistema operativo può interrogare più elementi del TLB contemporaneamente per capire se è presente il numero di pagina desiderato. Per fare ciò c'è bisogno che il TLB contenga solo pagine in RAM.

Per quanto riguarda la dimensione delle pagine possiamo dire che più piccola è una pagina, minore è la frammentazione all'interno delle pagine ma è anche maggiore il numero di pagine per processo, il che significa che è più grande la tabella delle pagine per ogni processo. Però, allo stesso tempo, più piccola è una pagina, maggiore è il numero di pagine che si trovano in memoria principale e lo stesso processo può riuscire ad avere un resident set più grande in modo da diminuire i page fault. D'altro canto la memoria secondaria è ottimizzata per trasferire grossi blocchi di dati, e quindi per avere pagine ragionevolmente grandi. Data questa forte dualità sono stati fatti degli esperimenti per cercare il migliore valore di dimensione delle pagine e si è notato che la migliore scelta è utilizzare dimensioni delle pagine piccole.

⁴Least Recently Used, usato meno di recente.

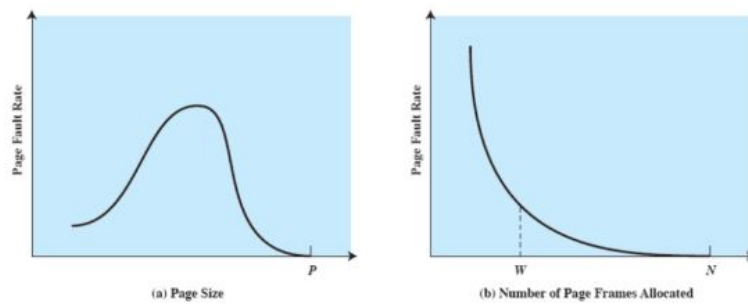
Osservazione 5.4.1 – Dimensione delle pagine

Figura 5.7: Esperimenti sulle dimensioni delle pagine

Il grafico di sinistra presenta la dimensione della pagina sulle ascisse e il tasso di page fault sulle ordinate (si punta ad avere questo valore più basso possibile). Il grafico mostra un andamento a campana dove per dimensioni di pagina piccole o molto vicine alla grandezza del processo il page fault rate tende a rimanere basso, mentre verso metà processo cresce notevolmente. In realtà con pagine grandi, si ottengono pochi fault di pagina, ma allo stesso tempo si ha poca multiprogrammazione. Per questo motivo si è preferito adottare dimensioni delle pagine piccole.

Il grafico di destra presenta il numero di page frame necessari per il processo sulle ascisse e il tasso di page fault sulle ordinate. Ovviamente, se si mettono in RAM tutte le pagine del processo (N) non vengono generati page fault, ma allo stesso tempo non si ha lo spazio per tutti gli altri processi. Un ottimo compromesso si ottiene per W.

Segmentazione con memoria virtuale

Abbiamo visto come la segmentazione permette al programmatore di vedere la memoria come un insieme di spazi di indirizzi, di dimensione variabile o dinamica, per semplificare la gestione delle strutture dati, per permettere di modificare e ricompilare i programmi in modo indipendente o per condividere e proteggere i dati.

Ogni processo ha una sua tabella dei segmenti e il control block di un processo punta a tale tabella. Ogni entry di questa tabella contiene: un bit (P) per indicare se il segmento è in memoria principale; un bit (M) per indicare se il segmento è stato modificato in seguito all'ultima volta che è stato caricato in memoria principale; la lunghezza (Length) del segmento; l'indirizzo di partenza (Segment Base) in memoria principale del segmento.



Per la traduzione degli indirizzi, il numero di segmento va sommato con il Seg Table Ptr, che indica il punto di partenza della segment table, e moltiplicato per il valore di length della entry. In questo modo si ottiene l'indirizzo di partenza, in memoria principale, del segmento che sommato con l'offset ci dà l'indirizzo fisico (vedi Figura 5.8).

La paginazione è trasparente al programmatore nel senso che il programmatore non ne è (o non ne deve essere) a conoscenza. La segmentazione, invece, è visibile al programmatore, se programma in assembler, altrimenti ci pensa il compilatore ad usare i segmenti. Tipicamente paginazione e segmentazione sono combinate, nel senso che ogni segmento viene diviso in più pagine. In questo modo si accede, con una parte di indirizzo, alla segment table, e il segmento trovato è paginato da una tabella delle pagine da cui è possibile ottenere il frame in RAM (vedi Figura 5.9).

Con la segmentazione, implementare la protezione viene naturale, dato che ogni segmento ha una base ed una lunghezza, è facile controllare che i riferimenti siano contenuti nel giusto intervallo. Per la condivisione, basta definire un segmento che serve più processi, o al contrario che non ne serve nessuno.

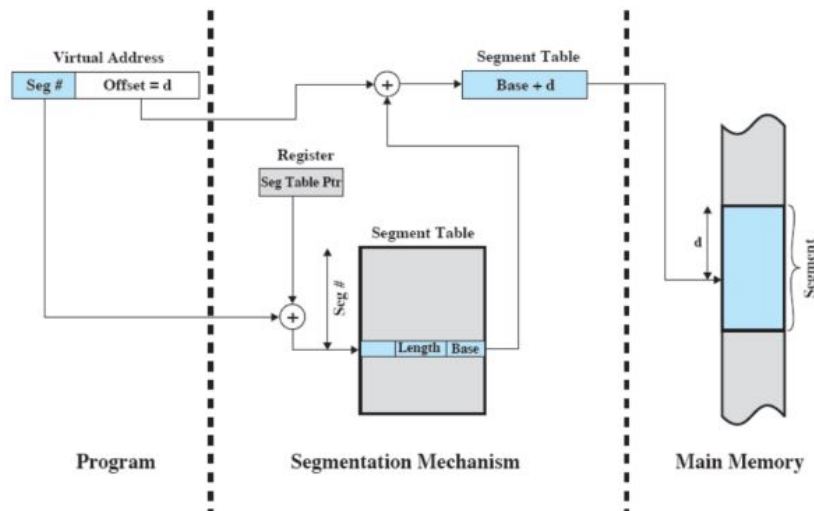


Figura 5.8: Traduzione di indirizzi con la segmentazione

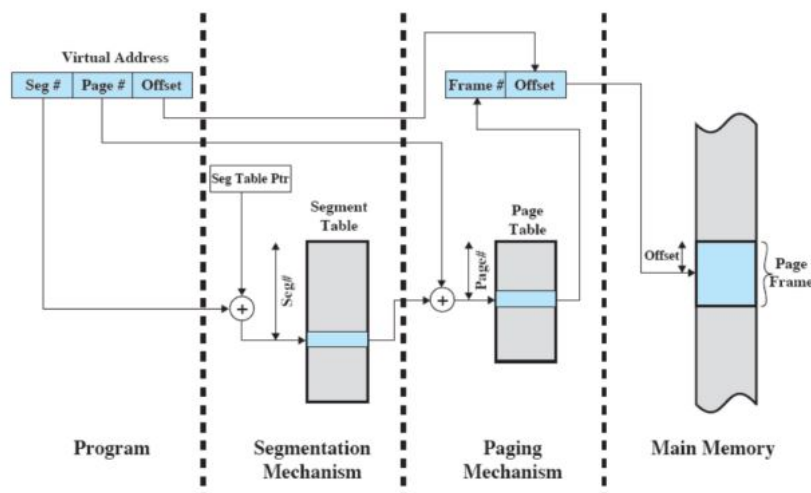
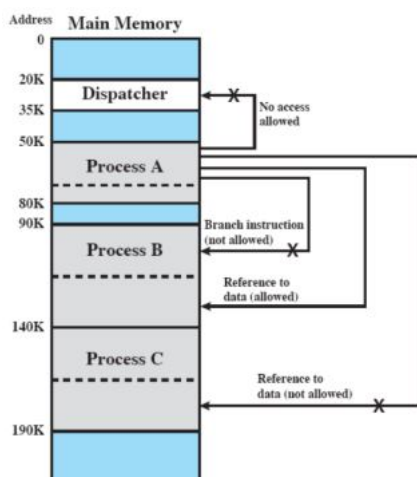


Figura 5.9: Traduzione di indirizzi con paginazione e segmentazione

Esempio 5.4.2 – Protezione e condivisione



Tre processi A, B, C sono divisi in una parte privata (quella più in alto) e in una pubblica (quella più in basso). Ad esempio la parte privata di A non può eseguire un branch nella parte privata di un altro processo, ma può far riferimento ai dati pubblici del processo B.

Sta al programmatore la scelta di quali parti dei processi A, B e C siano condivise e quali no.

5.5 Memoria virtuale e sistema operativo

Quando il sistema operativo deve decidere se utilizzare la paginazione e/o la segmentazione deve definire: una politica di prelievo (*fetch policy*); una politica di posizionamento (*placement policy*); una politica di sostituzione (*replacement policy*); altro (gestione del resident set, politica di pulizia, controllo del carico). Per ogniuna di queste politiche ci sono degli algoritmi che cercano di minimizzare i page fault.

Fetch policy

La politica di prelievo decide quando una pagina debba essere portata in memoria principale. Si usano principalmente due politiche:

- con la paginazione su richiesta (*demand paging*) una pagina viene portata in memoria principale nel momento in cui un qualche processo la richiede, generando molti page fault nei primi momenti di vita del processo;
- la prepaginazione (*prepaging*) cerca di anticipare le richieste del processo portando in memoria principale più pagine di quelle richieste sfruttando il principio di località.

Placement policy

La politica di posizionamento decide dove mettere una pagina in memoria principale quando c'è almeno un frame libero. Tipicamente, si sceglie il primo frame libero ma, in realtà, può essere messa in qualunque spazio libero grazie alla presenza dell'hardware per la traduzione degli indirizzi.

Replacement policy

La politica di sostituzione decide quale pagina sostituire nel caso in cui tutti i frame della RAM sono occupati. Il tutto va fatto in modo da minimizzare la probabilità che la pagina appena sostituita venga subito richiesta di nuovo. Bisogna tener presente che se un frame è bloccato (*Frame Locking*), non si può sostituire, ad esempio i frame a livello di kernel del sistema operativo.

Gli algoritmi di sostituzione che si possono utilizzare sono: sostituzione della pagina usata meno di recente (LRU), sostituzione a coda (FIFO), sostituzione ad orologio (clock).

Esempio 5.5.1 – Algoritmi di sostituzione

Supponiamo di avere 3 frame in memoria principale e la seguente sequenza di richieste delle pagine:

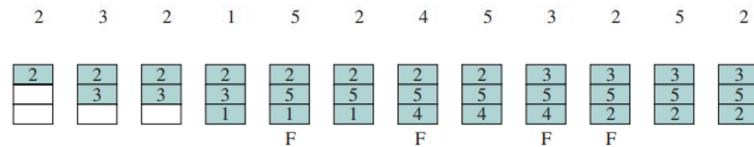
2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2

Analizziamo il comportamento degli algoritmi di sostituzione delle pagine, sulla base della sostituzione ottima che rappresenta la soluzione migliore da utilizzare quando si conosce l'esito di un esperimento:

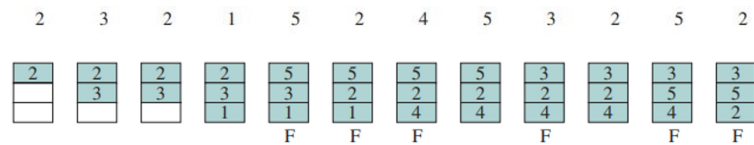
- La *soluzione ottimale* è quella che sostituisce la pagina che verrà richiesta più in là nel futuro. Ovviamente, non è implementabile ma è definita per fare i successivi confronti sperimentali che dovranno avvicinarsi il più possibile ai 3 page fault generati da questo algoritmo;

2	3	2	1	5	2	4	5	3	2	5	2
2	2	2	2	2	2	4	4	4	2	2	2
	3	3	3	3	3	3	3	3	3	3	3
			1	5	5	5	5	5	5	5	5
				F		F			F		

- La *sostituzione LRU* sostituisce la pagina in cui non sia stato fatto riferimento per il tempo più lungo, basandosi sul principio di località, dovrebbe essere la pagina che ha meno probabilità di essere usata nel prossimo futuro. Il problema di questa soluzione è che la sua implementazione è problematica perché occorre etichettare ogni frame con il tempo dell'ultimo accesso;

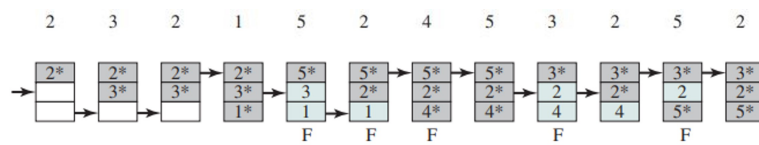


- Nella *sostituzione FIFO* i frame allocati ad un processo sono trattati come una coda circolare, da cui le pagine vengono rimosse a turno (round robin). In questo modo si rimpiazzano le pagine che sono state in memoria per più tempo anche se potrebbero servire nel breve termine;



- La *sostituzione a clock* è un compromesso tra LRU e FIFO, ovvero c'è uno *use bit* per ogni frame, che indica se la pagina è stata riferita di recente. Questo bit è settato ad 1 quando la pagina viene caricata in memoria principale, e poi rimesso ad 1 per ogni accesso al suo interno. Quando occorre sostituire una pagina, il sistema operativo cerca come nella FIFO, e seleziona il frame contenente la pagina che ha per prima lo *use bit* a 0, se nel frattempo incontra una pagina che lo ha a 1, lo mette a 0 e procede con la prossima.

Questo algoritmo è alla base di molti algoritmi di sostituzione delle pagine attualmente utilizzati nei sistemi operativi. Con * viene indicato lo *use bit* a 1, mentre senza è a 0:



Di seguito un confronto grafico sulle prestazioni degli algoritmi analizzati:

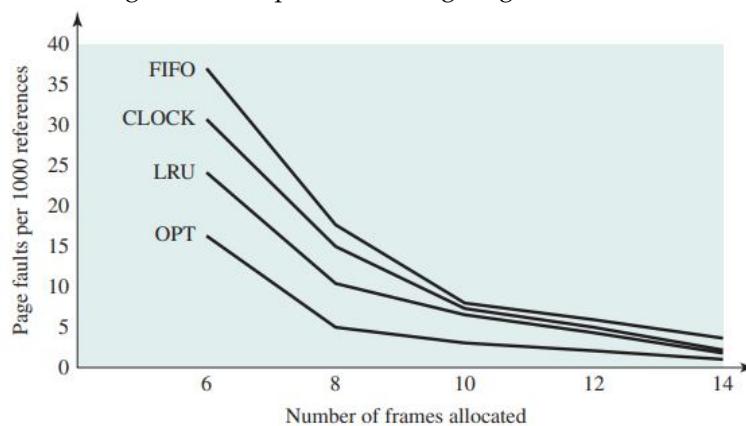


Figura 5.10: Confronto degli algoritmi di sostituzione

Un'altra tecnica che viene spesso utilizzata è il *page buffering*, dove si toglie un pò di memoria al processo per utilizzarla per salvare la pagina in una cache prima di rimpiazzarla. In questo modo se poi viene nuovamente referenziata, si può subito riportarla in memoria.

Gesione del resident set

La gestione del resident set risponde a due necessità:

- per ogni processo in esecuzione, quanti frame di RAM vanno allocati (*resident set management*)? Si possono utilizzare due approcci: o si fa un'allocazione fissa decidendo il numero di frame

necessari durante la creazione di un processo, oppure, con allocazione dinamica il numero di frame varia durante la vita del processo in base alle statistiche raccolte.

- quando si rimpiazza un frame, bisogna scegliere solo tra i frame che appartengono al processo corrente, oppure si può sostituire un frame qualsiasi (*replacement scope*)? Anche in questo caso ci sono due approcci: si utilizza una politica locale rimpiazzando un frame con uno dello stesso processo o, al contrario, con la politica globale si può scegliere un frame di qualsiasi processo.

Politica di pulitura

Se un frame è stato modificato, va riportata la modifica anche sulla pagina corrispondente non appena avviene la modifica oppure non appena il frame viene sostituito. Si fa tipicamente una via di mezzo, intrecciata con il page buffering, dove si raccolgono un pò di richieste di frame da modificare e si eseguono insieme così da essere più efficienti.

Controllo del carico

Il controllo del carico, con il medium-term scheduler, cerca di avere il numero più alto possibile di processi attivi, ma senza arrivare al thrashing, perché se ci sono troppi processi i resident set diventano troppo piccoli e si generano troppi page fault. Si aumentano e diminuiscono i processi attivi gestendo opportunamente gli stati Suspend e Non-Suspend attraverso delle politiche di monitoraggio.

5.6 Gestione della memoria in Linux

In Linux si ha una distinzione netta tra richieste di memoria da parte del kernel e quelle dei processi utente, questo perché Linux si fida del kernel e fa pochi controlli, se non nessuno, per questo tipo di richieste. Per i processi utente, invece, sono necessari dei controlli di protezione e di rispetto dei limiti assegnati. Il kernel può usare sia la memoria a lui riservata che quella normalmente usata dai processi utente e le richieste di memoria del kernel sono ottimizzate sia per le richieste piccole che per le grandi: se la richiesta è piccola (pochi bytes), fa in modo di avere alcune pagine già pronte da cui può prendere i pochi bytes richiesti (slab allocator); se la richiesta è grande (fino a 4MB), fa in modo di allocare più pagine contigue in frame contigui utilizzando il buddy system.

In Linux vengono utilizzate la fetch policy di paging on demand, il placement policy con il primo frame libero e il replacement policy con l'algoritmo dell'orologio, anche se il kernel più recente usa un LRU "corretto". Per l'algoritmo di clock si usano due flag in ogni entry delle page table: PG_referenced (pagine referenziate) e PG_active (pagine attive). Sulla base di PG_active, il kernel mantiene due liste di pagine: attive ed inattive. Il kswapd è un kernel thread in esecuzione periodica, che scorre solo le pagine inattive. Il PG_referenced è settato quando la pagina viene richiesta, dopodiché o arriva prima kswapd o un altro riferimento. Nel primo caso viene settata la pagina attiva, nel secondo il PG_referenced di nuovo a zero. In questo sistema solo le pagine inattive possono essere rimpiazzate.

La gestione del resident set avviene con una politica dinamica con replacement scope globale, anche qui, c'è un buddy system per richieste grandi. La politica di pulitura è ritardata il più possibile e le scritture vengono effettuate o quando la page cache è troppo piena con molte richieste pending, o si hanno troppe pagine sporche da troppo tempo, oppure si ha una richiesta esplicita di un processo (flush, sync o simili). Infine il controllo del carico è totalmente assente.

Capitolo 6

Gestione dell'Input/Output

6.1 Dispositivi di I/O

L'input/output è un'area assai problematica per la progettazione di sistemi operativi perché c'è una grande varietà di dispositivi di I/O e anche una grande varietà di applicazioni che li usano, per questo motivo è difficile scrivere un sistema operativo che tenga conto di tutte queste diversità. Possiamo dividere i dispositivi di I/O in tre categorie:

- i dispositivi *leggibili dall'utente* che sono usati per la comunicazione diretta con l'utente, come stampanti, monitor, tastiera, mouse...
- i dispositivi *leggibili dalla macchina* sono usati per la comunicazione con materiale elettronico, come dischi, chiavi USB, sensori, controllori, attuatori...
- i *dispositivi di comunicazione* sono usati per la comunicazione con dispositivi remoti, come modem, schede Ethernet, Wi-Fi...

Un dispositivo di input prevede di essere interrogato sul valore di una certa grandezza fisica al suo interno. Ad esempio in una tastiera troviamo il codice Unicode degli ultimi tasti premuti, in un mouse le coordinate dell'ultimo spostamento effettuato, su disco il valore dei bit che si trovano in una certa posizione al suo interno. Un processo che effettua una `syscall read` su un dispositivo del genere vuole conoscere questo dato per poterlo elaborare.

Un dispositivo di output prevede di poter cambiare il valore di una certa grandezza fisica al suo interno. Ad esempio un monitor moderno cambia il valore RGB di tutti i suoi pixel, una stampante moderna ottiene un file PDF o PS da stampare, un disco cambia il valore dei bit che devono sovrascrivere quelli che si trovano in una certa posizione al suo interno. Un processo che effettua una `syscall write` su un dispositivo del genere vuole cambiare qualcosa.

Ci sono quindi, principalmente, due `syscall read` e `write` e, tra i loro argomenti, c'è un identificativo del dispositivo da leggere o scrivere. Il kernel, che comanda l'inizializzazione del trasferimento di informazioni, mette il processo in `Blocked` e passa ad altro, questa parte del kernel che gestisce un particolare dispositivo di I/O è chiamata *driver*. A trasferimento completato, si genera l'interrupt che termina l'operazione e il processo ritorna `Ready`.

I dispositivi di I/O possono differire sotto molti aspetti:

- *data rate* indica la frequenza di accettazione/emissione di dati. Possono esserci differenze di diversi ordini di grandezza tra le velocità di trasferimento dati, la Figura 6.1 fornisce alcuni esempi;
- *l'applicazione* a cui viene destinato un dispositivo, ad esempio un disco utilizzato per i file richiede il supporto del software di gestione dei file, un disco utilizzato come archivio di backup per le pagine di memoria virtuale dipende dall'uso dell'hardware e del software della memoria virtuale. Oppure un terminale può essere utilizzato da un utente normale o da un amministratore di sistema, questi usi implicano diversi livelli di privilegio e forse diverse priorità nel sistema operativo;

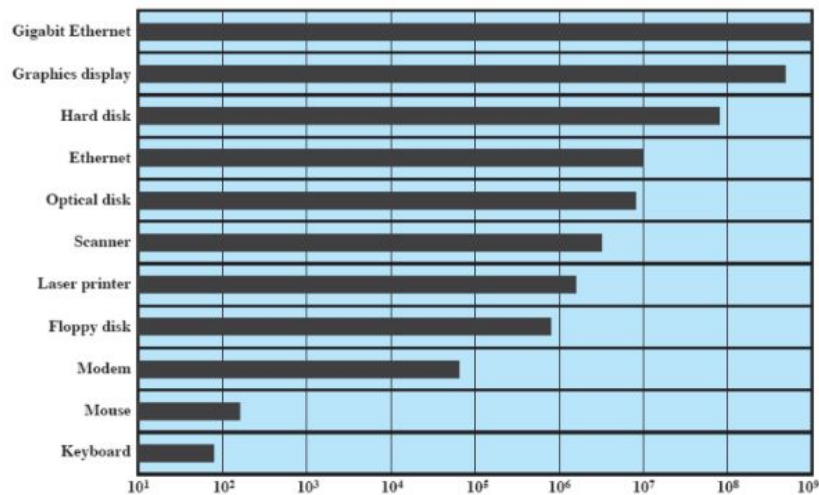


Figura 6.1: Data rate in bit per second

- le *difficoltà di controllo*, come in una stampante che richiede un'interfaccia di controllo relativamente semplice al contrario di un disco che è molto più complesso. L'effetto di queste differenze sul sistema operativo è filtrato in una certa misura dalla complessità del modulo I/O che controlla il dispositivo;
- l'*unità di trasferimento dati* che trasferisce i dati dal dispositivo che li genera alla memoria o viceversa, come flusso di byte o in blocchi più grandi di dimensione fissa;
- la *rappresentazione dei dati* in quanto i dati sono rappresentati secondo codifiche diverse in dispositivi diversi;
- le *condizioni di errore* perché in base alla natura degli errori varia di molto dal tipo di dispositivo. Ad esempio varia nel modo in cui gli errori vengono notificati, sulle loro conseguenze (fatali/ignorabili) o su come possono essere gestiti.

6.2 Organizzazione della funzione di I/O

Come abbiamo visto nella Sezione 1.3, le tecniche per gestire l'I/O sono: utilizzando la CPU, con o senza interruzioni, o utilizzando la memoria con il DMA. Nell'ultimo caso, il processore delega le operazioni di I/O al modulo DMA, che trasferisce i dati direttamente da o verso la memoria principale. Quando l'operazione è completata, il modulo DMA genera un interrupt per il processore.

Evoluzione della funzione di I/O

Nel corso degli anni la funzione di I/O ha subito notevoli migliorie che possono essere riassunte nei seguenti punti:

1. Il processore controlla direttamente un dispositivo periferico;
2. Viene aggiunto un modulo (o controllore) di I/O, direttamente sul dispositivo e il processore utilizza I/O programmato senza interruzioni;
3. Vengono aggiunti gli interrupt in modo da migliorare l'efficienza del processore che non deve aspettare il completamento dell'operazione di I/O;
4. Al modulo I/O viene dato il controllo diretto della memoria tramite DMA. Ora può spostare un blocco di dati da o verso la memoria senza coinvolgere il processore, tranne all'inizio e alla fine del trasferimento;
5. Il modulo I/O è stato migliorato per diventare un processore separato (*I/O channel*), con un set di istruzioni specializzate su misura per l'I/O. La CPU dirige il processore di I/O per eseguire

un programma di I/O nella memoria principale. Il processore di I/O recupera ed esegue queste istruzioni senza l'intervento del processore. Ciò consente al processore di specificare una sequenza di attività I/O e di essere interrotto solo quando l'intera sequenza è stata completata;

6. Il modulo I/O ha una propria memoria locale ed è, di fatto, un computer a sé stante (*I/O processor*). Con questa architettura è possibile controllare un ampio insieme di dispositivi I/O, con un coinvolgimento minimo del processore. Un uso comune di tale architettura è stato quello di controllare le comunicazioni con terminali interattivi.

6.3 Problemi nel progetto del sistema operativo

Obiettivi di progettazione

Due obiettivi sono fondamentali nella progettazione della struttura di I/O: efficienza e generalità. L'*efficienza* è importante perché la maggior parte dei dispositivi I/O sono estremamente lenti rispetto alla memoria principale e al processore. Un modo per affrontare questo problema è la multiprogrammazione che, come abbiamo visto, consente ad alcuni processi di attendere operazioni di I/O mentre un altro processo è in esecuzione, tuttavia spesso accade che l'I/O non tenga il passo con le attività del processore. E' necessario quindi cercare soluzioni software dedicate, a livello di sistema operativo, in particolare per l'I/O del disco.

Con la *generalità* si vogliono gestire tutti i dispositivi in modo uniforme, per semplicità e per evitare errori. Ciò vale sia per il modo in cui i processi visualizzano i dispositivi I/O sia per il modo in cui il sistema operativo gestisce i dispositivi e le operazioni I/O. Ciò si può fare nascondendo la maggior parte dei dettagli dell'I/O del dispositivo nelle routine di livello inferiore in modo che i processi utente e i livelli superiori del sistema operativo vedano i dispositivi in termini di funzioni generali (lettura, scrittura, apertura, chiusura, blocco e sblocco).

Struttura logica della funzione I/O

Con la progettazione gerarchica, le funzioni del sistema operativo vengono separate in base alla loro complessità, alla loro scala temporale e al loro livello di astrazione. Seguendo questo approccio si arriva ad un'organizzazione del sistema operativo in una serie di livelli, dove ogni livello, si basa su quello inferiore per eseguire funzioni più primitive e fornisce servizi al livello superiore. La modifica dell'implementazione di un livello non dovrebbe avere effetti sugli altri livelli e, per l'I/O, ci sono 3 macro-tipi maggiormente usati:

- Nel *Dispositivo Locale*, il Logical I/O: tratta il dispositivo come una risorsa logica per semplici operazioni (open, close, read, ...), il Device I/O: trasforma richieste logiche in sequenze di comandi di I/O e lo Scheduling and Control: esegue e controlla le sequenze di comandi (vedi Figura 6.2);
- Nel *Dispositivo di Comunicazione* (scheda Ethernet, WiFi, ...), al posto del Logical I/O, c'è una Architettura di Comunicazione, nella quale il dispositivo viene visto come una risorsa logica. A sua volta, questa consiste in un certo numero di livelli come vedremo nelle architetture TCP/IP (vedi Figura 6.2);
- Nei *File System* (SSD, CD, DVD, floppy disk, USB key ...), vengono aggiunti un Directory Management, che permette di definire tutte le operazioni utente che hanno a che fare con i file (crearli, cancellarli, spostarli, ...), il File System, che rappresenta la vera struttura logica delle operazioni (apri, chiudi, leggi, scrivi, ...) e l'Organizzazione fisica, che si occupa dell'allocazione/de-allocazione di spazio su disco per i file (vedi Figura 6.2).

6.4 Buffering dell'I/O

Se un processo emette un comando I/O, viene sospeso in attesa del risultato e quindi viene scambiato prima dell'inizio dell'operazione, il processo viene bloccato in attesa dell'evento I/O e l'operazione

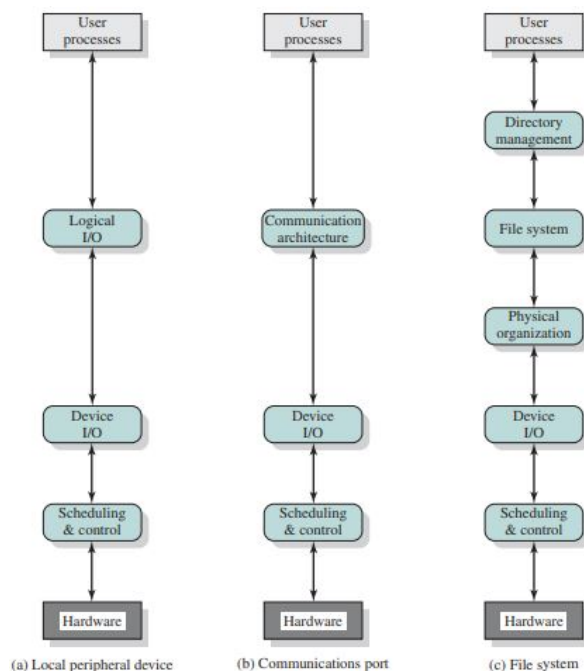


Figura 6.2: Livelli nei dispositivi locali, dispositivi di comunicazione e File System

di I/O viene bloccata in attesa affinché il processo venga scambiato. Per evitare questo deadlock¹, a volte è conveniente eseguire i trasferimenti di input prima che vengano effettuate le richieste ed eseguire i trasferimenti di output qualche tempo dopo l'esecuzione della richiesta. Questa tecnica è nota come *buffering*.

Prima di discutere i vari approcci al buffering è importante fare una distinzione tra due tipi di dispositivi di I/O: orientati ai blocchi e orientati al flusso. Un dispositivo orientato ai blocchi memorizza le informazioni in blocchi che solitamente hanno dimensioni fisse e i trasferimenti vengono effettuati un blocco alla volta. Un dispositivo orientato al flusso trasferisce i dati in entrata e in uscita come un flusso di byte.

Buffer singolo

Il tipo più semplice di supporto che il sistema operativo può fornire è il *buffering singolo* (vedi Figura 6.3). Quando un processo utente emette una richiesta I/O, il sistema operativo assegna all'operazione un buffer nella parte di sistema della memoria principale.

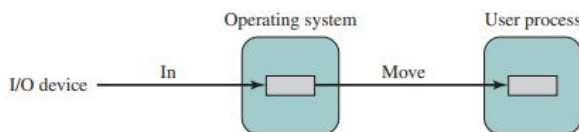


Figura 6.3: Buffer singolo

- Per i dispositivi orientati ai blocchi, i trasferimenti di input vengono effettuati nel buffer di sistema e, una volta completato il trasferimento, il processo sposta il blocco nello spazio utente e richiede immediatamente un altro blocco. Questo si chiama lettura anticipata (o input anticipato) e viene fatta perché, nella maggior parte dei casi, i dati vengono acceduti in sequenza.

Considerazioni simili si applicano all'output orientato ai blocchi. Quando i dati vengono trasmessi a un dispositivo, vengono prima copiati dallo spazio utente nel buffer di sistema, dal quale verranno infine scritti.

¹Indica una situazione in cui due o più processi o azioni si bloccano a vicenda, aspettando che uno esegua una certa azione che serve all'altro e viceversa.

- Per l'I/O orientato al flusso, lo schema di buffering singolo può essere utilizzato in modo lineare o byte. Il funzionamento a linea è appropriato per i terminali dove l'input e l'output dell'utente sono una riga alla volta, con un ritorno a capo che segnala la fine di una riga. In questo sistema il processo utente viene sospeso durante l'input, in attesa dell'arrivo dell'intera linea e vengono bufferizzate linee intere di input o di output. Il funzionamento byte alla volta viene utilizzato quando ogni pressione di un tasto è significativa, e l'interazione tra il sistema operativo e il processo utente segue il modello produttore/consumatore.

Buffer doppio

È possibile ottenere un miglioramento rispetto al buffering singolo assegnando due buffer di sistema all'operazione (vedi Figura 6.4). Un processo ora trasferisce i dati a (o da) un buffer, mentre il sistema operativo svuota (o riempie) l'altro.

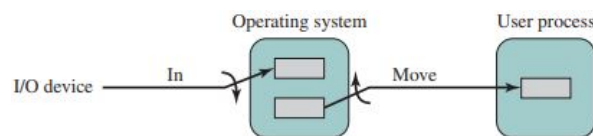


Figura 6.4: Buffer doppio

Buffer circolare

Se le prestazioni di un particolare processo sono al centro della nostra preoccupazione, allora vorremmo che le operazioni di I/O fossero in grado di tenere il passo con il processo e il doppio buffering potrebbe essere inadeguato. Quando vengono utilizzati più di due buffer, l'insieme dei buffer viene esso stesso definito *buffer circolare* (vedi Figura 6.5), dove ogni singolo buffer rappresenta un'unità nel buffer circolare.

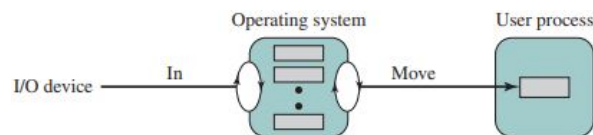


Figura 6.5: Buffer circolare

L'utilità del buffering

Il buffering smussa i picchi di richieste di I/O, tuttavia, nessuna quantità di buffering consentirà a un dispositivo I/O di tenere il passo con un processo quando la domanda media è maggiore di quella che il dispositivo può soddisfare. Tuttavia, in un ambiente di multiprogrammazione, quando sono presenti una varietà di attività di I/O e di processi da servire, il buffering è uno strumento che può aumentare l'efficienza del sistema operativo e le prestazioni dei singoli processi.

6.5 Scheduling del disco

Un disco HDD (Hard Drive Disk) è composto da tracce (corone circolari) e da settori (partizioni di una traccia), vedi Figura 6.6. Per leggere o scrivere su disco bisogna selezionare una traccia tramite una testina e, sulla traccia scelta, individuare il giusto settore. Per selezionare una traccia bisogna spostare una testina, se il disco ha testine mobili oppure selezionarla, se il disco ha testine fisse, per selezionare un settore, invece, bisogna aspettare che il disco ruoti.

Prestazioni del disco

Un diagramma temporale generale del trasferimento I/O del disco è mostrato nella Figura 6.7. In un primo momento il sistema operativo deve attendere che il disco sia libero (Wait for device) e

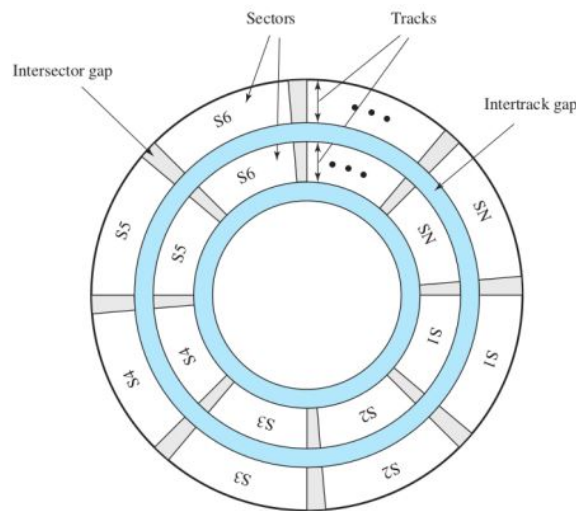


Figura 6.6: Disco

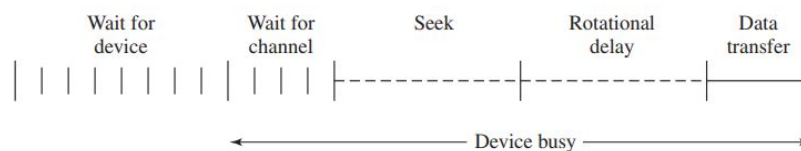


Figura 6.7: Diagramma temporale su disco

selezionare un suo canale (*wait for channel*). Una volta fatto ciò, quando l'unità disco è in funzione, il disco ruota a velocità costante e la testina deve essere posizionata sulla traccia desiderata e all'inizio del settore desiderato su quella traccia. Il tempo necessario per posizionare la testa sulla traccia è noto come tempo di ricerca (*Seek*) e una volta selezionata la traccia, il controller del disco attende che il settore appropriato ruoti per allinearsi con la testina. Il tempo impiegato dall'inizio del settore per raggiungere la testa è noto come ritardo rotazionale (*Rotational delay*). La somma del tempo di ricerca, se presente, e del ritardo di rotazione è uguale al tempo di accesso, ovvero il tempo necessario per mettersi in posizione per leggere o scrivere. Una volta che la testina è in posizione, l'operazione di lettura o scrittura viene poi eseguita man mano che il settore si sposta sotto la testina, questa è la parte dell'operazione relativa al trasferimento dei dati.

Politiche di scheduling per il disco

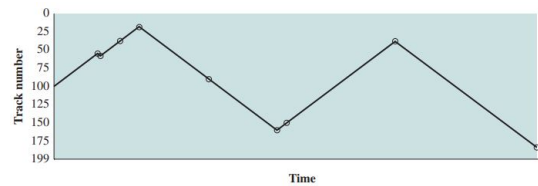
Anche per il disco sono stati pensati diversi algoritmi per rendere più efficiente l'operazione di lettura o scrittura.

Esempio 6.5.1 – Algoritmi di scheduling per il disco

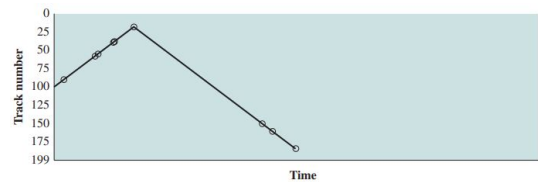
Supponiamo di avere un disco con 200 tracce con la testina del disco inizialmente posizionata sulla traccia 100. Le tracce richieste, nell'ordine ricevuto dallo scheduler del disco, sono:

55, 58, 39, 18, 90, 160, 150, 38, 184

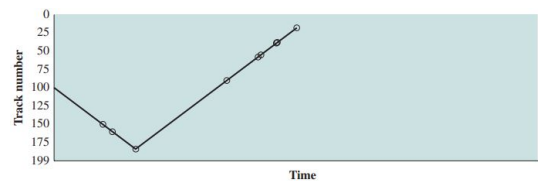
- Con il *FIFO* (First In First Out) le richieste vengono servite in modo sequenziale e la testina si muove continuamente avanti e dietro, se ci sono molti processi in esecuzione, le prestazioni sono simili ad uno scheduling random;



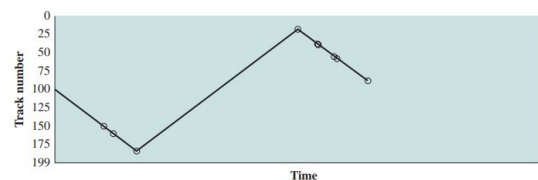
- Con il SSTF (Shortest Service Time First) si sceglie la richiesta che minimizza il movimento della testina dalla posizione attuale. Naturalmente, scegliere sempre il tempo di ricerca minimo non garantisce che il tempo di ricerca medio su un certo numero di movimenti del braccio sia minimo. Inoltre c'è la possibilità di ricorrere nella starvation delle richieste perché potrebbero arrivare continuamente richieste più vicine;



- Con lo SCAN si scelgono le richieste in modo tale che il braccio si muova sempre in un verso, e poi torni indietro. Questo sistema elimina la starvation delle richieste, in quanto possono accumularsi solo in un verso e successivamente vengono terminate, ma è poco equo perché favorisce le richieste vicino ai bordi o quelle appena arrivate. Questi problemi possono essere risolti con il C-SCAN e F-SCAN.



- Con il C-SCAN si servono le richieste solo in una direzione, in questo modo i bordi non sono più visitati 2 volte in poco tempo;



- Con l'F-SCAN si utilizzano due code Front (F) e Rear (R) e, mentre SCAN serve la coda F, ogni nuova richiesta è aggiunta alla coda R, quando SCAN finisce di servire F, si scambiano F ed R. Ogni nuova richiesta deve aspettare che tutte le precedenti vengano servite e quindi le richieste vecchie non perdono di priorità;
- Si può generalizzare l'F-SCAN con l'N-step-SCAN dove si accodano le richieste nella coda i-esima finché non si riempie, poi si passa alla $(i + 1) \bmod N$.

6.6 Cache del disco

La cache del disco è un buffer in memoria principale che contiene una copia di alcuni settori del disco. Quando si fa una richiesta di I/O per dati che si trovano su un certo settore, si vede prima se tale settore è nella cache, se non è presente viene anche copiato nella cache.

Ci sono svariati metodi per gestire il rimpiazzo di un settore nella cache piena: un algoritmo comunemente utilizzato è LRU, dove si sostituisce il blocco che è rimasto nella cache da più tempo senza referenze. La cache viene puntata da uno "stack" di puntatori e il settore riferito più di frequente è alla cima dello stack. Quindi, ogni volta che un settore viene referenziato o copiato nella cache, il suo puntatore viene portato in cima allo stack. Un'altra possibilità è LFU che sostituisce il

settore con il minor numero di riferimenti. LFU potrebbe essere implementato associando un contatore a ciascun blocco e quando viene introdotto un blocco, gli viene assegnato un conteggio pari a 1. Ad ogni riferimento il suo conteggio viene incrementato di 1 e, quando è richiesta la sostituzione, viene selezionato il blocco con il conteggio più piccolo. Un semplice algoritmo LFU presenta il seguente problema: è possibile che, a causa della località, un settore viene acceduto svariate volte di fila, al termine di tale intervallo, il conteggio dei riferimenti potrebbe avere un valore abbastanza alto quando in realtà il settore andrebbe sostituito. L'idea è quella di combinare LRU e LFU con uno "stack" di puntatori diviso in due parti una nuova e una vecchia ed operare come segue (vedi Figura 6.8):

1. Quando un blocco viene referenziato per la prima volta (cache miss), lo si sposta all'inizio della nuova sezione con un conteggio pari a 1;
2. Se il blocco viene referenziato quando è nella nuova sezione il suo conteggio dei riferimenti non viene incrementato; ciò fa sì che i conteggi per i blocchi a cui viene fatto ripetutamente riferimento rimangano invariati;
3. Quando il blocco passa alla vecchia sezione per scorrimento (quando un blocco vecchio viene riferito e diventa nuovo, spinge l'ultimo dei nuovi a diventare primo dei vecchi) e viene referenziato, ritorna alla cima dello stack con il conteggio incrementato di 1.

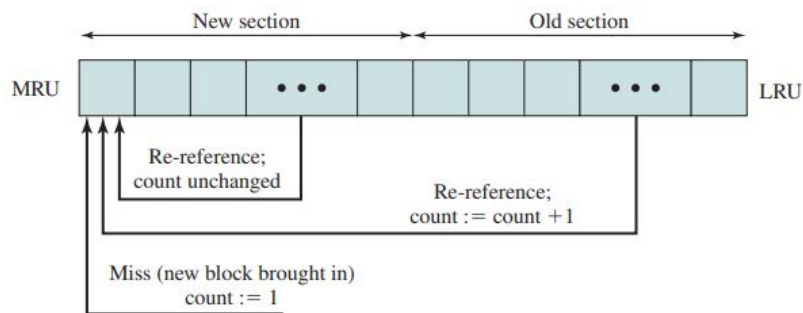


Figura 6.8: Sostituzione di blocchi basata sulla frequenza con due sezioni

C'è ancora un problema in questo sistema: se un nuovo blocco, che è passato alla vecchia sezione, non viene ri-referenziato abbastanza rapidamente, è molto probabile che venga sostituito perché ha necessariamente il conteggio di riferimenti più piccolo tra quelli che sono nella vecchia sezione. Per risolvere questo problema si può dividere lo stack in tre sezioni: nuova, media e vecchia (vedi Figura 6.9). Sia nella parte nuova che in quella media non possono essere effettuati i rimpiazzamenti, con la differenza che nella media e nella vecchia sezione i contatori vengono incrementati.

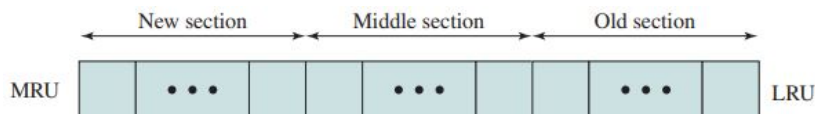


Figura 6.9: Sostituzione di blocchi basata sulla frequenza con tre sezioni

6.7 RAID

Con i dischi *RAID* (Redundant Array of Independent Disks) si hanno a disposizione più dischi fisici che si possono trattare separatamente. Su Windows, per esempio, si mostrano esplicitamente dischi diversi, con etichette diverse (C, D...), in Linux, alcuni files/directory sono memorizzati su un disco, altri su un altro e, tramite LVM (Logical Volume Manager), l'utente non deve occuparsi di decidere dove salvare i file. Un'altra possibilità è considerare diversi dischi fisici come un unico disco.

L'LVM va bene per pochi dischi, ed in generale non si è interessati alla ridondanza, ovvero non si occupa di memorizzare dati uguali su diversi dispositivi come backup. Con i dischi RAID, invece, non solo si risolve questo problema ma si riesce anche a velocizzare alcune operazioni, grazie ad una gerarchia a più livelli.

RAID livello 0

Il livello più basso della gerarchia di RAID è quello 0 (vedi Figura 6.10), dove i dischi vengono divisi in *strip*, a loro volta sono divisi in settori, un insieme di strips sui vari dischi (una riga) si chiama *stripe*. In questo livello non c'è ridondanza dei dati ma si utilizza la parallelizzazione su più strips per maggiore efficienza nell'accesso.

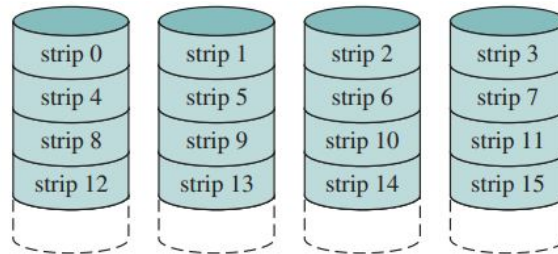


Figura 6.10: Raid livello 0

RAID livello 1

Nel RAID livello 1, metà della memoria di archiviazione viene utilizzata per duplicare ogni disco (vedi Figura 6.11). In questo modo se si rompe un disco si ha un recupero sicuro dei dati.

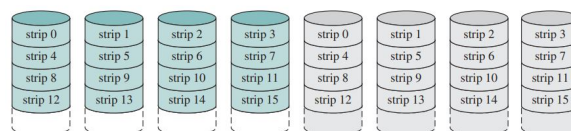


Figura 6.11: Raid livello 1

RAID livello 2

Nella maggior parte dei casi, un disco non si rompe per intero, ma al più una sua parte o qualche suo bit. Per tanto è inutile replicare l'intera informazione, è più efficiente utilizzare dei codici particolari, per esempio, il codice di Hamming che può correggere errori su singoli bit. In questo modo non saranno utilizzati N dischi di overhead, ma ne serviranno il giusto numero per memorizzare il codice scelto (vedi Figura 6.12).

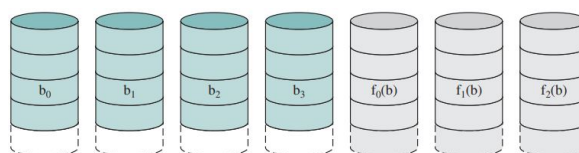


Figura 6.12: Raid livello 2

RAID livello 3

Nel RAID livello 3, viene utilizzato un solo disco di overhead (vedi Figura 6.13). In questo livello ogni strip è considerato come un byte e, sul disco di overhead, viene memorizzato, per ogni bit, la parità dei bit che hanno la stessa posizione, in questo modo resta possibile recuperare i dati quando fallisce un unico disco.

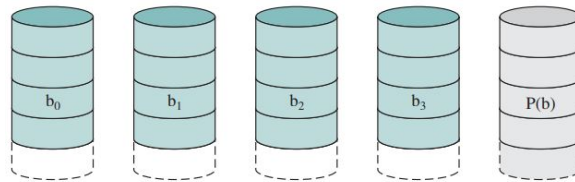


Figura 6.13: Raid livello 3

RAID livello 4

In questo livello si hanno le stesse caratteristiche del RAID livello 3, ma con la differenza che ogni strip è un "blocco", potenzialmente grande, recuperabile in caso di fallimento di un unico disco (vedi Figura 6.14).

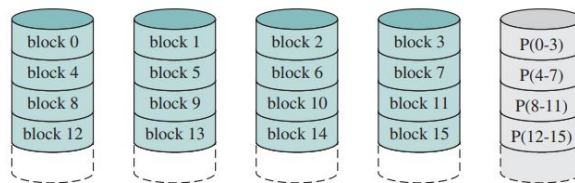


Figura 6.14: Raid livello 4

RAID livello 5

Nel livello 5, le informazioni di parità non sono tutte su un unico disco (vedi Figura 6.15). In questo modo si evita il collo di bottiglia del disco di parità e non c'è un disco "privilegiato" che salva le informazioni.

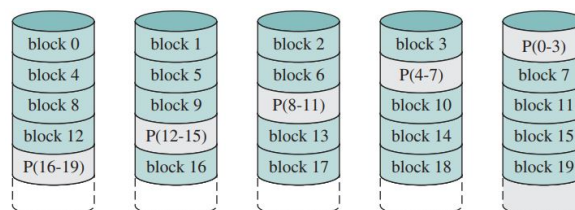


Figura 6.15: Raid livello 5

RAID livello 6

Il RAID livello 6 è come il livello 5, ma con 2 dischi di parità indipendenti in modo da poter recuperare anche 2 fallimenti di disco (vedi Figura 6.16).

Da notare che se viene effettuata un'operazione sul RAID, tutti i dischi effettuano in sincrono quell'operazione, e un'operazione sul RAID è un'operazione su un sottoinsieme dei suoi dischi, questo permette il completamento in parallelo di richieste I/O distinte.

6.8 I/O in Linux

In Linux si ha un'unica page cache per tutti i trasferimenti tra disco e memoria, compresi quelli dovuti alla gestione della memoria virtuale, questo permette di condensare le scritture e di sfruttare la località dei riferimenti per risparmiare accessi su disco. Viene effettuata una scrittura su disco solo quando è rimasta poca memoria o quando l'età delle pagine "sporche" va sopra una certa soglia. Inoltre non c'è una politica di replacement separata, ma è la stessa usata per il rimpiazzamento delle pagine.

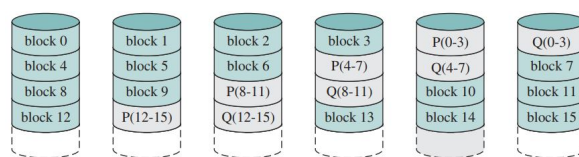


Figura 6.16: Raid livello 6

Capitolo 7

File System

Il file system è una delle parti del sistema operativo più importanti per l'utente perché si occupa della gestione dei file. I file, a loro volta, sono l'elemento principale della maggior parte delle applicazioni perché hanno un'esistenza a lungo termine (in genere "sopravvivono" ai processi), possono essere condivisi con altri processi e sono strutturati in directory gerarchiche. Le operazioni tipiche sui file sono: creazione, cancellazione, apertura, scrittura e chiusura. I file sono gestiti da un insieme di programmi e librerie di utilità che costituiscono il *File Management System*, e vengono eseguiti come processi privilegiati. I File Management Systems forniscono servizi agli utenti e alle applicazioni per l'uso di file e definiscono anche il modo in cui i file sono usati, in questo sollevano i programmatori dal dover scrivere codice per gestire i file.

L'organizzazione tipica richiede un Directory Management che si occupa di tutte le operazioni utente che hanno a che fare con i file; un File System che gestisce la struttura logica e le operazioni (apri, chiudi, leggi, scrivi...); l'organizzazione fisica che identifica i file con indirizzi fisici su disco.

7.1 Directory

Una directory è un file "speciale" che permette di mappare i nomi dei file alle loro informazioni. Le operazioni consentite su una directory sono: ricerca, creazione file, cancellazione file, lista del contenuto, modifica. A parte le informazioni dei file (nome, tipo e organizzazione del file), una directory deve contenere: il volume, indica il dispositivo su cui il file è memorizzato; l'indirizzo di partenza, ad es.: da quale settore o traccia di disco; la dimensione attuale, in byte, word o blocchi; la dimensione allocata, dimensione massima del file; il proprietario, può concedere/negare i permessi ad altri utenti, e può anche cambiare tali impostazioni; le informazioni sull'accesso, potrebbe contenere username e password per ogni utente autorizzato; le azioni permesse, per controllare lettura, scrittura, esecuzione, spedizione tramite rete; i time stamp di creazioni, accessi, identità e backup.

Il metodo usato per memorizzare queste informazioni è con uno schema gerarchico ad albero, dove c'è una *directory master* che contiene le *directory utente* e, ogni directory utente può contenere altre directory oppure altri file (vedi Figura 7.1). La struttura ad albero permette agli utenti di trovare un file seguendo un percorso nell'albero (*directory path*) e file con nomi duplicati sono possibili purché con path diversi.

Dover dare ogni volta il path completo prima del nome del file può essere lungo e noioso, quindi solitamente, gli utenti o i processi interattivi hanno associata una *directory di lavoro* o corrente dove tutti i nomi di file sono dati relativamente a questa directory.

7.2 Gestione della memoria secondaria

Il sistema operativo è responsabile dell'assegnamento di blocchi a file e per tanto occorre allocare e mantenere traccia dello spazio per i file e occorre tener traccia dello spazio allocabile. I file vengono allocati in porzioni o blocchi, ovvero in sequenze contigue di settori di disco.

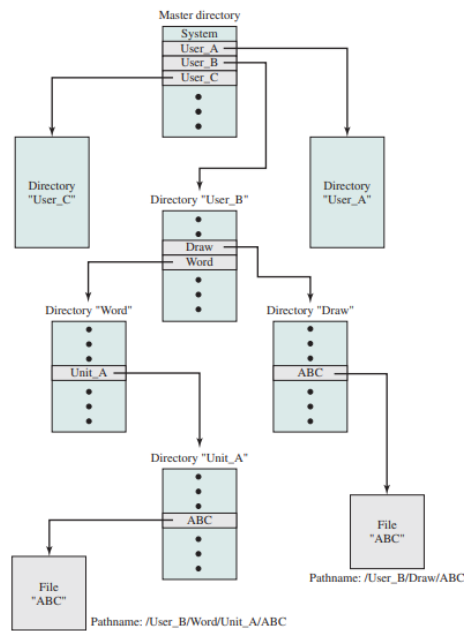


Figura 7.1: Esempio di struttura ad albero

Allocazione dei file

L'allocazione dinamica è quasi sempre preferita, perché altrimenti con la preallocazione viene riservato lo spazio massimo stimato durante la sua creazione del file con risultante spreco di spazio su disco.

Per l'allocazione delle porzioni ci sono due possibilità: o si alloca una porzione larga a sufficienza per l'intero file, è un approccio efficiente per il processo che vuole creare il file, oppure si alloca un blocco alla volta in una sequenza di settori contigui, è efficiente per il sistema operativo che deve gestire tanti file. Si cerca un punto d'incontro tra efficienza del singolo file ed efficienza del sistema con porzioni grandi di dimensione variabile oppure con porzioni fisse piccole.

Per allocare spazio per i file si usano principalmente tre metodi: contiguo, concatenato, indicizzato. Vediamoli nel dettaglio:

- Con l'*allocazione contigua* viene allocato un insieme di blocchi per il file quando quest'ultimo viene creato, quindi è necessaria la preallocazione. Nella tabella di allocazione c'è una sola entry contenente il blocco di partenza e la lunghezza del file e c'è bisogno di compattare i settori quando i file vengono eliminati (frammentazione esterna).
- Con l'*allocazione concatenata* viene allocato un blocco alla volta e ogni blocco contiene una parte relativa ai dati del file e una per il puntatore al blocco successivo. E' necessaria una sola entry nella tabella di allocazione dei file che contiene il blocco di partenza e la lunghezza del file. Questo metodo risulta utile quando bisogna accedere sequenzialmente ai file e utilizzando il consolidamento si inseriscono i file contigui in modo da migliorare anche l'accesso non sequenziale.
- L'*allocazione indicizzata* è una via di mezzo tra i due approcci precedenti dove i blocchi possono contenere dati oppure indici. Infatti, nella tabella di allocazione dei file, si ha una sola entry con l'indirizzo di un blocco, questo blocco contiene degli indici per ogni porzione allocata al file. I blocchi possono avere lunghezza fissa o variabile e nel secondo caso oltre agli indici deve essere specificata anche la lunghezza di ogni blocco.

Esempio 7.2.1 – Allocazione

Supponiamo di avere 35 settori e analizziamo gli approcci di allocazione di spazio per i file. Con l'allocazione contigua per ogni file viene deciso il blocco di partenza e la sua lunghezza tenendo conto della dimensione massima stimata:

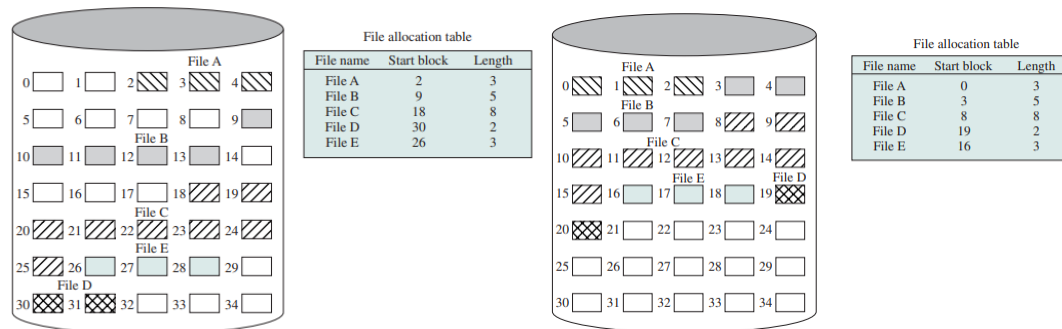


Figura 7.2: Allocazione contigua e compattazione

Con l'allocazione concatenata nella tabella di allocazione dei file troviamo solo il file iniziale con la sua lunghezza:

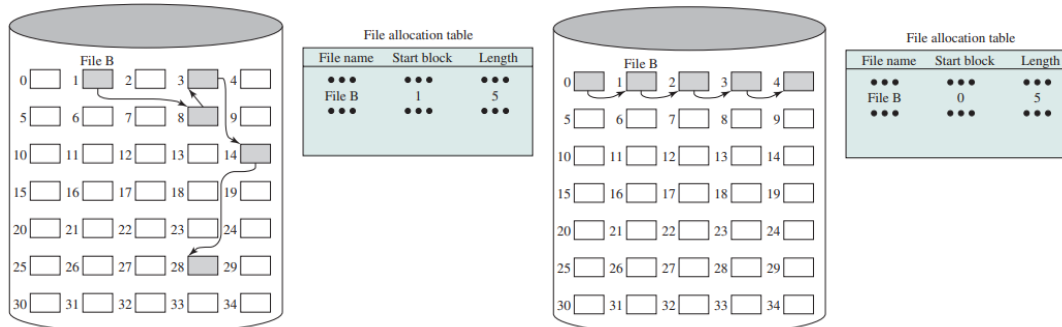


Figura 7.3: Allocazione concatenata e consolidamento

Con l'allocazione indicizzata l'indirizzo dell'unica entry della tabella contiene gli indici di ogni porzione allocata al file:

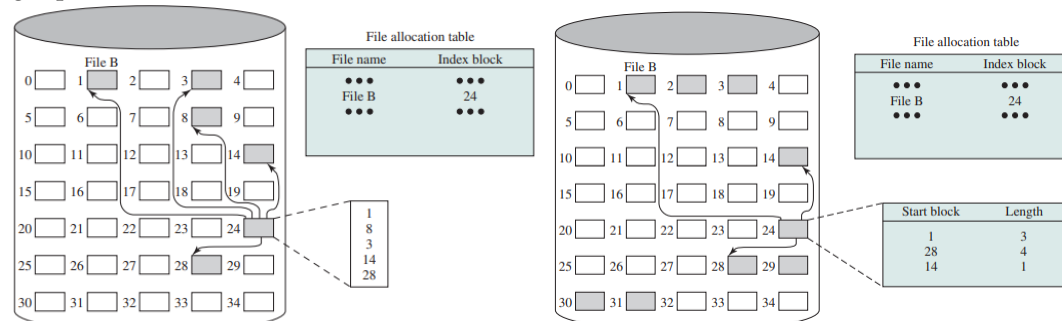


Figura 7.4: Allocazione indicizzata con porzioni fisse e variabili

Gestione dello spazio libero

La gestione dello spazio libero è altrettanto importante di quello occupato. Per gestire lo spazio libero serve una tabella di allocazione di disco, oltre che a quella di allocazione per i file e ogni volta che si alloca o si cancella un file, lo spazio libero va aggiornato. Per fare ciò ci sono vari approcci:

- La *tabella di bit* è un vettore con un bit per ogni blocco su disco (0 libero, 1 occupato). In questo modo si minimizza lo spazio richiesto alla tabella di allocazione del disco ma se il disco è quasi pieno, la ricerca di uno spazio libero può richiedere molto tempo.
- Le porzioni libere possono essere *concatenate* tra loro usando, per ogni blocco libero, un puntatore ed un intero per la dimensione, il tutto senza overhead di spazio. Però, se c'è molta frammentazione, la lista diventa inevitabilmente lunga e se bisogna allocare molti file si devono leggere altrettanti blocchi per sapere qual'è il prossimo libero.

- Si può trattare lo spazio libero come un unico file, ovvero con una entry e con un *indice* che gestisce le porzioni come se fossero di lunghezza variabile.
- Con la *lista dei blocchi liberi* ad ogni blocco libero viene assegnato un numero sequenziale e la lista di questi numeri viene memorizzata in una parte dedicata del disco.

Un *volume* è un insieme di settori in memoria secondaria, che possono essere usati dal sistema operativo o dalle applicazioni.

I *dati* rappresentano il contenuto dei file, mentre i *metadati* sono tutte le altre informazioni (lista di blocchi liberi, data dell'ultima modifica...). I metadati devono essere su disco, perché devono essere persistenti, ma mantenerli sempre costanti è inefficiente per questo, tramite il *journaling*, anziché scrivere le informazioni nelle opportune porzioni di disco, le si scrive in una zona di disco dedicata (*log*).

7.3 Gestione dei file in UNIX

In UNIX ci sono sei tipi di file: normale, directory, speciale (mappano su nomi di file i dispositivi di I/O), named pipe (per far comunicare processi tra loro), hard link (collegamenti, nome di file alternativo), link simbolico (il suo contenuto è il nome del file cui si riferisce).

Inode

Un *inode* (index node) è una struttura dati che contiene le informazioni essenziali per un dato file e ogni inode è identificato con un numero per garantire un accesso più semplificato. Dal sistema operativo viene mantenuta una tabella di tutti gli inode corrispondenti a file aperti in memoria principale, e tutti gli altri i-node sono in una zona di disco dedicata (i-list). Ogni inode contiene le seguenti informazioni: tipo e modo di accesso del file; identificatore dell'utente proprietario e del gruppo cui tale utente appartiene; tempo di creazione e di ultimo accesso (lettura o scrittura); flag utente e flag per il kernel; numero sequenziale di generazione del file; dimensione delle informazioni aggiuntive; altri attributi (controllo di accesso e altro); dimensione; numero di blocchi o numero di file (per le directory); dimensione dei blocchi; sequenze di puntatori a blocchi.

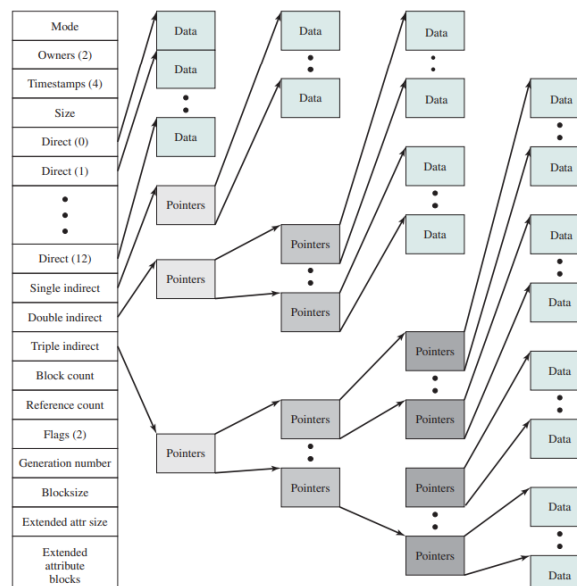


Figura 7.5: inode

Allocazione, directory e accesso ai file

L'*allocazione* in UNIX è dinamica a blocchi e l'indicizzazione tiene traccia dei blocchi dei file.

Le *directory* sono file che contengono una lista di coppie formata da un puntatore all'inode e il nome del file. Alcuni di questi file potrebbero essere a loro volta directory, quindi si forma una struttura gerarchica dove ogni directory può essere modificata solo dal sistema operativo, ma letta da ogni utente.

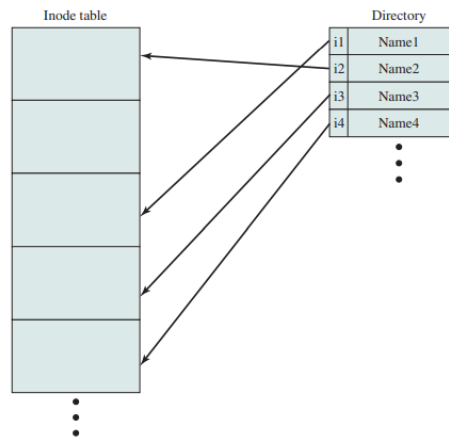


Figura 7.6: Directory e inode

Per l'*accesso ai file* ci sono tre terne di permessi: lettura, scrittura e esecuzione che a loro volta vengono gestiti nella terna di proprietario, il suo gruppo e tutti gli altri. Quindi, per ogni file, si gestiscono opportunamente i permessi di lettura, scrittura ed esecuzione per user, group e other.

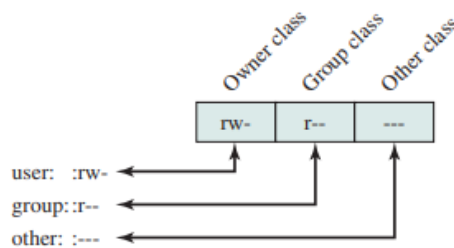


Figura 7.7: Accesso ai file

7.4 Gestione dei file su Windows

In Windows ci sono due file system principali: *FAT* (File Allocation Table - vecchio file system) con allocazione concatenata e blocchi di dimensione fissa; *NTFS* (New Technology File System - nuovo file system) con allocazione con bitmap e blocchi di dimensione fissa.

File Allocation Table

La *File Allocation Table*, è una tabella ordinata di puntatori, ancora in uso per le chiavette USB, e usa un puntatore (riga nella tabella) per ogni cluster del disco. Ogni cluster ha dimensione variabile tra 2 e 32 KB, ed è costituito da settori di disco contigui, di conseguenza, la tabella cresce con la grandezza della partizione. La parte della FAT relativa ai file aperti deve essere sempre mantenuta interamente in memoria principale per consentire di identificare i blocchi di un file e accedervi sequenzialmente seguendo i collegamenti nella FAT.

Ogni FAT contiene delle regioni di informazioni (vedi Figura 7.8): *Regione Boot Sector* contiene informazioni necessarie per l'accesso al volume; *Regione FAT* contiene due copie della file allocation table e mappa il contenuto della regione dati, indicando a quali directory/file i diversi cluster appartengono; *Regione Root Directory* è una directory table che contiene tutti i file entry per la directory root del sistema; *Regione Dati* è la regione del volume in cui sono effettivamente contenuti i dati dei file e directory.

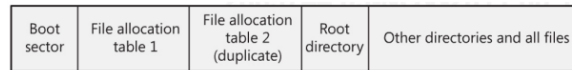


Figura 7.8: Regioni della FAT

Data la tabella FAT di puntatori, il puntatore i -esimo, se è 0, indica che l' i -esimo cluster del disco è libero; se è tutti 1 (valore speciale), indica che ci troviamo sull'ultimo blocco del file; se non è 0 e non è un valore speciale, indica il cluster dove trovare il prossimo pezzo del file, oltre che la prossima entry della FAT per questo file (vedi Figura 7.9).

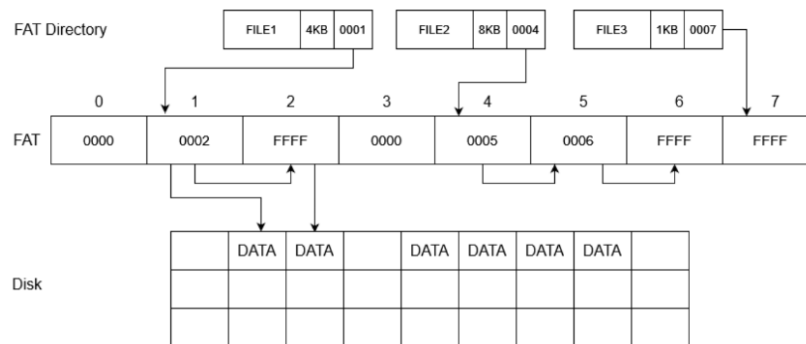


Figura 7.9: Funzionamento della FAT

New Technology File System

Nell'NTFS i file sono definiti da un insieme di attributi rappresentati come un byte stream. Anche in questo file system ci sono diverse regioni: *Regione Boot Sector* è basata sull'equivalente FAT; *Master File Table* è la principale struttura dati del file system implementata come un file con una sequenza lineare di record, dove ogni record descrive un file identificato da un puntatore.

NTFS cerca sempre di assegnare ad un file sequenze contigue di blocchi e per ogni file, esiste un record base nell'MFT. NTFS usa una tecnica simile agli i-node UNIX/Linux: il record base è un puntatore a n record secondari, eventuale spazio rimanente nel record base contiene le prime sequenze del file; se i record estesi non rientrano in MFT per mancanza di spazio, vengono quindi salvati in un file dedicato, con un apposito record nell'MFT.

Capitolo 8

Gestione della concorrenza

Per i sistemi operativi moderni è essenziale supportare più processi in esecuzione. In tal senso un grosso problema da affrontare è la *concorrenza*, ovvero la gestione del modo con cui questi processi interagiscono. In generale la concorrenza si manifesta quando ci sono delle applicazioni multiple che condividono il tempo di calcolo, quando ci sono delle applicazioni strutturate per essere parallele, oppure semplicemente quando il sistema operativo stesso ha più processi o thread in esecuzione.

Race condition

Una *sezione critica* è una parte di un processo in cui avviene un accesso ad una risorsa condivisa. Con la *mutua esclusione* si cerca di imporre che solo un processo possa accedere alla sezione critica e quindi si vuole evitare: *race condition* violazione della mutua esclusione; *deadlock* o stallo, quando due o più processi non possono procedere nell'esecuzione perché ciascuno attende l'altro; *livelock* o stallo attivo, quando due o più processi cambiano continuamente il proprio stato; *starvation* o morte per fame, quando un processo non viene mai scelto dallo scheduler.

La difficoltà principale con la concorrenza è che non si può fare nessuna assunzione o previsione sul comportamento dei processi e sullo scheduler.

Esempio 8.0.1 – Risorse condivise

Dato il seguente pseudocodice di una funzione che prende un valore in input e lo scrive in output:

```
void echo()
{
    chin = getchar();
    chout = chin;
    putchar(chout);
}
```

supponiamo che due processi P_1 e P_2 cercano di eseguire questo codice su un processore.

Process P_1	Process P_2
.	.
chin = getchar();	.
.	chin = getchar();
chout = chin;	.
.	chout = chin;
putchar(chout);	.
.	putchar(chout);
.	.

Notiamo come chin viene sovrascritto dal processo P_2 generando problemi di race condition su risorse condivise. Il problema si può risolvere facendo sì che la funzione echo possa essere eseguita solo da un processo alla volta.

Il sistema operativo si deve assicurare che processi ed output siano indipendenti dalla velocità di computazione (ovvero dallo scheduling), quindi quando un processo ha la necessità di accedere alle risorse deve fare una richiesta al sistema operativo (tramite `syscall`). Tuttavia, non è sempre possibile, ad esempio se occorre accedere ad una risorsa che potrebbe essere condivisa con altri processi, occorre fare una richiesta di bloccaggio della risorsa. Quindi gli stessi processi devono essere scritti pensando già alla cooperazione, usando opportune `syscall` (es. semafori) e scrivendo le funzioni di `entercritical` e `exitcritical`.

Requisiti per la mutua esclusione

Qualsiasi meccanismo si usi per offrire la mutua esclusione, deve soddisfare i seguenti requisiti: solo un processo alla volta può essere nella sezione critica per una risorsa; non si deve generare deadlock o starvation; un processo deve entrare subito nella sezione critica se nessun altro processo usa la risorsa; un processo che si trova nella sua sezione non-critica non deve subire interferenza da altri processi; un processo che si trova nella sezione critica ne deve prima o poi uscire.

8.1 Mutua esclusione: supporto hardware

Una prima idea per effettuare la mutua esclusione è disabilitare gli interrupt prima di entrare nella sezione critica per poi abilitarli una volta usciti. Disabilitare le interruzioni garantisce la mutua esclusione ovvero, se il processo può decidere di non essere interrotto, allora nessun altro lo può interromperlo mentre si trova nella sezione critica, ma non funziona su sistemi con più processori e se i processi ne abusano peggiorano le prestazioni del sistema operativo.

Un'altra opzione è utilizzare delle istruzioni macchina speciali:

```
int compare_and_swap(int word, int testval, int newval)
{
    int oldval;
    oldval = word;
    if (word == testval) word = newval;
    return oldval
}

void exchange(int register, int memory)
{
    int temp;
    temp = memory;
    memory = register;
    register = temp;
}
```

L'hardware garantisce che un solo processo per volta possa eseguire una chiamata a tali funzioni anche se ci sono più processori (vedi Figura 8.1).

In entrambi i casi l'idea è che il processo che trova `bolt` a 0 può eseguire la `critical section` e, per impedire ad altri di entrare, mette `bolt` a 1. Una volta terminato, rimette `bolt` a 0, così gli altri processi possono accedere alla sezione o anche lui stesso, se è solo oppure se lo scheduler lo favorisce.

Questo tipo di approccio è applicabile a qualsiasi numero di processi, sia su un sistema ad un solo processore che ad un sistema a più processori con memoria condivisa e possono essere usate per gestire sezioni critiche multiple. Ma è anche vero che si possono generare *busy-waiting* (spreco di tempo di computazione), starvation e deadlock.

Per superare tutti questi ostacoli possiamo utilizzare i semafori.

8.2 Semafori

Un *semaforo* è una particolare struttura dati sulla quale è possibile fare solo tre operazioni atomiche con il supporto di un valore intero utilizzato per scambiare segnali con i processi. Queste operazioni sono: `initialize`, `decrement` o `semWait` che può mettere il processo in `blocked`, `increment` o

<pre> /* program mutualexclusion */ const int n = /* number of processes */; int bolt; void P(int i) { while (true) { while (compare_and_swap(bolt, 0, 1) == 1) /* do nothing */; /* critical section */; bolt = 0; /* remainder */; } } void main() { bolt = 0; parbegin (P(1), P(2), ... ,P(n)); } </pre>	<pre> /* program mutualexclusion */ int const n = /* number of processes */; int bolt; void P(int i) { int keyi = 1; while (true) { do exchange (&keyi, &bolt) while (keyi != 0); /* critical section */; bolt = 0; /* remainder */; } } void main() { bolt = 0; parbegin (P(1), P(2), ..., P(n)); } </pre>
---	---

Figura 8.1: Mutua esclusione con compare_and_swap e exchange

semSignal che può mettere un processo blocked in ready. Si tratta di syscall che sono eseguite in kernel mode e possono agire direttamente sui processi.

```

struct semaphore {
    int count;
    queueType queue;
};
void semWait(semaphore s)
{
    s.count--;
    if (s.count < 0) {
        /* place this process in s.queue */;
        /* block this process */;
    }
}
void semSignal(semaphore s)
{
    s.count++;
    if (s.count <= 0) {
        /* remove a process P from s.queue */;
        /* place process P on ready list */;
    }
}

```

Listing 8.1: Definizione di un semaforo

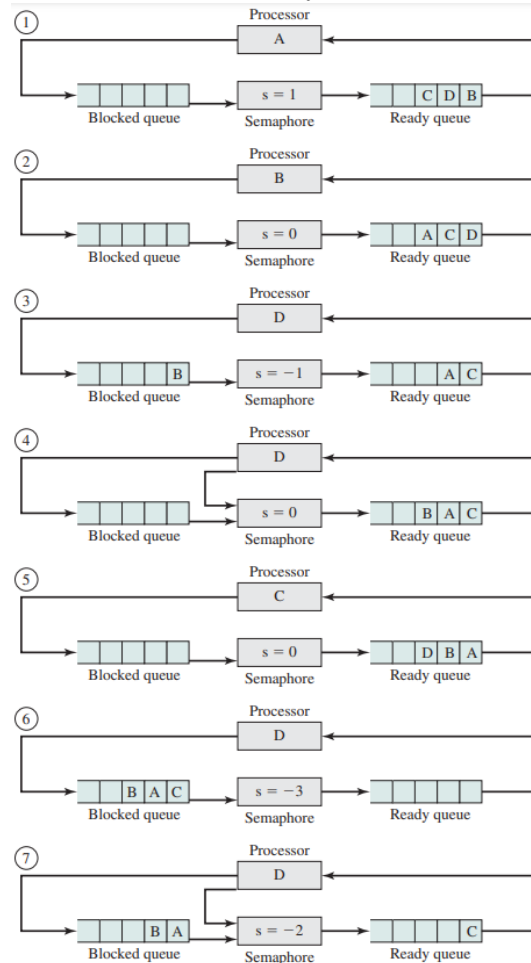
Il sistema operativo usa una coda per memorizzare i processi in attesa su un semaforo, ma quale politica viene usata per prendere il prossimo processo in attesa nella coda? Se si usa la FIFO, allora si parla di *strong semaphore* (semaforo forte); se la politica non viene specificata, allora si parla di *weak semaphore* (semaforo debole).

Esempio 8.2.1 – Funzionamento di un semaforo

Supponiamo di avere un semaforo s con il suo valore intero e la sua coda di processi.

① dal dispatcher viene scelto il processo A per andare in esecuzione e viene invocata la funzione `semWait` in modo da ② inserire A nella coda e mandare in esecuzione il processo B . Ora se viene invocata una `semWait` il valore del semaforo viene decrementato a -1 e ③ il processo B finisce nella coda dei blocked. ④ nel momento in cui viene eseguito D si effettua una `semSignal` e il processo B torna nella coda dei Ready. ⑤ ora è in esecuzione il processo C e

vengono eseguite tre `semWait`, ⑥ quindi il valore intero passa a -3 e nella coda dei blocked ci sono i processi B, A, C. ⑦ quando verrà eseguita una `semSignal` un semaforo forte sblocca il primo processo della coda e lo inserisce nei Ready.



```
/* program mutualexclusion */
const int n = /* number of processes */;
semaphore s = 1;
void P(int i)
{
    while (true) {
        semWait(s);
        /* critical section */;
        semSignal(s);
        /* remainder */;
    }
}
void main()
{
    parbegin (P(1), P(2), ..., P(n));
}
```

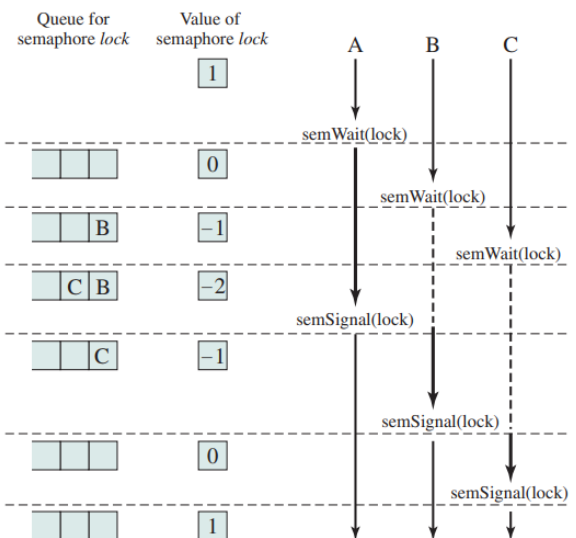


Figura 8.2: Mutua esclusione con i semafori

Problema dal produttore/consumatore

Ci sono uno o più processi produttori che creano dati e li mettono in un buffer, un consumatore prende dati dal buffer uno alla volta e al buffer può accedere un solo processo, sia esso produttore o consumatore. Il problema nasce quando si vuole assicurare che i produttori non inseriscano dati quando il buffer è pieno e che il consumatore non prenda dati quando il buffer è vuoto.

Ipotizzando di avere un buffer infinito possiamo scrivere il codice del produttore e consumatore come segue:

```
while(true) {
    /* produce item v */
    b[in] = v;
    in++;
}

while(true) {
    while(in <= out)
        /* do nothing */;
    w = b[out];
    out++;
    /* consume item w */
}
```

La corretta soluzione per risolvere i problemi visti è in Figura 8.3.



```
/* program producerconsumer */
int n;
binary_semaphore s = 1, delay = 0;
void producer()
{
    while (true) {
        produce();
        semWaitB(s);
        append();
        n++;
        if (n==1) semSignalB(delay);
        semSignalB(s);
    }
}
void consumer()
{
    int m; /* a local variable */
    semWaitB(delay);
    while (true) {
        semWaitB(s);
        take();
        n--;
        m = n;
        semSignalB(s);
        consume();
        if (m==0) semWaitB(delay);
    }
}
void main()
{
    n = 0;
    parbegin (producer, consumer);
}
```

```
/* program producerconsumer */
semaphore n = 0, s = 1;
void producer()
{
    while (true) {
        produce();
        semWait(s);
        append();
        semSignal(s);
        semSignal(n);
    }
}
void consumer()
{
    while (true) {
        semWait(n);
        semWait(s);
        take();
        semSignal(s);
        consume();
    }
}
void main()
{
    parbegin (producer, consumer);
}
```

Figura 8.3: Soluzione del produttore/consumatore con semafori binari e generali

Implementazione dei semafori

La vera implementazione dei semafori è con buffer finiti e circolari. Anche con questo tipo di semafori ci sono due puntatori *In* e *Out* grazie ai quali i produttori/consumatori svolgono le loro operazioni.

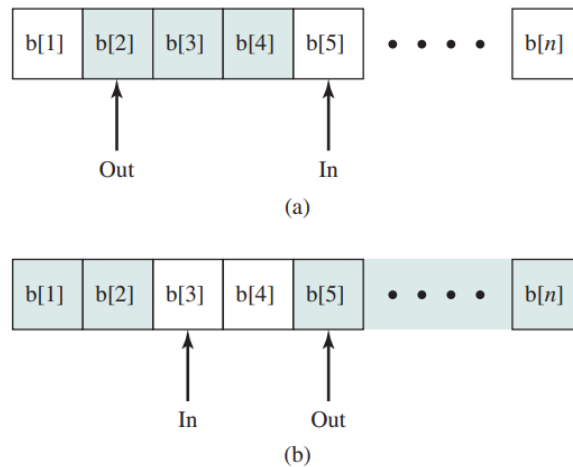


Figura 8.4: Buffer circolare finito

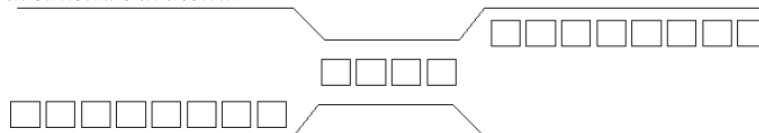
La dimensione effettiva del buffer è $n - 1$, altrimenti non si potrebbe capire se $in == out$ implichi buffer pieno o vuoto. I codici del produttore e consumatore diventano:

```
while(true) {
    /* produce item v */
    while((in + 1) % n == out)
        /* do nothing */;
    b[in] = v;
    in = (in + 1) % n;
}

while(true) {
    while(in == out)
        /* do nothing */;
    w = b[out];
    out = (out + 1) % n;
    /* consume item w */
}
```

Esempio 8.2.2 – Trastevere

Supponiamo di voler risolvere il seguente problema: sulla strada di Trastevere a doppio senso, per un breve tratto, la strada si restringe e diventa a senso unico alternato con al massimo quattro macchine di passaggio. Una volta impegnata da un lato, tutte le macchine dall'altro lato devono aspettare che non ci siano più macchine né sulla strettoia, né in coda. Scrivere una funzione che gestisca il comportamento delle macchine di sinistra e di destra.



La funzione richiesta è la seguente:

```
semaphore z = 1;
semaphore strettoia = 4;
semaphore sx = 1;
semaphore dx = 1;
```

```

/* program boundedbuffer */
const int sizeofbuffer = /* buffer size */;
semaphore s = 1, n = 0, e = sizeofbuffer;
void producer()
{
    while (true) {
        produce();
        semWait(e);
        semWait(s);
        append();
        semSignal(s);
        semSignal(n);
    }
}
void consumer()
{
    while (true) {
        semWait(n);
        semWait(s);
        take();
        semSignal(s);
        semSignal(e);
        consume();
    }
}
void main()
{
    parbegin (producer, consumer);
}

```

Figura 8.5: Produttori e consumatori con buffer circolari

```

int nsx = 0;
int ndx = 0;

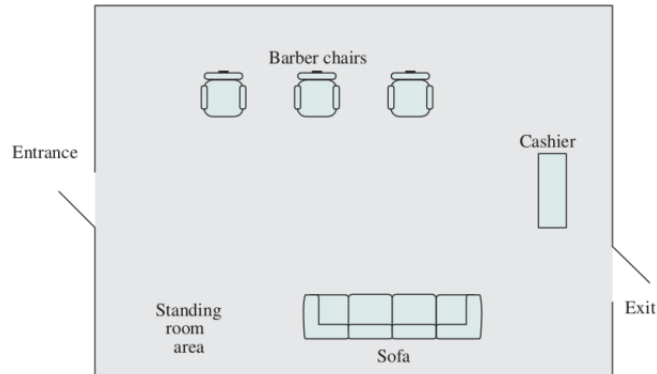
macchina_dal_lato_sinistro () {
    wait(z);
    wait(sx) ;
    ++nsx;
    if(nsx == 1)
        wait(dx);
    signal(sx);
    signal(z);
    wait(strettoia);
    passa_strettoia();
    signal(strettoia);
    wait(sx);
    --nsx;
    if(nsx == 0)
        signal(dx);
    signal(sx);
}

```

Esempio 8.2.3 – Il negozio del barbiere

Al negozio del barbiere può accedere un numero definito di persone (max_cust) e ogni cliente prima si siede sul divano, poi da lì accede ad una delle sedie. Si compete per entrambe le risorse: tra tutti quelli

nel negozio si compete per il divano, tra tutti quelli sul divano per le sedie. Il barbiere o dorme, o taglia capelli, o prende soldi dopo il taglio.



Una delle migliori soluzioni per questo problema è la seguente:

```

int next_barber ;

void Barber(i) {
    while(true) {
        wait(mutex);
        next_barber = i;
        signal(chair);
        wait(ready);
        signal(mutex);
        cut_hair();
        signal(finish[i]);
        wait(payment);
        accept_pay();
        signal(recpt);
    }
}

void Customer(){ int my_barber;
    wait(max_cust);
    enter_shop();
    wait(sofa);
    sit_on_sofa();
    wait(chair);
    get_up_from_sofa();
    signal(sofa);
    my_barber = next_barber;
    sit_in_chair();
    signal(ready);
    wait(finish[my_barber]);
    leave_chair();
    pay();
    signal(payment);
    wait(recpt);
    exit_shop();
    signal(max_cust);
}

```

8.3 Mutua esclusione: soluzioni software

Un altro modo per risolvere i problemi semplici di concorrenza è attraverso soluzioni software.

Algoritmo di Dekker

Una soluzione si può ottenere è con l'algoritmo di Dekker (vedi Figura 8.6).

<pre> p0: wants_to_enter[0] ← true while wants_to_enter[1] { if turn ≠ 0 { wants_to_enter[0] ← false while turn ≠ 0 { // busy wait } wants_to_enter[0] ← true } } // critical section ... turn ← 1 wants_to_enter[0] ← false // remainder section </pre>	<pre> p1: wants_to_enter[1] ← true while wants_to_enter[0] { if turn ≠ 1 { wants_to_enter[1] ← false while turn ≠ 1 { // busy wait } wants_to_enter[1] ← true } } // critical section ... turn ← 0 wants_to_enter[1] ← false // remainder section </pre>
---	---

Figura 8.6: Algoritmo di Dekker

Questo algoritmo vale solo per due processi con i flag inizializzati a 0 e turn che può assumere 0 o 1. Grazie a questo garantisce la non-starvation e il non-deadlock, inoltre non richiede supporto dal sistema operativo.

Algoritmo di Peterson

L'algoritmo di Peterson risolve il problema in maniera più semplice garantendo gli stessi vantaggi visti per l'algoritmo di Dekker (vedi Figura 8.7).

```

boolean flag [2];
int turn;
void P0()
{
  while (true) {
    flag [0] = true;
    turn = 1;
    while (flag [1] && turn == 1) /* do nothing */;
    /* critical section */;
    flag [0] = false;
    /* remainder */;
  }
}
void P1()
{
  while (true) {
    flag [1] = true;
    turn = 0;
    while (flag [0] && turn == 0) /* do nothing */;
    /* critical section */;
    flag [1] = false;
    /* remainder */;
  }
}
void main()
{
  flag [0] = false;
  flag [1] = false;
  parbegin (P0, P1);
}

```

Figura 8.7: Algoritmo di Peterson

8.4 Passaggio di messaggi

Quando un processo interagisce con un altro, deve soddisfare due requisiti: sincronizzazione (con la mutua esclusione) e la comunicazione. Lo scambio di messaggi (*message passing*) è una soluzione alla comunicazione utilizzabile sia con memoria condivisa che distribuita. Le operazioni fondamentali per lo scambio di messaggi sono: mando il messaggio, `send(destination, message)`, e ricevo il messaggio, `receive(source, message)`; spesso, oltre a queste due, c'è il test di ricezione che controlla solo se c'è un messaggio da ricevere.

Sincronizzazione

La comunicazione richiede anche la sincronizzazione, infatti, il mittente deve inviare prima che il ricevente riceva e sia mittente che destinatario possono essere bloccanti o non bloccanti:

- con Send e Receive bloccanti, sia il mittente che il destinatario sono bloccati finché il messaggio non è completamente ricevuto (tipicamente viene chiamato *rendezvous*);
- con Send non bloccante (`nbsend`) e Receive bloccante, il mittente non viene mai messo in blocked e continua con le sue operazioni e il destinatario rimane bloccato finché non ha ricevuto il messaggio;
- con Receive non bloccante (`nbreceive`), se il messaggio è presente viene ricevuto, altrimenti si va avanti. Si utilizza un bit dentro il messaggio per dire se la ricezione è avvenuta oppure no.

Tutte queste operazioni sono rese atomiche dal sistema operativo, ovvero un solo processo per volta le esegue finché non viene completata, oppure finché il processo non viene bloccato.

Indirizzamento

Il mittente e il destinatario devono poter dire a quale processo (o quali processi) voler mandare/ricevere il messaggio. Per questo scopo si possono usare l'indirizzamento diretto oppure indiretto.

Con l'indirizzamento diretto l'operazione di send include uno specifico identificatore per il destinatario, o per un gruppo di destinatari. Si può utilizzare lo stesso approccio per la receive in modo da ricevere solo se il mittente coincide con il sender, oppure si può ricevere da chiunque. Ogni processo ha una sua coda, e una volta piena, solitamente il messaggio si perde o viene ritrasmesso.

Con l'indirizzamento indiretto i messaggi sono inviati ad una casella di posta (*mailbox*) condivisa che viene esplicitamente creata da un qualche processo. Il mittente manda messaggi alla mailbox, da dove il destinatario se li va a prendere e se la ricezione è bloccante, e ci sono più processi in attesa su una ricezione, un solo processo viene svegliato.

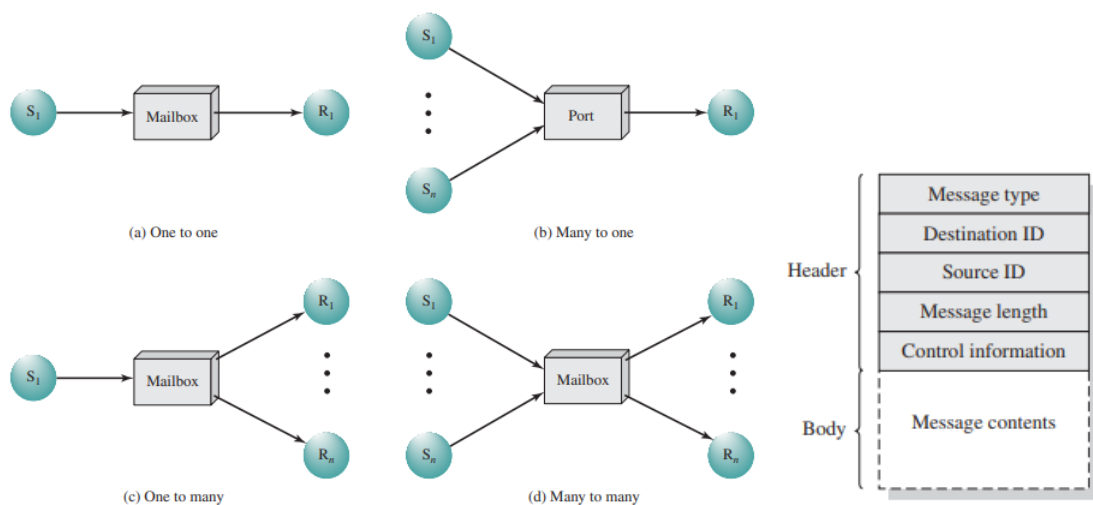


Figura 8.8: Tipi di comunicazione indiretta e formato dei messaggi

Mutua esclusione

Possiamo utilizzare i messaggi per la mutua esclusione:

```

const message null = /* null message */;
mailbox box;
void P(int i) {
    message msg;
    while(true) {
        receive(box, msg);
        /* critical section */;
        nbsend(box, msg) ;
        /* remainder */;
    } }

void main() {
    box = create_mailbox() ;
    nbsend(box, null) ;
    parbegin(P(1), P(2), ..., P(n));
}

```

Si può risolvere anche il problema del produttore/consumatore con i messaggi nel seguente modo:

```

const int capacity = /* buffering capacity */ ;
mailbox mayproduce, mayconsume ;
const message null = /* null message */;

void main ()
{
    mayproduce = create_mailbox(),
    mayconsume = create_mailbox();
    for(int i=1; i<= capacity; i++)
        nbsend(mayproduce, null);
    parbegin(producer, consumer);
}

void producer() {
    message pmsg;
    while(true) {
        receive(mayproduce, pmsg);
        pmsg = produce();
        nbsend(mayconsume, pmsg);
    } }

void consumer() {
    message cmsg;
    while(true) {
        receive(mayconsume, cmsg);
        consume(cmsg);
        nbsend(mayproduce, null);
    } }

```

8.5 Problema dei lettori/scrittori

Un problema che si può generare con la mutua esclusione è quello dei lettori/scrittori che nasce quando un'area dati è condivisa tra molti processi, dove alcuni la leggono e altri la scrivono. Le condizioni da soddisfare sono: più lettori possono leggere l'area contemporaneamente; solo uno scrittore può scrivere nell'area; se uno scrittore è all'opera sull'area, nessun lettore può effettuare letture. La differenza con il problema dei produttori/consumatori è che all'area condivisa si accede

per intero, quindi si eliminano i problemi di buffer pieno o vuoto ma bisogna permettere ai lettori di accedere contemporaneamente.

Una prima soluzione si può fare gestendo la priorità ai lettori o agli scrittori come mostrato in Figura 8.9.

```

/* program readersandwriters */
int readcount;
semaphore x = 1, wsem = 1;
void reader()
{
    while (true){
        semWait (x);
        readcount++;
        if(readcount == 1)
            semWait (wsem);
        semSignal (x);
        READUNIT();
        semWait (x);
        readcount--;
        if(readcount == 0)
            semSignal (wsem);
        semSignal (x);
    }
}
void writer()
{
    while (true){
        semWait (wsem);
        WRITEUNIT();
        semSignal (wsem);
    }
}
void main()
{
    readcount = 0;
    parbegin (reader,writer);
}

/* program readersandwriters */
int readcount, writecount;
void reader()
{
    while (true){
        semWait (z);
        semWait (rsem);
        semWait (x);
        readcount++;
        if (readcount == 1)
            semWait (wsem);
        semSignal (x);
        semSignal (rsem);
        semSignal (z);
        READUNIT();
        semWait (x);
        readcount--;
        if (readcount == 0) semSignal (wsem);
        semSignal (x);
    }
}
void writer ()
{
    while (true){
        semWait (y);
        writecount++;
        if (writecount == 1)
            semWait (rsem);
        semSignal (y);
        semWait (wsem);
        WRITEUNIT();
        semSignal (wsem);
        semWait (y);
        writecount--;
        if (writecount == 0) semSignal (rsem);
        semSignal (y);
    }
}
void main()
{
    readcount = writecount = 0;
    parbegin (reader, writer);
}

```

Figura 8.9: Precedenza ai lettori e precedenza agli scrittori

E' possibile anche utilizzare i messaggi per risolvere il problema dei lettori/scrittori:

```

void reader(int i)
{
    while(true) {
        nbsend(readrequest, null);
        receive(controller_pid, null);
        READUNIT();
        nbsend(finished, null);
    }
}

void writer(int j)
{
    while(true) {
        nbsend(writerequest, null);

```

```

    receive(controller_pid , null);
    WRITEUNIT();
    nbSEND(finished , null);
} }

void controller () {
    int count = MAX_READERS;
    while(true) {
        if(count > 0) {
            if(!empty(finished)) { /* da reader ! */
                receive(finished , msg);
                count ++;
            }
            else if(!empty(writerequest)) {
                receive(writerequest , msg);
                writer_id = msg.sender;
                count = count - MAX_READERS;
            }
            else if(!empty(readrequest)) {
                receive(readrequest , msg);
                count --;
                nbSEND(msg.sender , "OK");
            }
        }
        if(count == 0) {
            nbSEND(writer_id , "OK");
            receive(finished , msg); /* da writer ! */
            count = MAX_READERS;
        }
        while(count < 0) {
            receive(finished , msg); /* da reader ! */
            count ++;
        } /* while (count < 0) */
    } /* while (true) */
} /* controller */

```

8.6 Equivalenze

La condivisione di risorse può quindi essere implementata con tre metodi diversi: istruzioni hardware, sincronizzazione (semafori) e message passing. Se si può implementare un'applicazione con uno qualsiasi dei tre metodi, allora lo si può fare anche con gli altri due. Un particolare meccanismo potrà rivelarsi più conveniente degli altri, in termini di facilità di sviluppo, di prestazioni, e di gestione.

Capitolo 9

Deadlock

Il deadlock è un blocco permanente di un insieme di processi, che o competono per delle risorse di sistema o comunicano tra loro. Il motivo di base per la nascita di un deadlock è la richiesta contemporanea delle stesse risorse da parte di due o più processi.

Le situazioni di deadlock possono essere raffigurate con un *Joint Progress Diagram* come in Figura 9.1.

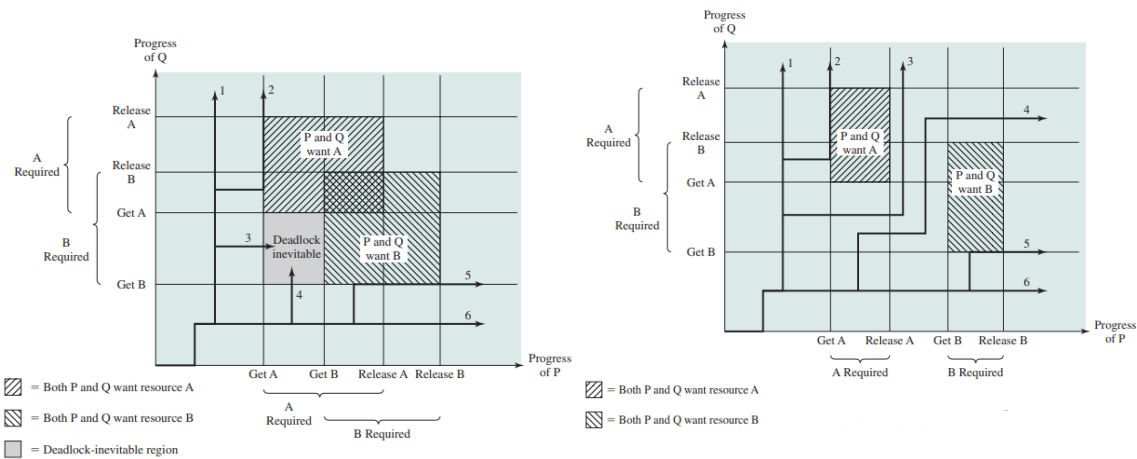


Figura 9.1: Joint Progress Diagram

Nel primo diagramma, nel momento in cui i processi P e Q chiedono una risorsa, prima di rilasciarla ne richiedono un'altra. Se il dispatcher manda in esecuzione P e Q prima di rilasciare le risorse si avrà sicuramente deadlock.

Nel secondo diagramma il processo P richiede una risorsa e la rilascia prima di chiederne un'altra. In questo modo non potrà mai verificarsi il deadlock.

9.1 Risorse

Facciamo una distinzione tra le risorse utilizzate dai processi perché cambiano la definizione di deadlock.

Risorse riusabili

Le risorse riusabili (*reusable resources*) sono usabili da un solo processo alla volta e non vengono consumate (ad esempio processori, I/O channels, memoria primaria e secondaria, dispositivi, file, database, semafori). Una volta che un processo ottiene una risorsa riusabile, prima o poi la rilascia cosicché altri processi possano usarla a loro volta. Lo stallo può esistere solo se un processo ha una risorsa e ne richiede un'altra.

Esempio 9.1.1 – Deadlock con risorse riusabili

Supponiamo di avere due processi *P* e *Q* che richiedono e rilasciano le risorse *D* e *T*.

Step	Process P Action	Step	Process Q Action
p ₀	Request (D)	q ₀	Request (T)
p ₁	Lock (D)	q ₁	Lock (T)
p ₂	Request (T)	q ₂	Request (D)
p ₃	Lock (T)	q ₃	Lock (D)
p ₄	Perform function	q ₄	Perform function
p ₅	Unlock (D)	q ₅	Unlock (T)
p ₆	Unlock (T)	q ₆	Unlock (D)

Notiamo che se il dispatcher esegue *P* fino a *p*₁ e poi passa a *Q* otteniamo un deadlock perché le risorse *T* in *P* e *D* in *Q* non possono essere più richieste.

Esempio 9.1.2 – Deadlock con risorse riusabili

Supponiamo che ci siano 200 KB di memoria disponibili, e che ci sia la seguente sequenza di richieste:

P1	P2
...	...
Request 80 Kbytes;	Request 70 Kbytes;
...	...
Request 60 Kbytes;	Request 80 Kbytes;

Notiamo che se il dispatcher esegue la prima richiesta di *P*₁ e poi passa a *P*₂ otteniamo un deadlock quando uno dei processi farà la seconda richiesta, perché in tutte e due i casi si supera lo spazio di memoria disponibile.

Risorse consumabili

Le risorse consumabili (*consumable resources*) vengono create (prodotte) e distrutte (consumate) (ad esempio interruzioni, segnali, messaggi, informazione nei buffer di I/O). Il deadlock viene generato se si fa una richiesta (bloccante) di una risorsa ancora non creata, ad esempio, un deadlock può avvenire su una ricezione bloccante.

Esempio 9.1.3 – Deadlock con risorse consumabili

Supponiamo che i processi facciano le seguenti richieste:

P1	P2
...	...
Receive (P2);	Receive (P1);
...	...
Send (P2, M1);	Send (P1, M2);

Nel momento in cui il dispatcher esegue la prima richiesta di *P*₁ e poi passa a *P*₂ otteniamo un deadlock perché non potrà essere esaudita né la Send di *P*₁ che quella di *P*₂.

Grafo dell'allocazione delle risorse

Il grafo dell'allocazione delle risorse è un grafo diretto che rappresenta lo stato di processi e risorse rappresentati rispettivamente come nodi tondi e quadrati. Con le risorse riusabili si indica con una freccia (a) per la richiesta e al contrario (b) quando la richiesta è accordata (*held*) (vedi Figura 9.2); mentre per le risorse consumabili non esiste l'*Held* e i pallini compaiono e scompaiono.

Le condizioni per il deadlock con le risorse riusabili/consumabili sono:

- *Mutua esclusione*: quando un solo processo alla volta può usare una data risorsa;
- Richiesta di una risorsa quando se ne ha già una (*hold-and-wait*): quando un processo può richiedere risorse mentre ne ha già bloccate altre/si può richiedere in modo bloccante una risorsa quando ancora non è stata creata;

- *Manca di preemption* per le risorse: quando non si può sottrarre una risorsa ad un processo senza che quest'ultimo la rilasci;
- *Attesa circolare*: quando esiste una catena chiusa di processi, in cui ciascun processo detiene una risorsa che è richiesta (invano) dal processo che lo segue nella catena / quando esiste una catena chiusa di processi, in cui ciascun processo dovrebbe creare una risorsa che è richiesta (invano) dal processo che lo segue nella catena.

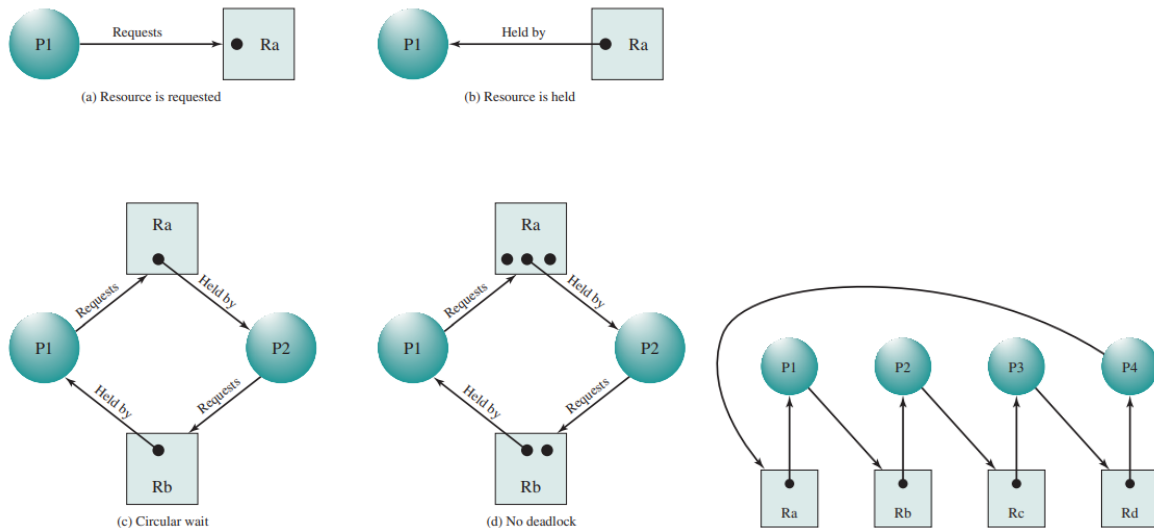


Figura 9.2: Grafi dell'allocazione delle risorse

L'esistenza vera e propria del deadlock si ha quando alle tre condizioni: mutua esclusione, richiesta di una risorsa quando se ne ha già una (*hold-and-wait*) e mancanza di preemption per le risorse (che dipendono dalla struttura del sistema operativo) si aggiunge l'attesa circolare.

9.2 Prevenzione del deadlock

Per prevenire il deadlock si cerca di far sì che una delle quattro condizioni sia sempre falsa. Ma non c'è modo per evitare la mutua esclusione, per l'*hold-and-wait* si impone ad un processo di richiedere tutte le sue risorse in una volta, per la mancanza di preemption per le risorse il sistema operativo può richiedere ad un processo di rilasciare le sue risorse e per l'attesa circolare si definisce un ordinamento crescente delle risorse, ovvero una risorsa viene data solo se segue quelle che il processo già detiene.

9.3 Evitare il deadlock

Per evitare il deadlock occorre decidere se l'attuale richiesta di una risorsa può portare ad un deadlock, se esaudita. Questo richiede la conoscenza delle richieste future e ci sono due possibilità: o non mandare in esecuzione un processo se le sue richieste possono portare a deadlock oppure non concedere una risorsa ad un processo se allocarla può portare a deadlock (algoritmo del banchiere).

Algoritmo del banchiere

L'algoritmo del banchiere è valido per le risorse riusabili e fa sì che il sistema proceda da uno stato non sicuro (*unsafe* - il cammino porta a un deadlock) ad uno sicuro (*safe* - il cammino non porta a un deadlock).

Le strutture dati utilizzate per questo algoritmo sono in Figura 9.3.

Resource = $\mathbf{R} = (R_1, R_2, \dots, R_m)$	Total amount of each resource in the system
Available = $\mathbf{V} = (V_1, V_2, \dots, V_m)$	Total amount of each resource not allocated to any process
Claim = $\mathbf{C} = \begin{pmatrix} C_{11} & C_{12} & \dots & C_{1m} \\ C_{21} & C_{22} & \dots & C_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ C_{n1} & C_{n2} & \dots & C_{nm} \end{pmatrix}$	C_{ij} = requirement of process i for resource j
Allocation = $\mathbf{A} = \begin{pmatrix} A_{11} & A_{12} & \dots & A_{1m} \\ A_{21} & A_{22} & \dots & A_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ A_{n1} & A_{n2} & \dots & A_{nm} \end{pmatrix}$	A_{ij} = current allocation to process i of resource j

Figura 9.3: Strutture dati dell'algoritmo del banchiere

Esempio 9.3.1 – Stato sicuro

Il seguente stato è sicuro perché esiste almeno un cammino che non porta a un deadlock:

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix C

	R1	R2	R3
P1	1	0	0
P2	6	1	2
P3	2	1	1
P4	0	0	2

Allocation matrix A

	R1	R2	R3
P1	2	2	2
P2	0	0	1
P3	1	0	3
P4	4	2	0

C - A

R1	R2	R3
9	3	6

Resource vector R

R1	R2	R3
0	1	1

Available vector V

(a) Initial state

	R1	R2	R3
P1	3	2	2
P2	0	0	0
P3	3	1	4
P4	4	2	2

Claim matrix C

	R1	R2	R3
P1	1	0	0
P2	0	0	0
P3	2	1	1
P4	0	0	2

Allocation matrix A

	R1	R2	R3
P1	2	2	2
P2	0	0	0
P3	1	0	3
P4	4	2	0

C - A

R1	R2	R3
9	3	6

Resource vector R

R1	R2	R3
6	2	3

Available vector V

(b) P2 runs to completion

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	3	1	4
P4	4	2	2

Claim matrix C

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	2	1	1
P4	0	0	2

Allocation matrix A

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	1	0	3
P4	4	2	0

C - A

R1	R2	R3
9	3	6

Resource vector R

R1	R2	R3
7	2	3

Available vector V

(c) P1 runs to completion

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	4	2	2

Claim matrix C

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	0	0	2

Allocation matrix A

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	4	2	0

C - A

R1	R2	R3
9	3	6

Resource vector R

R1	R2	R3
9	3	4

Available vector V

(d) P3 runs to completion

Esempio 9.3.2 – Stato non sicuro

Il seguente stato non è sicuro perché nessuno dei quattro processi può essere selezionato per andare in esecuzione:

	R1	R2	R3		R1	R2	R3		R1	R2	R3
P1	3	2	2	P1	1	0	0	P1	2	2	2
P2	6	1	3	P2	5	1	1	P2	1	0	2
P3	3	1	4	P3	2	1	1	P3	1	0	3
P4	4	2	2	P4	0	0	2	P4	4	2	0
Claim matrix C				Allocation matrix A				C - A			
Resource vector R				Available vector V							
9 3 6				1 1 2							

(a) Initial state

	R1	R2	R3		R1	R2	R3		R1	R2	R3
P1	3	2	2	P1	2	0	1	P1	1	2	1
P2	6	1	3	P2	5	1	1	P2	1	0	2
P3	3	1	4	P3	2	1	1	P3	1	0	3
P4	4	2	2	P4	0	0	2	P4	4	2	0
Claim matrix C				Allocation matrix A				C - A			
Resource vector R				Available vector V							
9 3 6				0 1 1							

(b) P1 requests one unit each of R1 and R3

Lo pseudocodice dell'algoritmo è il seguente:

```

/* Global data structures */
struct state {
    int resource[m];
    int available[m];
    int claim[n][m];
    int alloc[n][m];
}

/* Resource alloc algorithm */
if (alloc[i,*] + request[*] > claim[i,*])
    <error>; /* total request > claim */
else if (request[*] > available[*])
    <suspend process>;
else { /* simulate alloc */
    <define newstate by:
        alloc[i,*] = alloc[i,*] + request[*];
        available[*] = available[*] - request[*]>;
    }
    if (safe(newstate))
        <carry out allocation>;
    else {
        <restore original state>;
        <suspend process>;
    }
}

/* Test for safety algorithm */
boolean safe (state S) {
    int currentavail[m];
    process rest[<number of processes>];
    currentavail = available;
    rest = {all processes};
    possible = true;
    while (possible) {
        <find a process Pk in rest such that
            claim[k,*] - alloc[k,*] <= currentavail;

```

```

    if (found) { /* simulate execution of Pk */
        currentavail = currentavail + alloc [k,*];
        rest = rest - {Pk};
    }
    else possible = false;
}
return (rest == null);
}

```

Con questo algoritmo occorre dichiarare il massimo uso di risorse; i processi devono essere indipendenti senza requisiti di sincronizzazione ovvero, devono essere liberi di andare in esecuzione in un qualsiasi ordine, altrimenti, non si può simulare l'esecuzione fino al completamento; ci deve essere un numero fissato di risorse da allocare; nessun processo deve terminare senza rilasciare le sue risorse.

9.4 Rilevare il deadlock

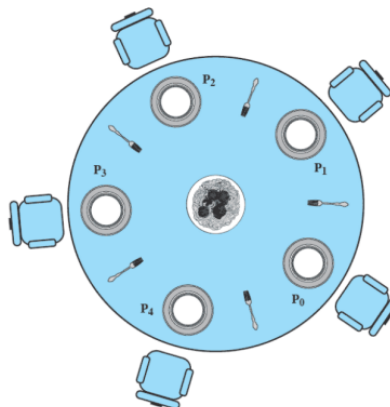
Rilevare il deadlock significa che il sistema operativo ogni tanto vede se si è verificato, e o notifica all'utente oppure prende delle decisioni per rimuoverlo. Per effettuare la rilevazione del deadlock si utilizza uno pseudocodice con le stesse strutture dati del banchiere, ma la claim matrix è sostituita da Q:

1. marca tutti i processi che non hanno allocato nulla
2. $w \leftarrow V$
3. sia i un processo non marcato t.c. $Q_{ik} \leq w_k \forall 1 \leq k \leq m$
4. se i non esiste, vai al passo 6
5. marca i e aggiorna $w \leftarrow w + A_i$; poi torna al passo 3
6. c'è un deadlock se e solo se esiste un processo non marcato

Una volta rilevato il deadlock si può: terminare forzosamente tutti i processi coinvolti nel deadlock; terminare forzosamente i processi coinvolti nel deadlock uno ad uno, finché lo stallo non c'è più; sottrarre forzosamente risorse ai processi coinvolti nel deadlock uno ad uno, finché lo stallo non c'è più.

Esempio 9.4.1 – Filosofi a cena

Supponiamo di avere 5 filosofi a cena ognuno con un suo piatto e una sua forchetta. I filosofi possono mangiare (ma hanno bisogno sia della forchetta a destra che di quella sinistra) oppure possono pensare. L'obiettivo è quello di far mangiare più filosofi possibili eliminando il deadlock.



Ci sono due soluzioni possibili a questo problema. La prima utilizza i semafori:

```

semaphore fork[N] = {1 , 1 , ... 1};
philosopher(int me) {
    int left , right , first , second;
    left = me;
    right = (me + 1) % N;
    first = right < left ? right : left;
    second = right < left ? left : right;
    while(1) {
        think();
        wait(fork[first]);
        wait(fork[second]);
        eat();
        signal(fork[first]);
        signal(fork[second]);
    }
}

```

Una seconda soluzione utilizza i messaggi:

```

philosopher(int me) {
    int left , right;
    message fork1 , fork2;
    left = me;
    right = (me + 1) % N;
    while(1) {
        think_for_a_random_time();
        if(nbreceive(fork[left] , fork1)) {
            if(nbreceive(fork[right] , fork2)) {
                eat();
                nbsend(fork[left] , fork2);
            }
            nbsend(fork[right] , fork1);
        }
    }
}

```

9.5 Deadlock e Linux

In Linux, come sempre, si ha gestione minimale ma il più possibile efficiente. Quindi se dei processi utente sono scritti male e vanno in deadlock allora saranno tutti bloccati e sta all'utente accorgersene e killarli. Per quanto riguarda il kernel, c'è la prevenzione dell'attesa circolare.

Capitolo 10

Sicurezza nei sistemi operativi

Dal manuale sulla sicurezza informatica del NIST (National Institute of Standards and Technology) la sicurezza è la protezione offerta da un sistema informatico automatico al fine di conservare integrità, disponibilità e confidenzialità delle risorse del sistema stesso.

Gli obiettivi che costituiscono il cuore della sicurezza sono:

- *Integrità*: riferita tipicamente ai dati, che non devono essere modificati senza le dovute autorizzazioni;
- *Confidenzialità*: riferita tipicamente ai dati, che non devono essere letti senza le dovute autorizzazioni;
- *Disponibilità*: riferita tipicamente ai servizi, che devono essere disponibili senza interruzioni;
- *Autenticità*: riferita tipicamente agli utenti, che devono essere chi dichiarano di essere.

L'RFC 2828 descrive quattro conseguenze delle minacce informatiche:

- Accesso non autorizzato (*Unauthorized disclosure*) quando un'entità ottiene l'accesso a dati per i quali non ha autorizzazione;
- Imbroglione (*Deception*) quando un'entità autorizzata riceve dati falsi e pensa siano veri;
- Interruzione (*Disruption*) quando si ha l'impedimento al corretto funzionamento dei servizi;
- Usurpazione (*Usurpation*) quando il sistema viene direttamente controllato da chi non ne ha l'autorizzazione.

Le risorse (o assets) da proteggere sono: hardware, software, dati, linee di comunicazione e reti.

10.1 Autenticazione

Dal punto di vista del sistema operativo un primo passo per garantire meno minacce è l'utilizzo di autenticazione. L'autenticazione si basa su due passi: identificazione e verifica. Determina se un utente è abilitato ad accedere al sistema e i privilegi dell'utente abilitato. Rende possibile il discretionary control access (controllo di accesso discrezionale) ovvero un utente può decidere a quali utenti concedere determinati permessi.

L'autenticazione può essere effettuata in diversi modi:

- con *password*, è il sistema più noto e usato, ed è importante che le password siano memorizzate non in chiaro.
- con *token* che rappresentano oggetti fisici posseduti da un utente per l'autenticazione come memory card o smartcard.
- con *biometrica* come impronta digitale, riconoscimento vocale...

10.2 Controlli di accesso

I controlli di accesso possono essere:

- *Discrezionale* quando un utente può concedere i suoi stessi privilegi ad altri utenti (vedi Figura 10.1);
- *Obbligatorio* quando un utente non può concedere i suoi stessi privilegi ad altri utenti;
- *Basato su ruoli* dove ciascun ruolo deve contenere il minimo insieme di diritti d'accesso per il ruolo stesso e un utente viene assegnato ad un ruolo, che lo abilita ad effettuare le operazioni richieste per quel ruolo (vedi Figura 10.2).

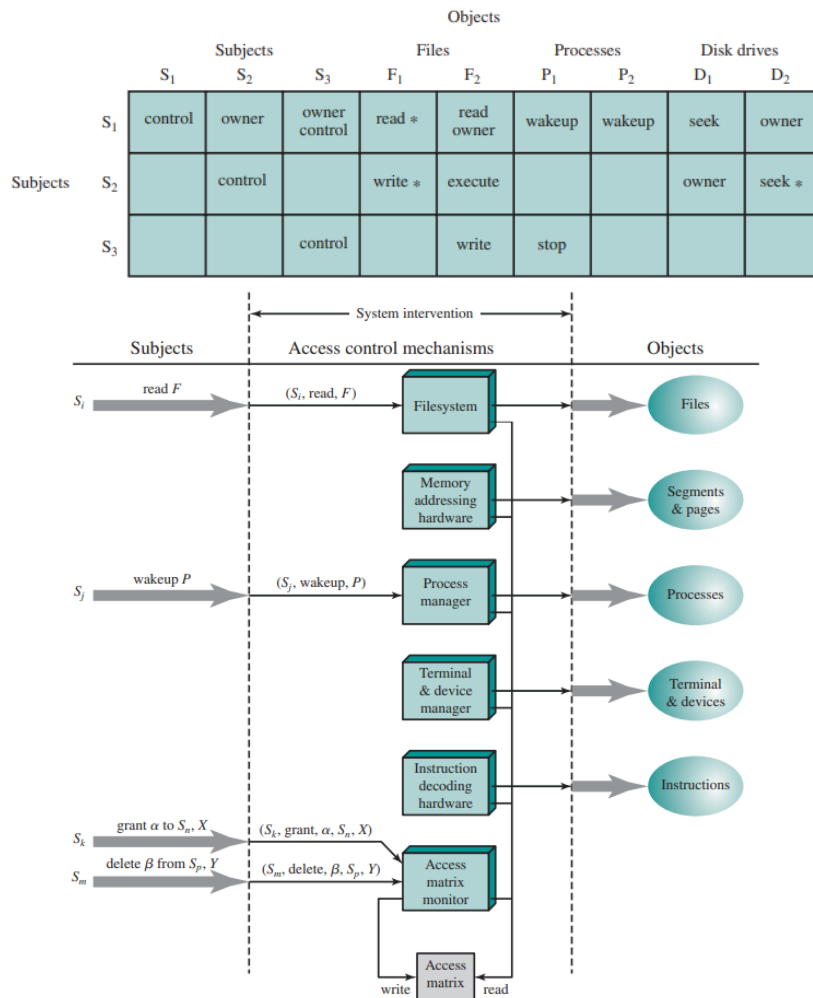


Figura 10.1: Controllo di accesso discrezionale

10.3 Protezione in UNIX

In UNIX la sicurezza è basata sull'autenticazione dell'utente (User-Oriented Access Control) e sui diritti o permessi di accesso ai dati (Data-Oriented Access Control).

Per ogni utente c'è uno username (alfanumerico) e un uid (numerico intero). Lo uid è usato ogni volta che occorre dare un proprietario ad una risorsa e ogni utente appartiene ad un gruppo, analogamente identificato da *groupname* e *gid*. Ci sono infine i file di sistema chiamati */etc/group* e */etc/passwd* che contengono le informazioni su utenti e gruppi.

Il login può essere fatto su un terminale della macchina (processo *getty*) o tramite rete (*telnet*, *ssh*) e richiede una coppia username e password. Se questa corrisponde ad una entry di */etc/passwd*,

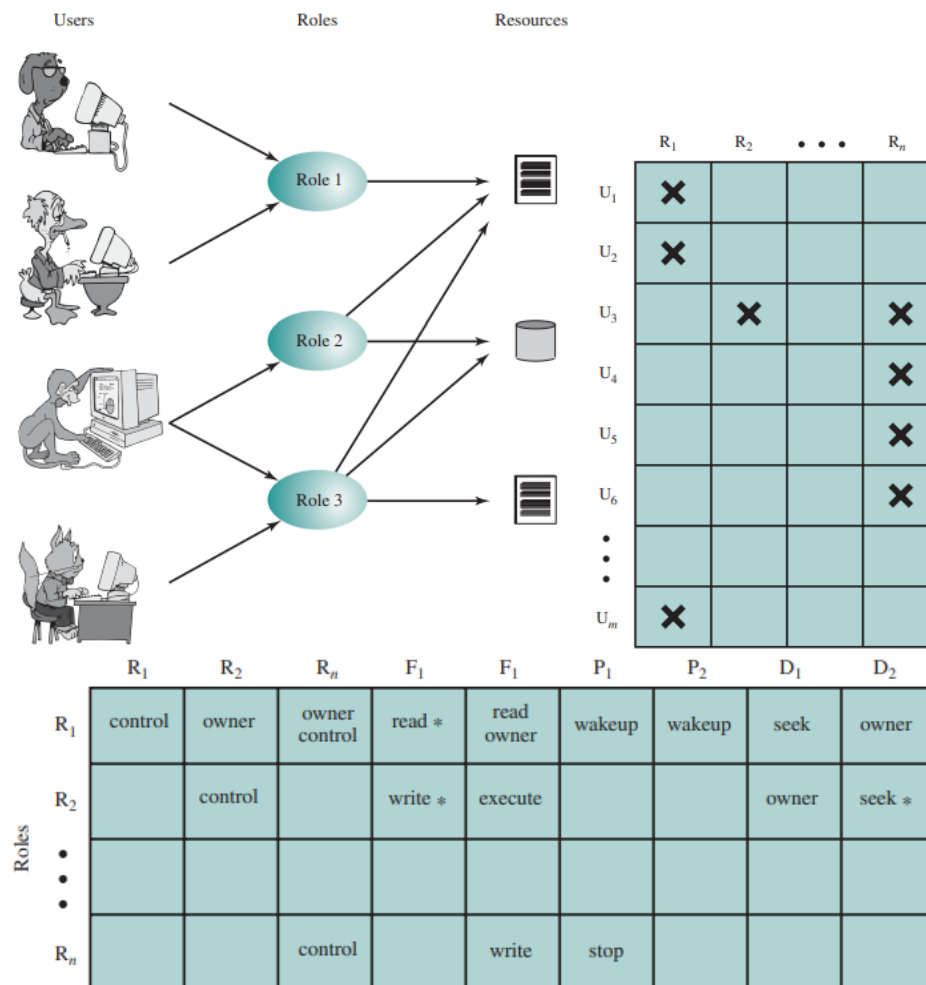


Figura 10.2: Controllo di accesso basato su ruoli

viene eseguita la shell indicata, a partire dalla directory di home indicata. Quando la shell esegue `exit`, o si ritorna al `getty` o si chiude la connessione di rete.

Per ogni file ci sono tre terne di permessi: lettura, scrittura, esecuzione. La prima terna è per il proprietario del file, la seconda per il gruppo cui il proprietario del file appartiene, la terza per tutti gli altri utenti. Le terne di diritti sono usate ogni volta che un processo richiede l'accesso ad un file e se proprietario del file e del processo coincidono, si guarda la prima terna, altrimenti la seconda terna se almeno appartengono allo stesso gruppo, altrimenti la terza terna. Si prende poi l'elemento della terna corrispondente all'accesso richiesto.

10.4 Password

Una password è una stringa di caratteri utilizzata per confermare l'identità di un utente e consentirgli di accedere ad un dispositivo o un servizio.

Password in Linux

Linux usa due file per gestire utenti e le relative password chiamati `/etc/passwd` e `/etc/shadow`. Entrambi sono normali file di testo con una sintassi simile, ma hanno funzioni e permessi diversi infatti per ogni riga (utente) in `passwd`, esiste una corrispondente riga in `shadow` che indica la sua password.

`/etc/passwd` è un plaintext file, contenente l'intera lista di utenti (account) presenti nel sistema che include non solo utenti "normali", ma anche utenti standard di sistema e utenti speciali. Ciascuna

riga del file `passwd` indica informazioni fondamentali su un utente del sistema, ed ha il seguente formato:

- *Username*: nome dell'utente usato per il login. Stringa alfanumerica di max 32 caratteri;
- *Password*: inutilizzato oggi. Il carattere "x" indica che l'hash della password è nel file `shadow`;
- *User ID* (uid): ogni utente nel sistema ha uno user id numerico assegnato;
- *Group ID* (gid): una volta creato ogni utente è assegnato ad un primary group (gruppo primario);
- *GECOS*: campo descrittivo che contiene informazioni generali sull'utente;
- *home directory*: path assoluto alla home directory dell'utente;
- *shell*: path assoluto alla command shell usata dall'utente (eseguibile).

`/etc/shadow` è un plaintext file contenente, per ciascun utente del sistema, l'hash della sua password ed altre informazioni aggiuntive. Data la criticità delle informazioni contenute, sottrarre e decifrare lo `shadow` file spesso è uno degli obiettivi principali di un attaccante, conseguentemente ha permessi molto più restrittivi di `passwd`. Ciascuna riga del file `shadow` contiene informazioni sulla password del rispettivo utente:

- *Username*: nome dell'utente a cui la password appartiene, definito in `passwd`;
- *Password*: password dell'utente salvata usando il Modular Crypt Format;
- *Last changed*: data dell'ultimo cambiamento della password, espresso in giorni trascorsi da Unix Epoch (01/01/1970);
- *Min Age*: minimo numero di giorni dall'ultimo cambio prima che la password possa essere nuovamente cambiata;
- *Max Age*: massimo numero di giorni dopo dei quali è necessario cambiare la password;
- *Warn*: quanti giorni prima della scadenza della password va avvisato l'utente.

Il *Modular Crypt Format* è il formato usato nello `shadow` file per salvare gli hash delle password solitamente nel formato `$ID$salt$hash`. L'ID è l'algoritmo di hashing usato per questa password (MD5, blowfish, SHA256, ...); il salt è usato nel processo di hashing; l'hash della password è calcolato con l'algoritmo ID e salt.

La *hash functions* trasforma input di lunghezza variabile in output di lunghezza fissa in maniera deterministica (data la stessa stringa in input, darà sempre lo stesso output). Inoltre, una funzione hash è detta crittografica se:

- E' computazionalmente difficile calcolare l'inverso della funzione hash;
- E' computazionalmente difficile, dato un input x ed il suo hash d , trovare un altro input x_1 che abbia lo stesso hash d ;
- E' computazionalmente difficile trovare due input diversi di lunghezza arbitraria x_1 e x_2 che abbiano lo stesso hash d .

Attacchi a password

Ma perché non cifrare direttamente la password? In entrambi i casi, le password non sono leggibili da un attaccante: se si usa cifratura ed un attaccante ottiene la chiave, potrebbe decifrare ed ottenere tutte le password in plaintext; le funzioni hash sono one-way ovvero una volta fatto l'hash, non si torna più indietro; se un attaccante ottiene l'hash, non potrà mai scoprire la password che l'ha generato (a parte eccezioni, vedremo poi); è comunque semplice verificare se una password corrisponde a quella salvata in formato hash infatti basta fare l'hash della password e verificarne l'equivalenza (hashing è deterministico). Questo non vuol dire che gli hash di password siano inattaccabili, esistono due tipi di attacchi: attacco dizionario e attacco Rainbow Table.

Per l'attacco *dizionario* il problema nasce quando ci sono un numero infinito di password che possono risultare nello stesso hash ma gli utenti sono pigri e non vogliono (o non possono) ricordarsi password molto complesse e non vogliono ricordarsi molte password. Di conseguenza vengono utilizzate password brevi e semplici e riusate molte volte. E' possibile dunque compilare una lista di password comunemente utilizzate e fare bruteforce. Un attacco dizionario è molto semplice da effettuare (richiede solo una lista di password), è versatile perché funziona per qualsiasi funzione hash... e ci sono moltissimi tool disponibili per automatizzare il tutto. D'altro canto può essere molto lento, in quanto richiede la computazione in real time degli hash e la password può non essere presente nel dizionario.

Con l'attacco *Rainbow Table* le funzioni hash sono deterministiche in quanto data una lista di password ed una funzione hash, l'hash computato sarà sempre lo stesso per ciascuna password. Una Rainbow Table è un dizionario di coppie (valore hash, plaintext password) creato offline e usato per trovare velocemente quale password corrisponde ad un hash. Un attacco Rainbow Table è molto semplice da effettuare, data la rainbow table precompilata ed è molto più veloce rispetto a dictionary bruteforce. Però una Rainbow Table funziona solo per la funzione di hash per la quale è stata creata la rainbow table, se si vogliamo più funzioni, servono più rainbow tables.

Come fare per proteggersi da questi attacchi? Il salt è un valore randomico, generato quando un utente sceglie la password, che viene aggiunto alla computazione dell'hash. Il salt viene poi salvato in chiaro assieme all'hash calcolato e rende impossibile l'uso di rainbow tables se per ogni utente c'è un salt randomico diverso e se due utenti con la stessa identica password avranno due hash diversi, perché molto probabilmente i loro salt saranno diversi.

10.5 Buffer overflow

L'area di memoria di un processo caricato in memoria è diviso in stack, heap, data e text.

Lo stack è costituito da stack frames, dove ciascun frame contiene dei parametri passati alla funzione, variabili locali, indirizzo di ritorno, instruction pointer. Lo stack pointer punta alla cima dello stack (indirizzo più basso) e il frame pointer punta alla base del frame corrente. Vengono aggiunti/salvati sullo stack: i parametri passati alla funzione; l'indirizzo di ritorno; il puntatore allo stack frame; spazio ulteriore per le variabili locali della funzione chiamata (vedi Figura 10.3).

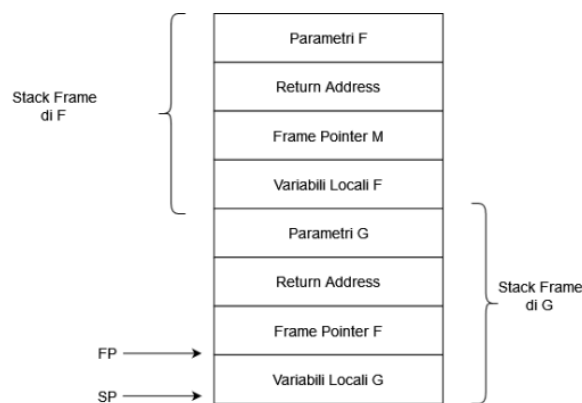


Figura 10.3: Stack di una chiamata a funzione

Ipotizziamo di avere il seguente codice:

```
void foo(char *s) {
    char buf[10];
    strcpy(buf, s);
    printf("buf is %s\n", s);
}
foo("stringatroppolungaperbuf");
```

Stiamo inserendo troppi dati rispetto alla dimensione del buffer, il computer non sa la dimensione di buffer, per lui sono solo indirizzi di memoria... Quindi continua a copiare "stringatroppolunga" a partire da primo indirizzo di memoria di buf[], fino a che non ha occupato tutti gli indirizzi di me-

moria del buffer e poi continua a sovrascrivere qualsiasi cosa trovi, finché non completa l'operazione richiesta.

Di solito, fare overflow di un buffer in questo modo porta alla terminazione del programma. Tuttavia, se i dati che sono usati nell'overflow del buffer sono preparati in modo accurato, è possibile eseguire codice arbitrario. Modificare l'indirizzo di ritorno arbitrariamente è utile, ma come eseguiamo codice arbitrario? Ci sono principalmente quattro modi per farlo:

1. *Shellcode*. E' un piccolo pezzo di codice che viene eseguito quando si sfrutta una vulnerabilità per attaccare un sistema. E' chiamato shellcode, perché solitamente avvia una command shell, dalla quale l'attaccante può prendere il controllo della macchina. L'idea è inserire codice eseguibile arbitrario nel buffer, e cambiare il return address con l'indirizzo del buffer.
2. *Return-to-libc*. Non sempre è possibile inserire shellcode arbitrario, tuttavia, esiste del codice utile ad attacchi, che è sempre presente in RAM ed è sempre raggiungibile dal nostro processo. Invece di usare shellcode, possiamo quindi usare come indirizzo di ritorno l'indirizzo di una funzione di sistema utile per un attacco. L'attacco chiamato return-to-libc perché modifica l'indirizzo di ritorno solitamente con l'indirizzo di una funzione standard della libreria C.

Esistono due tipi di contromisure: difese a tempo di compilazione e difese a tempo di esecuzione.

Nel primo caso si possono usare linguaggi di programmazione e di funzioni sicure (l'overflow è possibile perché in C ci sono alcune funzioni che spostano dati senza limiti di dimensione) o con Stack Smashing Protection, dove il compilatore inserisce del codice per generare un valore casuale (canary) a runtime; il valore canary viene inserito tra il frame pointer e l'indirizzo di ritorno; se il valore canary viene modificato prima che la funzione ritorni, interrompi l'esecuzione perché è stato sovrascritto da un possibile attacco.

Nel secondo caso si possono usare Executable Space Protection, dove se un attaccante cerca di eseguire del codice nello stack (shellcode), il sistema terminerà il processo con un errore, oppure Address Space Layout Randomization dove, ad ogni esecuzione, si randomizzano gli indirizzi dove sono caricati i diversi segmenti del programma in modo che se l'attaccante non sa dove inizia lo stack, è molto più difficile indovinare l'indirizzo del buffer contenente lo shellcode.

Bibliografia

- [1] William Stallings. *Operating Systems: Internals and Design Principles*. 7^a ed. Prentice Hall, 2012.